

CONSTRUYENDO SISTEMAS DISTRIBUIDOS

Iván Abad Andréu

Diseña, Escala y Evoluciona

Construyendo Sistemas Distribuidos

Diseña, Escala y Evolucion

- por Iván Abad Andréu

Derechos de Autor © 2025 por Iván Abad Andréu. Todos los derechos reservados.

Ninguna parte de esta publicación puede ser reproducida, distribuida o transmitida en cualquier forma o por cualquier medio, incluyendo fotocopiado, grabación, o cualquier otro sistema de almacenamiento y recuperación de información, sin el permiso previo por escrito del autor, excepto en el caso de citas breves incorporadas en reseñas críticas o ciertos otros usos no comerciales permitidos por la ley de derechos de autor.

La reproducción no autorizada de este material, en parte o en su totalidad, puede resultar en severas sanciones civiles y penales, y será perseguida hasta el máximo grado posible bajo la ley.

ISBN: 978-84-09-69645-1

Índice

El mundo de los monolitos	11
Microservicios y sistemas distribuidos	13
El uso de microservicios en empresas	14
Comunicación síncrona y asíncrona	17
Vocabulario en los sistemas distribuidos.....	19
¿Qué pasa con el <i>FrontEnd</i> ?.....	20
Coordinación entre equipos.....	22
Separación de los microservicios	26
Escalado de microservicios	33
Escalado vertical	33
Escalado horizontal	35
Auto escalado	37
¿Qué es un Balanceador de carga o Load Balancer?	38
Comunicación Síncrona	45
API REST	46
GraphQL	53
gRPC.....	55
Websockets	59
Comunicación asíncrona	63
El patrón Productor Consumidor	63
Aplicaciones comunes para la comunicación asíncrona	74
La arquitectura de un <i>Event Bus</i>	75
Entrevista de diseño Junior	91
Diseño de un sistema de reportes para usuarios de venta.	91
API Gateway	99
Autenticación y autorización	101
Rate Limiting	107

Diferencia entre API Gateway y un Load Balancer	113
Caché	114
Monitorización de clientes	116
API Composition	121
BackEnd for FrontEnd	124
Componentes clave para la coordinación de servicios	129
Gestión de la configuración	129
Service Registry & Service Discovery	136
Gestión de credenciales	141
Arquitectura de eventos	143
Uso de la base de datos	144
Patrón CQRS	151
Patrón Event Sourcing.....	156
Idempotencia.....	164
Patrón <i>outbox</i> y Consistencia eventual	168
Patrón SAGA.....	172
Entrevista de diseño nivel medio.....	183
Diseño de un sistema de críticas.....	184
El arte de observar sistemas.....	193
LOGS	193
<i>Health Check</i>	201
Monitorización y alertas	205
Observabilidad.....	213
Diseño de aplicaciones robustas	221
Unificación de respuestas de una API.....	222
<i>Breaking changes</i>	228
Despliegues a producción	235
<i>Feature flags</i>	241

Migración de monolitos a microservicios y sistemas distribuidos ...	246
Introducción a los servicios en la nube	253
<i>Serverless</i>	254
Servicios específicos populares	257
Infraestructura como código	267
Regiones y <i>Availability Zones</i>	270
Entrevista final de diseño	275
Diseño de una aplicación de mensajería instantánea	275

Prefacio

Sobre mí

Soy Iván Abad, autor del libro construyendo sistemas distribuidos, que, tras más de una década de pelearme con todo tipo de problemas, he decidido plasmar mis aprendizajes en un libro. Hace unos años hice un curso sobre sistemas distribuidos en YouTube. Desde el principio, mi idea era sacar un libro junto a ese curso, pero, por motivos personales, esa acción nunca sucedió y, siendo sincero, es la mejor decisión que tomé, ya que este libro es mucho más completo, profesional y lleno de experiencias de lo que el primero hubiera sido.

En mi vida profesional he trabajado en todo, como dato curioso diré que fui guía turístico los fines de semana, mientras trabajaba en mi primer trabajo como programador, para sacarme un dinero extra. Pero, en el mundo de IT he pasado desde QA cuando hice las prácticas a Arquitecto de sistemas en una empresa mediana. Por supuesto *backend*, *frontend* y *full stack* o ingeniero de infraestructura. Pero, mi parte favorita, con diferencia, es la de diseño de sistemas *backend* de alto rendimiento y alta disponibilidad, que, por fin, es lo que vamos a ver aquí en detalle.

Este libro no contiene código, por lo que da igual cual sea tu lenguaje preferido, te puede y te va a servir. Un apartado muy importante como desarrollador *backend* es entender los sistemas distribuidos; ya que la gran mayoría de empresas modernas los utilizan. Comprenderlos te dará una ventaja competitiva y salarial.

Quiero agradecer mi mujer, que es la principal causante de que este libro sea posible. Sin las tareas extra en casa que mi mujer ha estado

realizando mientras yo escribía un libro, no hubiera sido posible, así que un agradecimiento eterno.



Sin olvidarme de todos los que semana a semana ven mi contenido ya bien sea en la web o a través de los vídeos. Últimamente estoy más centrado en arquitectura que en *.NET* o *C#*, lo cual, en relación con este libro, es perfecto.

Contenido del libro

Este libro va dirigido a aquellas personas que ya tienen conocimientos de programación y quieren dar el salto a entender cómo se diseñan y mantienen los sistemas distribuidos. Muchas veces, las decisiones de arquitectura que se toman en proyectos de software pueden parecer confusas o innecesarias hasta que no comprendemos cómo encajan dentro del sistema completo. Estas dudas, si no se resuelven, van a generar frustración en los desarrolladores, ya sea con su trabajo, con su equipo, o, incluso, con la empresa en la que están.

En este libro, explicaré de forma clara y práctica los procesos, patrones y prácticas claves utilizadas en el diseño de sistemas distribuidos, basándome en mis propias experiencias. Creo firmemente que los ejemplos del mundo real aportan un punto de vista extra muy importante a la hora de tomar decisiones. No se trata sólo de entender conceptos teóricos, sino de ver cómo se aplican en escenarios reales, y qué retos surgen al hacerlo.

Uno de los puntos más importantes que abordaremos es que no todos los patrones funcionan en todos los escenarios. Comprender cuándo y por qué utilizar cada patrón puede marcar la diferencia entre un sistema eficiente y uno lleno de problemas. A lo largo del libro, exploraremos para qué sirve cada elemento de un sistema distribuido, qué ventajas aporta y cuáles son sus limitaciones.

Este libro no sólo te enseñará los conceptos clave para diseñar aplicaciones distribuidas, sino que también te mostrará cómo estas decisiones afectan tanto al código como a la dinámica de los equipos. Entenderás que trabajar en sistemas distribuidos requiere una mentalidad diferente, y cómo esa mentalidad puede ayudarte a evitar problemas comunes y a crear aplicaciones más robustas y escalables.

Al final del libro, tendrás una visión práctica y realista de lo que significa diseñar y mantener sistemas distribuidos, y estarás mejor preparado para enfrentar los desafíos que esto conlleva, ya trabajes en el *backend*, en el *frontend* o en *DevOps*.

Si buscas una implementación de código para un sistema distribuido, ya que puede facilitarte para entender ciertos aspectos, tengo un ejemplo escrito en *C#* en *GitHub*, en el enlace <https://github.com/ElectNewt/Distribt> (no olvides darle una estrellita).



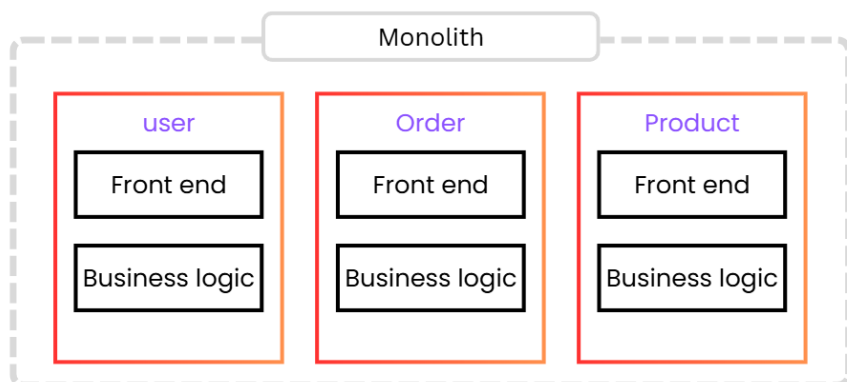
El mundo de los monolitos

Quiero empezar con el elefante en la habitación. Hablemos de si necesitamos o no sistemas distribuidos o microservicios.

Depende mucho del proyecto o sistema donde te encuentres. Si estás haciendo un proyecto de *hobby*, o no solo un proyecto *hobby*, sino empezando un sistema nuevo en una empresa nueva, lo más probable es que no necesites microservicios ni sistemas distribuidos. De hecho, la gran mayoría de las aplicaciones comienzan a ser desarrolladas como monolitos, y es lo mejor que puede suceder, ya que al empezar un sistema lo más previsible es que no necesites microservicios ni sistemas distribuidos.

Un monolito no es más que una aplicación donde todo el código vive junto. Tienes el *backend* con la lógica de negocio, y el *Front End*, que será la parte a la que accedan los usuarios en un único lugar. Y eso no es malo, de hecho, trae ventajas, especialmente al inicio de un proyecto, ya que son más simples de construir y desplegar, hay menos puntos donde una app pueda fallar, son fáciles de entender para un equipo pequeño, y permite iterar rápido, dándole valor a los usuarios.

Lo ideal de los monolitos sería estructurarlos de una forma correcta, en lo que hoy en día llamamos monolito modular. Esto significa que la aplicación tiene módulos independientes, cada uno con su propia responsabilidad, pero el código sigue “viviendo junto”.



Esto permite mantener todo bajo control, como en los monolitos, y con la ventaja de que cada parte está bien separada. Y, en teoría, deberíamos tener una capa de acceso para que otros módulos accedan a la información de dicho módulo.

Desafortunadamente, hay un punto en el que los monolitos empiezan a ser un problema. El más obvio es que los equipos crecen y la gente empieza a trabajar en paralelo pisándose unos a otros, lo que causa problemas con los conflictos. Puede darse el caso de que los ingenieros principales del sistema se marchen de la empresa y esa separación por módulos empiece poco a poco a dejar de respetarse, lo que causa inconvenientes de que el código pierda esa organización ideal, y pase a ser muy difícil de mantener.

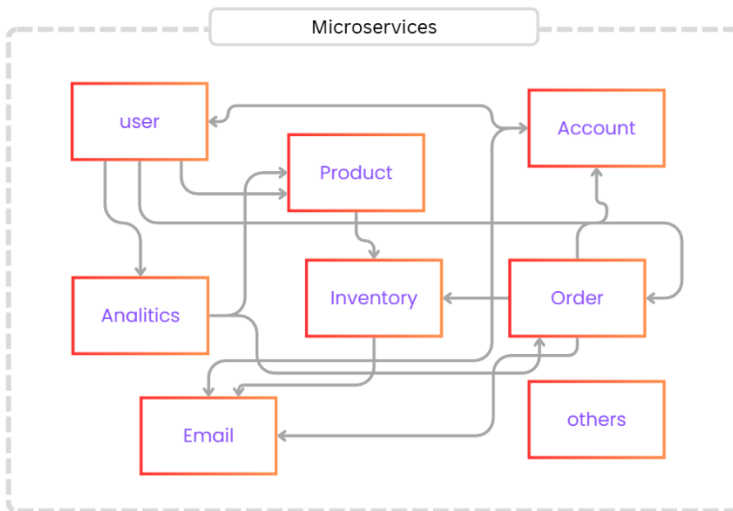
Pero no solo eso, hay otros problemas que quizá no se ven tan fácilmente. El tiempo de despliegue aumenta, porque con cada despliegue tienes que desplegar la aplicación completa, y cada despliegue es un riesgo donde, en teoría, deberíamos testear el sistema entero para verificar que todo funciona correctamente.

Aquí es cuando nos tenemos que plantear la migración a microservicios y a los sistemas distribuidos. Y es lo que vamos a ver en este libro.

Microservicios y sistemas distribuidos

Da igual si has llegado aquí porque tienes que dividir un monolito entre microservicios, estás creando un sistema de microservicios, o estas empezando en un nuevo puesto de trabajo y tienen microservicios. Todos los problemas, situaciones, y soluciones de los que vamos a hablar en este libro son comunes en cada uno de ellos.

Imaginémonos que tenemos un sistema distribuido de una tienda. Para simplificar, no pondremos todos los microservicios necesarios, pero cogeremos un *subset* de los mismos; en este caso, necesitas pedidos (*orders*), productos, usuarios, inventario, analytics, email, entre otros.



Si bien es cierto que no es necesario para el transcurso de este libro, en el mundo real una parte muy muy importante de los sistemas distribuidos y de microservicios es saber cómo se relacionan entre ellos.

Los puristas te pueden contar algún cuento como que “los microservicios son independientes y me da igual lo que haya fuera de él.” “Yo publico mi evento y el consumidor que se apañe.” Si bien es cierto así es como nos lo venden. En el mundo real esto no tiene sentido, porque conocer el dominio es tan importante como conocer la tecnología.

Personalmente, no me he encontrado mucho con este tipo de gente, pero existe, y es mejor no hablar con ellos porque están encerrados en su silo y de ahí no salen.

Por norma general la palabra “microservicio” hace referencia a los propios servicios con la lógica de dominio de la empresa, mientras que cuando hablamos de sistemas distribuidos se hace referencia no solo a los microservicios, sino también a todos los elementos de la infraestructura necesarios para que estos funcionen.

Por supuesto, cada empresa es un mundo y puede usar una terminología diferente, pero, en cualquier caso, si tu empresa usa microservicios, van a necesitar una infraestructura, por lo tanto, nunca es únicamente los propios microservicios con código, sino también todas las aplicaciones que vienen con ellos.

El uso de microservicios en empresas

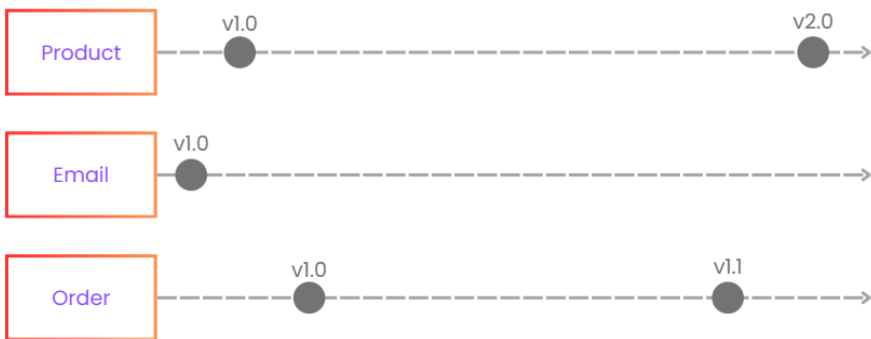
Una arquitectura de microservicios no deja de ser una solución a un problema; tan válida como un monolito pero que cumple o, mejor dicho, que se adapta de forma diferente.

Hay varios motivos principales por los que las empresas suelen utilizar microservicios.

El principal es por la escalabilidad. Te permite configurarlo de tal forma que, si tienes mucha carga en una parte del sistema, esa parte con más carga es la que escala, ya bien sea de forma vertical u horizontal.

El segundo de los motivos es la facilidad para desplegar y administrar las distintas aplicaciones. Me explico. Cuando una empresa crece, empiezas a tener mucha gente tocando el mismo código. Convirtiendo el código a microservicios y asignando microservicios a distintos equipos te aseguras de que los cambios que realiza el equipo A sobre un servicio no se ven afectados por los cambios del equipo B en un servicio diferente.

Esto permite que los despliegues sucedan también de forma individual. Si estamos modificando la funcionalidad para leer productos, no necesitamos desplegar el sistema de email. No tiene sentido. Pero si lo piensas, es lo que ocurre en un monolito, y cuando un sistema es pequeño no pasa nada, pero cuando tienes cientos de miles o incluso millones de usuarios, cuanto más simples sean los cambios, mejor.



En el mundo de los monolitos, se despliega toda una aplicación de golpe, y se suelen hacer, o bien una vez al mes, cada seis meses, o yo he llegado a ver una vez al año.

Ponte en la situación de tener una empresa con 10 o 12 desarrolladores que están trabajando un año en nuevas funcionalidades y todas las desplegamos de golpe el mismo día. Esto lleva a que, durante unos días,

o, incluso semanas, todo el mundo está haciendo lo que se llama test de regresión (*regression testing*), que es testear la aplicación entera una y otra vez para asegurarse de que todo funciona bien. En resumen, una pesadilla.

Pero no es oro todo lo que reluce. Introducir microservicios es muy complejo, necesitamos un sistema robusto y fiable para asegurarnos de que todo funciona bien, e incluso así, no nos podemos fiar al 100%, y necesitaremos una plataforma para la monitorización y observabilidad de dicho sistema, que veremos más adelante.

Un sistema de microservicios bien montado y estructurado nos va a permitir tener un sistema que no deja de funcionar cuando algo falla en un microservicio concreto, además permite tener equipos trabajando de forma individual y teniendo responsabilidad sobre cierto grupo de microservicios. Lo más normal es que el Equipo A se encargue del microservicio de pedidos y productos, entre otros, y el equipo B se encarga del microservicio de usuarios y email, entre otros; siendo la norma común no tocar los microservicios de otro equipo.

Esto hace que los equipos sean expertos en su dominio y el trabajo fluya más rápido.

Al final, cuando evaluamos costes no solo debemos evaluar los costes técnicos de ejecutar una aplicación, sino también el gasto de personal y el impacto que desplegar, o no, ciertas funcionalidades nos pueden traer.

Quiero dar mi opinión sobre lo que siempre se dice de: “los microservicios pueden estar en cualquier lenguaje: puedes tener uno en C#, otro en Java, otro en Python. Porque son independientes.” Desde mi punto de vista, esta frase es muy problemática, porque si bien es cierto, en parte tiene razón, los microservicios son independientes, pero por norma general evita el utilizar múltiples lenguajes sin motivo aparente.

He visto empresas medianas donde hay media docena de lenguajes diferentes dentro de un mismo sistema y siendo mantenidos por el mismo equipo. Esto es porque a alguien se le ocurrió utilizar Go para

hacer unos microservicios, al año se marchó y entró otra persona experta en *JS* y, de repente, todo lo que se hacía era en *Node*, todo cuando el lenguaje principal de la empresa era *C#*.

Utilizar múltiples lenguajes conlleva varios problemas. El primero es tener que saber todos los lenguajes para poder hacer cambios y mantener dichos servicios. El segundo es la arquitectura de los propios lenguajes, ni *Go*, ni *Node*, ni *C#* se estructuran de la misma forma, por lo que cambiar de una aplicación a otra no es intuitivo y lleva más tiempo del que debería.

Comunicación síncrona y asíncrona

Tenemos un sistema que requiere de comunicación entre servicios. Llegados a este punto tenemos dos opciones: comunicarnos de forma asíncrona, o comunicarnos de forma síncrona.

Cuando la comunicación es síncrona lo que tenemos es una llamada directa del servicio A al servicio B. Esta llamada puede ser una llamada HTTP o RPC. Los detalles no importan por ahora. Lo que sí que es muy importante de saber es que es una llamada que espera una respuesta.



Por ejemplo, tenemos nuestro servicio de pedidos donde están ubicados los propios pedidos, junto con los productos, de los cuales únicamente almacenamos el ID. Si queremos mostrar al usuario el nombre del producto, debemos leer del microservicio del producto dichos IDs.

Esto trae consigo varias situaciones que no se ven en el mundo de los monolitos. La más evidente es:

¿Qué sucede cuando un microservicio, del cual dependemos, falla?

Lo más probable es que nuestro servicio también esté fallando. Pero, si, como en el supuesto anterior, tenemos una dependencia directa, que el servicio de productos no funcione correctamente, nos afecta directamente a nosotros.

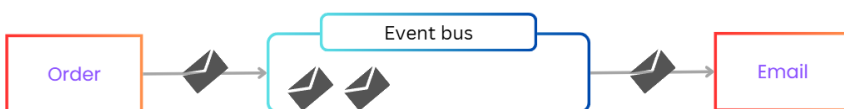
Otro escenario muy común es la latencia.

Entre cada una de las llamadas vamos a tener un tiempo de respuesta, puede ser tan corto como un par de milisegundos o llevarnos varios segundos. La forma en la que programemos nuestras aplicaciones y servicios va a hacer que esta latencia tenga un mayor o menor impacto.

Por ejemplo, en el caso de tener pedidos con 10 productos, podemos, o consultar todos los productos de golpe, o hacerlo de uno en uno. Esta decisión va a tener un efecto en toda la cadena de microservicios dependientes.

En el otro caso, cuando tenemos comunicación asíncrona, lo que hacemos es lanzar un evento del sistema, el cual será leído (o no) por otro servicio, pero quien genera el primer evento se “olvida” de lo que pase después de haberlo creado. Veremos un capítulo en detalle para este apartado

Pero resumiendo, tenemos un servicio que genera un evento, el cual va a un sistema intermediario, y, al otro lado de este sistema intermediario, tenemos otro servicio que reacciona a dicho evento, pero nuestro sistema no espera una respuesta.



Por ejemplo, cuando alguien realiza una compra y le queremos enviar un email lo que hacemos es generar un evento, el cual va al *event/service bus*, y el servicio de email lee dicho evento para, posteriormente, enviar el email.

En la práctica, nos da igual si este email se envía de forma instantánea o tarda unos segundos, o incluso minutos, ya que no esperamos su respuesta.

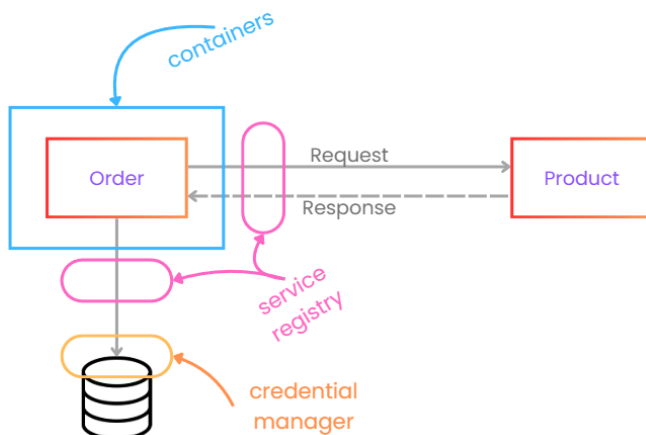
Ambas formas son compatibles, y cubren diferentes situaciones con las que nos podemos enfrentar cuando desarrollamos sistemas.

Vocabulario en los sistemas distribuidos

Los sistemas distribuidos, al contrario que un monolito, son sistemas que están completamente separados. Eso quiere decir que para hacer la acción que hemos visto en el punto anterior de crear un pedido, y enviar un email, necesitamos varios elementos de infraestructura.

El primero de todos es un sistema para poner dichas aplicaciones. Lo más común hoy en día son contenedores, principalmente en *Kubernetes*, aunque si utilizas AWS, tienes EKS, y si usas Azure, tienes AKS, lo que es lo mismo que Kubernetes, pero de los diferentes proveedores.

Una vez tenemos nuestro servicio, debemos poder llamar al otro servicio, y para ello debemos saber su URL. Ya bien sea otro microservicio o un servicio de la infraestructura, necesitamos la URL de donde está ubicado: para ello utilizaremos un sistema de registro de servicios.



Y lo mismo sucede para la gestión de credenciales, si necesitamos acceder a un servicio que requiera de control de acceso, como puede ser la base de datos.

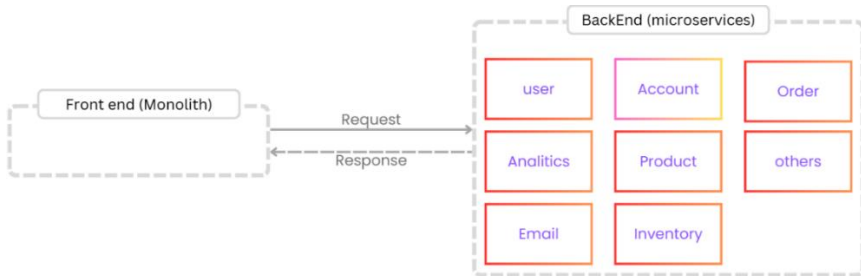
Estos son algunos de los términos que vamos a ver durante este libro, pero no nos vamos a limitar a ellos, ya que hay muchos más.

¿Qué pasa con el *FrontEnd*?

Al hablar de sistemas distribuidos o microservicios, el 100% de las veces nos referimos al *backend* y a la parte de infraestructura; la parte del *FrontEnd* se suele dejar abandonada. La verdad es que no tengo el motivo de tal abandono muy claro, pues lo normal es migrar el *FrontEnd* también.

Está bien, es cierto que el *FrontEnd* se simplifica, mientras que el *backend* se complica, pero, aun así, me parece que hay motivos más que de sobra para meter cierta información, por lo menos para una sección en este libro.

Cuando migramos a microservicios lo que suele pasar es que tenemos nuestra aplicación monolítica con lo que comúnmente se llama “*code behind*”, que es el propio código del monolito, pero ahora, en vez de ir al *code behind*, lo que haremos será una llamada HTTP hacia uno de los microservicios.

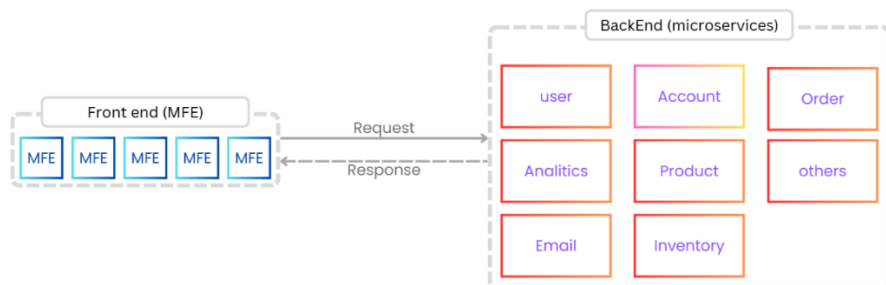


Pero, rápidamente, la empresa se dará cuenta de que el *FrontEnd* también necesita una división. Al fin y al cabo, algunos de los beneficios de los microservicios son separar las diferentes partes del sistema para que sean independientes unos de otros. Esta misma independencia puede ser aplicada de igual forma al *FrontEnd*.

Aquí es donde entran los micro *FrontEnds*, que nos dan esa ventaja de tener independencia unos de otros con lo que ello implica. El cambio en un micro *FrontEnd* no afecta al resto y esto suele ser lo primero en lo que nos damos cuenta, necesitamos librerías o paquetes que contengan los estilos y características comunes de nuestra aplicación, para así poderlos reusar.

En mi experiencia profesional he trabajado con *microfrontends* con React, Angular y Vue y todos siguen el mismo patrón, tenemos un puñado de ficheros JavaScript, que son la propia aplicación “desplegados” de forma estática en S3 o Blob Storage y toda la lógica pasa en el lado del cliente. Esto puede parecer algo “lento” de primeras, pues el usuario tiene que descargar toda la aplicación al visitar la web por primera vez. Pero en el mundo actual, con la velocidad y ancho de banda que tenemos, no suele ser un problema.

Implementar *microfrontends* nos dejará una arquitectura como la siguiente:



Básicamente, esta será la división que se hará en las aplicaciones de la empresa de cara al *FrontEnd*. Si tienes una app móvil, la lógica es la misma; lo más probable es que llame al *BackEnd* de forma directa, o incluso utilizar los propios *microfrontends* desde la app móvil. Pero bueno, eso son discusiones para alguien experto en *frontend* y móvil.

Coordinación entre equipos

Cuando uno trabaja solo, ya bien sea en un proyecto propio, o incluso, si eres el único miembro del equipo de desarrollo en una empresa, la parte más complicada va a ser el código. Lo mismo aplica cuando trabajamos en equipo dentro de un entorno monolítico. El código es la parte más complicada y cuando necesitas realizar un cambio, lo haces y ya está.

Pero, desafortunadamente, esto no es así en los entornos de microservicios, es más, el código suele ser el menor de los problemas. Lo más difícil, o más bien dicho, lo que más frustraciones trae, va a ser la colaboración y, por ende, las dependencias en otros equipos.

Lo primero que debemos saber es qué equipo es el dueño de cada uno de los servicios, y cuál es la funcionalidad de cada uno de ellos.

Idealmente, esta información debería estar representada en la documentación de la empresa, y tendrá una clara representación sobre qué servicios están en cada equipo, sección u organización, ya que depende del tamaño de la empresa puede haber varios niveles.

La información contendrá qué equipos son responsables, y aunque es posible que un equipo participe en varias secciones no suele ser lo habitual, y, por supuesto, un punto de contacto.

Vamos a poner el ejemplo de una tienda, donde tenemos varios equipos, donde cada uno de ellos se encarga de una serie de microservicios.

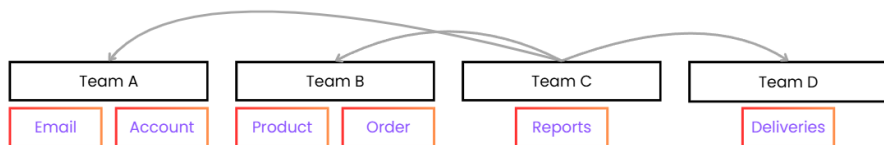
Lo más normal es que los microservicios de los que se encarga cada equipo estén relacionados entre ellos porque son dominios relacionados, y cuanto más contexto e información tengas, mucho mejor; tanto para desarrollar nuevas funcionalidades, como para arreglar cualquier tipo de problema.



Nota: Si utilizas micro *FrontEnd* lo más normal es que el mismo equipo sea dueño tanto del *BackEnd* como del *FontEnd*.

Ahora bien, qué pasa si el Equipo C necesita crear un nuevo reporte al final del día, dónde vamos a leer los envíos que se han creado con los productos y mandarlos a los clientes.

Lo que va a suceder es que el Equipo C va a tener que contactar tanto con el equipo D, como con el B y con el A para poder realizar dicha funcionalidad.



Aquí es donde entra el tener un buen sistema de comunicación y coordinación en la empresa.

Lo primero que debemos hacer es descubrir si cualquiera de los equipos tiene ya disponible una forma de conseguir la información que necesitamos.

Por ejemplo, en el C, necesitamos saber si tienen una API la cual contiene los envíos que se han realizado para cierto cliente en el día anterior, o quizá tengan una API para devolver los envíos realizados que ya contiene el ID del cliente.

Con el equipo B, sabemos que hay una API, la cual nos da los pedidos y los productos, pero debemos saber si hay un API de pedidos; la cual devuelve los nombres de los productos, o de si debemos hacer dicha llamada a los productos nosotros desde nuestro lado.

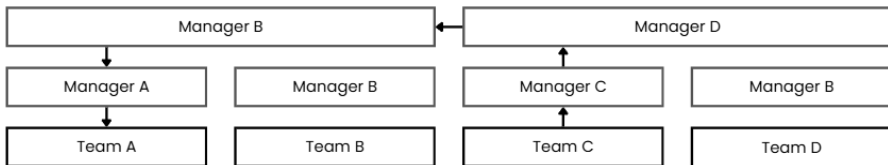
No debemos quedarnos en un “ah, tienen estas dos API, las consultamos y ya.”. El motivo de preguntar es saber cuáles son las mejores prácticas, y lo más probable es que tengan lo que estamos pidiendo y si no lo tienen, al involucrar dos de sus servicios deberían intuir cual es la mejor aplicación en ese caso. Dependiendo cual sea el escenario es posible que puedan aplicar cierta lógica en su lado para que el rendimiento sea mejor.

Finalmente, en el Equipo A, tenemos una situación similar. Dependiendo de cómo el microservicio de *deliveries* esté implementado, deberemos leer todas las cuentas o el ID de la cuenta nos viene en el propio *delivery* y únicamente necesitamos que los emails sean capaces de ser enviados para una cuenta en específico, o tener que leer dicha cuenta y enviar al email de la cuenta. Aquí es importante que el equipo A esté involucrado para que sean ellos quienes decidan si enviar emails para una cuenta, o consultar la cuenta y enviar los emails por el email que nos devuelven.

Aquí es donde vienen las dificultades. Tienes que invertir varios días quedando con los diferentes miembros de los diferentes equipos explicándoles qué es lo que quieres hacer, por qué lo quieres hacer, y el impacto que puede tener en sus servicios.

Es muy importante tener una buena comunicación, porque yo he visto y vivido el que te pongan trabas, el que el equipo C intente contactar con el equipo A y este te diga, “en mi equipo estamos muy ocupados, pero podemos quedar durante media hora dentro de 3 semanas”. Si esta situación sucede, lo que tienes que hacer es ir a tu mánager, y contarle lo que te ha sucedido y este tiene que escalar, posiblemente con su mánager, que es el mánager de otro, que a su vez es el mánager del equipo que te pone trabas.

También es común que un mánager sea el mánager de varios equipos, pero a su vez sea el mánager de otros mánagers. Todo depende del tamaño de la empresa.

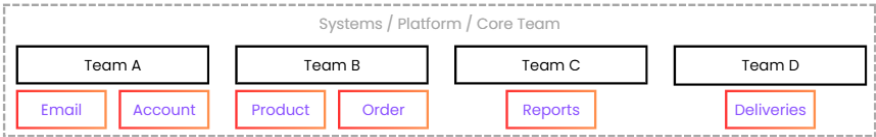


En el supuesto anterior, si estamos creando nuestro reporte y el problema lo tenemos con el equipo que lleva los emails y las cuentas, el mensaje para la prioridad tiene que pasar por cuatro mánagers diferentes.

Por ese motivo suele haber un equipo, o una serie de personas que tienen una funcionalidad multi equipo. Este equipo, en cada empresa, se llama de una forma diferente. En las que yo he trabajado han sido *Systems Team*, *Platform Team*, *Core Team*.

Lo que este equipo hace es estar supervisando todos los proyectos. Un miembro de dicho equipo estará presente en la gran mayoría de reuniones para cada proyecto y son personas técnicas. El motivo es simple, tienen la potestad de meter presión a cada uno de los equipos; no para que realicen cambios, pero sí para facilitar reuniones. Esta persona

es la que se encargaría de saber cómo funcionan cada uno de los microservicios. Y, si dicha persona lleva lo suficiente en la empresa, el Equipo C no tiene ni que preguntar al resto de equipos cómo tienen las funcionalidades implementadas, pues posiblemente esta persona ya lo sepa.



Si estamos en la situación en la que no hay un *endpoint* que nos devuelva todos los pedidos de un día específico, será el propio equipo C, junto con el Equipo D, y la persona del *Systems Team*, quien tendrá una reunión para decidir los pasos a seguir y se acordará un contrato, que el equipo D implementará cuando tenga tiempo, pero el equipo C no estará bloqueado en su progreso de desarrollo a la respuesta del D. Y lo mismo sucederá con cualquier otro equipo que sea necesario.

De esta forma, estos bloqueos tan exagerados no pasan, y no hay tanta guerra entre equipos.

Luego nos queda saber cuándo el resto de los equipos implementarán sus funcionalidades, pero esto es más un tema de prioridades de los jefes de producto de cada una de las áreas que un tema técnico.

Separación de los microservicios

A la hora de migrar a microservicios siempre nos preguntamos, ¿cuán micro es micro? ¿Cuánta funcionalidad debe tener cada uno de nuestros microservicios? Por norma general, esta decisión no solo radica en la parte técnica, sino en la parte de dominio de la aplicación.

En este punto hablamos de qué es lo que queremos que nuestro servicio sea capaz de hacer y dónde ponemos los límites de este. Suele ser una de las partes más difíciles dentro del desarrollo de microservicios. Por norma general, si vienes de un monolito vas a dividir los microservicios en servicios que tengan que ver con las secciones de la interfaz. Y si bien esto no es la situación perfecta, es un comienzo, y es un inicio totalmente válido.

A partir de ahí tenemos múltiples formas o guías para decidir cómo queremos dividir nuestra arquitectura de sistemas.

Separación lógica

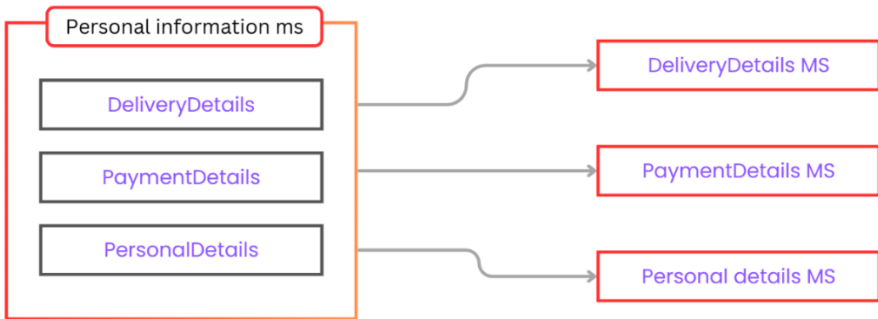
La primera es la separación lógica, y es la forma más común en la que los equipos y empresas separan los microservicios. Básicamente nos basamos en la lógica de negocio para decidir dicha separación.

Por ejemplo, puedes tener un servicio que contenga la lógica de productos y el mismo microservicio contendrá la lógica de los pedidos, quizá incluso le pongas la parte de *analytics* al mismo servicio. Como punto de partida está bien, y este tipo de microservicios existen y se ven en empresas. Pero la realidad es que se convierten en mini-monolitos, o como a mí me gusta llamarlos, microlitos.

En este supuesto, la división es clara, separamos en tres microservicios, uno de productos, uno de pedidos y, finalmente, el de *analytics*, pero no siempre es tan fácil detectar y justificar estas separaciones.

Te pongo un ejemplo donde la separación, pese a ser obvia en términos lógicos y de dominio, puede no ser rentable del todo.

Supongamos que estamos en nuestra tienda y vamos a almacenar la información del envío, la información de pago, y también tenemos un apartado con tu nombre legal y tu número de teléfono, para así poder recuperar tu cuenta si hay un problema y tal.



Si seguimos al pie de la letra la arquitectura de microservicios, deberíamos separar los tres servicios, pues no tienen mucho que ver uno con el otro. Pero, en la práctica, puede no ser la mejor opción, porque el uso de estas funcionalidades puede ser muy bajo, y los costes adicionales de administrar y mantener tres microservicios diferentes son relativamente altos.

Esto lo podemos ver de una forma muy fácil si creamos nuestro microservicio de *personal Information*. Cada uno de esos puntos será un controlador diferente y los cambios de uno no afectarán al otro, pero a la vez están más o menos relacionados en la lógica de nuestro sistema.

Entonces, como puedes observar, no siempre es una solución sencilla, o, mejor dicho, no tenemos una regla que nos sirva para todo. Personalmente, yo tendría un único microservicio, pese a que eso rompa la regla de que los microservicios deban tener una única responsabilidad.

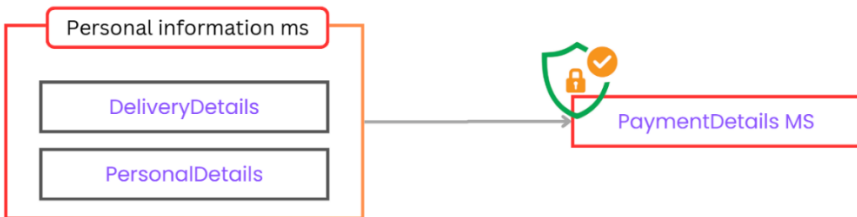
Separación por seguridad

Otro de los motivos por los que podemos justificar una migración a microservicios puede ser por seguridad, tanto para cumplir con leyes, como por el hecho de tener los datos de forma segura.

Un ejemplo muy claro son los datos personales. En muchos países estás obligado a no solo almacenar los datos de los usuarios de ese país en dicho país o región, sino que el proceso de los datos debe estar en dicho país.

Esto se ve sobre todo con temas bancarios, donde se quieren minimizar los riesgos al mínimo.

Si volvemos al escenario de antes, podríamos sacar un microservicio únicamente para los datos de pago y dejar el otro microservicio con los datos personales y los datos de envío.



Escalabilidad y rendimiento

El último de los motivos más populares para dividir un microservicio es por escalabilidad y rendimiento, básicamente el mismo motivo por el que hemos migrado de un monolito a microservicios, pero llevado a un nivel granular mayor.

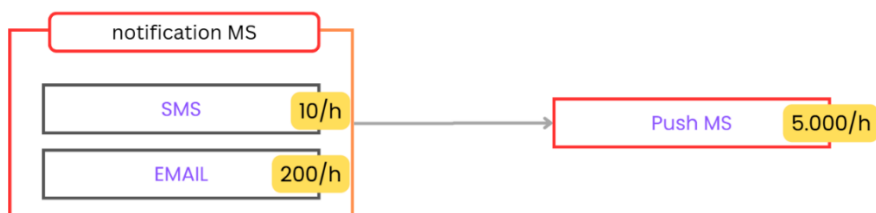
La forma más sencilla de ver este ejemplo es con un ejemplo, en este caso, en vez de ser la tienda vamos a ponernos en el lado del banco. Entiendo que todos los que estáis leyendo esto tenéis la aplicación de vuestro banco en el móvil. Y si os habéis dado cuenta, cada vez que pagáis algo con la tarjeta os llega una notificación a la app del móvil (si la

tenéis configurada, claro). A su vez, si realizáis ciertas acciones en la web, como enviar una transferencia de forma manual os llega un email, y hasta es posible que tengáis los SMS configurados para ciertas acciones.

Esto quiere decir que, dependiendo que acción suceda, vamos a recibir una notificación, ya bien sea un *push* en la app, un SMS, o un email. Por lo que, en el *BackEnd* del banco, tendrán un servicio para hacer dichas acciones.

Pero claro, por lo que hemos mencionado, el número de SMS que van a ser enviados va a ser mínimo, si es que se envía alguno, de hecho, el número de emails va a ser relativamente bajo en comparación al número de notificaciones *push*.

Aquí es donde entra nuestra separación basada en escalabilidad y rendimiento. Sabemos que el número de emails y SMS es bajo y estable durante el día, pero el número de notificaciones *push* es muy elevado. Por lo tanto, en este caso, separamos el servicio de notificaciones *push* de los otros dos:



El microservicio de *push*, no solo necesita gestionar 5000 *push* por hora de media, sino que dependiendo de la hora del día pueden ser 100.000 durante la hora de comer y bajar a mil durante la noche. Por lo tanto, deberá ser capaz de escalar de forma efectiva.

Pero esta división nos abre otro frente, los nombres de los microservicios. En este caso hemos dividido en *notification service* y *push service*, y si bien es cierto que no son terribles, tampoco son ideales, pues el *push* es técnicamente una notificación.

Si dividimos por completo el microservicio de notificaciones para sacar SMS fuera y email fuera podríamos llamarlos *SMS service*, *push service* y *email service*, pero sin tener esa división completa, vamos a tener que utilizar nombres que no son perfectos y pueden causar confusión para aquellos que no están acostumbrados al sistema. Pero como tantas cosas en la informática, no hay una solución perfecta, y tenemos un compromiso en una de las decisiones.

CAPÍTULO 3

Escalado de microservicios

Uno de los motivos principales que hemos mencionado a la hora de migrar a microservicios es el escalado. Pero que nadie te engañe, un monolito desplegado en un servidor puede escalar de la misma forma en la que escala un microservicio corriendo en un contenedor, tanto de forma vertical, como de forma horizontal, lo único que es más costoso en términos monetarios. Pero si estás trabajando en una solución monolítica y necesitas escalar una o dos veces al año, en fechas puntuales, escalar no es un motivo para migrar a microservicios.

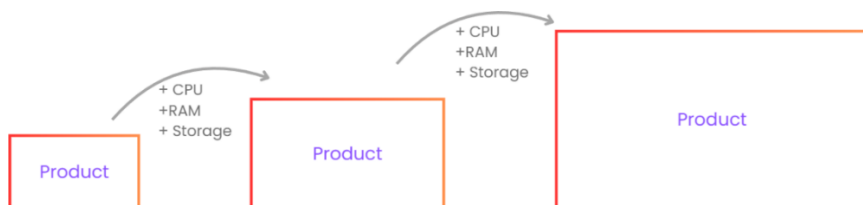
Si bien es cierto hoy en día podemos escalar con instancias que están pensadas para una mayor potencia computacional, memoria, o incluso almacenamiento. En las que nos vamos a centrar es en las que comúnmente se les llama instancias de uso general que van a ser las que nuestros servicios de lógica de dominio van a utilizar.

Escalado vertical

La primera de las opciones de escalado es el escalado vertical, lo que significa poner más recursos a la máquina donde se está ejecutando el código.

Por volver a nuestro caso anterior, tenemos una aplicación de una tienda, y por la noche el uso de la tienda es un 10% al que tenemos durante la hora pico. Eso quiere decir que nuestro sistema funciona con un 10% de los recursos necesarios. Por lo tanto, el escalado vertical significa que vamos a instalar o configurar en nuestro servidor más RAM y más CPU para poder administrar la carga necesaria durante etapas concretas. Y ya está, el escalado vertical no tiene más misterio. Aunque lo importante es saber cuándo tenemos que utilizarlo.

Una forma muy fácil de entenderlo es si tenemos nuestra aplicación en “bare metal” (un servidor físico). Entendemos que si tenemos más carga de la que el servidor puede aguantar, necesitamos un servidor mejor.



Pues aquí lo mismo. Ya tengas un servidor físico, o contenedores, le pondrás más RAM y procesador cuando veas que necesita más, pero OJO, no vas a estar poniendo y quitando RAM todo el día, de hecho, ni una vez al día. Lo ideal es tener una instancia, ya bien sea capaz de aguantar cierto porcentaje de la carga máxima, siempre por encima de la carga mínima (el 10% de por la noche), y combinarlo con el escalado horizontal para encontrar una solución correcta y económica.

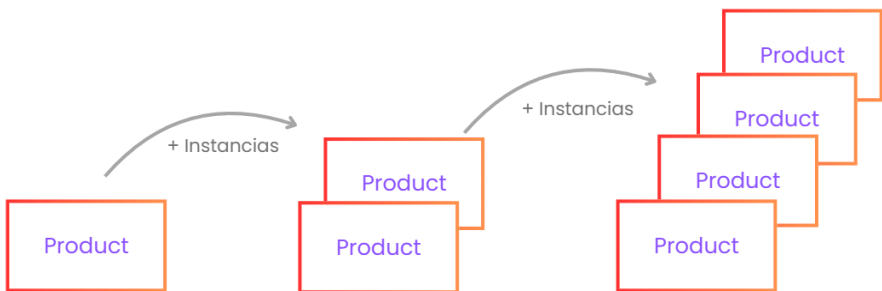
Por eso esta solución es fiable cuando tenemos un monolito que tiene mucha carga durante, por ejemplo, la época navideña. Durante ese mes o dos, podemos tener una única instancia mucho más grande la cual es capaz de gestionar el 100% de la carga sin tener que preocuparnos por el escalado automático, ya que la regla beneficio/coste sería muy desfavorable si tuviéramos que migrar.

Escalado horizontal

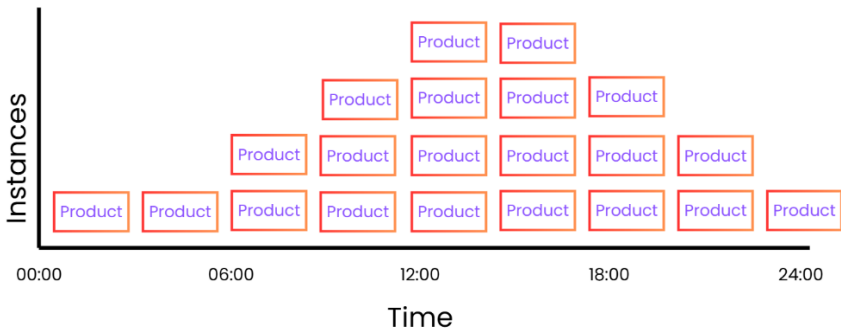
La segunda de las opciones que tenemos a la hora de escalar es el escalado horizontal, lo que significa que vamos a crear más instancias de nuestra aplicación. Esto nos permite ir variando el número de instancias que van a estar disponibles para cubrir la demanda generada.

Como en el caso de antes, si por la noche tenemos una única instancia que es capaz de cubrir el 20% de la demanda pico (recuerda, por la noche hay un 10% de demanda), en las horas pico necesitaremos 5 máquinas para cubrir el 100%.

Desde mi punto de vista y experiencia, el escalado horizontal es el que más se utiliza cuando tenemos microservicios y contenedores, ya que crear una nueva instancia se hace de forma sencilla, mientras que cambiar tamaños de instancias puede ser tedioso y requerir un nuevo despliegue, etc.



Podemos volver al supuesto anterior, donde tenemos el software de una tienda, y por la tarde se realizan muchos más pedidos que por la mañana y que, por supuesto, por la noche. Ello quiere decir que por la noche tendremos un mínimo de instancias, mientras que las iremos aumentando conforme tengamos más carga, para volver a bajar cuando la carga del sistema disminuya.



Sabemos que una de las ventajas de utilizar microservicios con respecto a un monolito es la habilidad de escalar la parte del sistema que necesitamos escalar y nada más.

Si en un monolito escalas de forma horizontal, en las horas pico, estás creando una copia del sistema entero, pero realmente no lo necesitas, lo que necesitas es escalar tu servicio de pedidos, el de productos y alguno más, pero los de *analytics* o usuarios no tienes por qué hacerlo.

Cada situación es un mundo, y *Shopify*, por ejemplo, es un monolito enorme que escala de forma horizontal todo el sistema durante la semana del *Black Friday* o las navidades. Les funciona bien y sin problemas porque son épocas muy concretas con una carga más o menos estable durante el día. Si tuvieras picos de carga, como suele ser lo normal en el software empresarial, lo más recomendable es tener microservicios para poder escalar conforme a dichos picos.

Auto escalado

No podemos hablar del escalado en microservicios sin hablar del auto escalado, ya que, de no ser por él, tendríamos que estar mirando y monitorizando los servidores de forma manual todo el día.

El auto escalado es la habilidad de diferentes servicios de la nube para asignar recursos computacionales. En otras palabras, gracias al auto escalado y la monitorización automática de las instancias podemos dejar que nuestro proveedor de servicios ya bien sea AWS, Azure, o GCP monitorice las instancias actuales y poniendo un poco de configuración pongan ellos dichos recursos.

El auto escalado tiene dos grandes beneficios, en mi opinión.

El primero es que nadie tiene que estar mirando de forma manual si nuestro sistema está siendo utilizado mucho o no; es la propia plataforma la que utiliza distintos parámetros que podemos configurar. Los más comunes son el tráfico, uso de la CPU, o de la Memoria. Con estos datos, el sistema escalará tanto hacia arriba, creando más instancias, como hacia abajo, terminando dichas instancias. Siempre con configuración y especificando un mínimo y un máximo del número de instancias que queremos tener ejecutándose.

Especificar este número de instancias es muy importante, porque cada instancia cuesta dinero, e, inicialmente, puedes pensar que no pones un número límite de instancias por arriba, ya que tienes mucho tráfico, mejor. Pero la realidad es que hacer esto es peligroso ya que, si, por ejemplo, recibes un ataque DDOS o tienes un *bug* en alguna aplicación que está llamando a tu servicio sin parar con mucha carga, puede costarte mucho dinero.

Las ventajas del auto escalado no terminan aquí, ya que el sistema de monitorización del escalado tiene un propio procedimiento que comprobará que las instancias estén funcionando de forma correcta, normalmente por el *health check*. Y en caso de que alguna no lo esté, la reemplazará. Ojo, que puede ser que esté mal porque el código esté roto,

pero asumiendo que no lo está, el problema de una instancia errónea sería temporal.

¿Qué es un Balanceador de carga o Load Balancer?

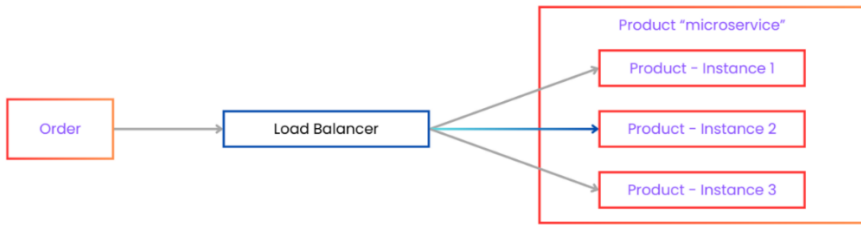
Una parte que no podemos ignorar, si tenemos escalado horizontal, es el Balanceador de carga (*Load Balancer*). Y, la verdad, es un elemento que en las entrevistas de diseño de sistemas está olvidado, no sé muy bien por qué.

El *Load Balancer* es una aplicación, o proceso, que se encarga de distribuir el tráfico de la red a través de diferentes servidores o instancias, con la idea principal de no saturar una instancia en concreto, para así garantizar que la aplicación funciona sin problemas de cara al cliente final.

En términos fáciles de entender, pondremos un *Load Balancer* delante de nuestra instancia, o, mejor dicho, *clúster* de instancias de cada microservicio.

Desde mi experiencia lo mejor es tener un *Load Balancer* por cada microservicio, aunque también se puede hacer con un único *Load Balancer* para todo el sistema.

Por lo tanto, cuando estamos llamando de un servicio a otro, lo que hacemos en realidad es realizar la *request* a un *Load Balancer*, el cual seleccionará la instancia a la que vamos a hacer la llamada. No sin antes comprobar que dicha instancia está funcionando correctamente.



De hecho, dependiendo de cómo y de dónde tengamos la configuración del escalado, el *Load Balancer* es capaz de indicar que se necesitan más instancias de cierto servicio para seguir funcionando.

Esto quiere decir que nuestro microservicio, que en verdad son dos, tres, o N instancias, funciona de forma uniforme de cara a quién consume el servicio, ya que para el consumidor hay un único punto de entrada.

Por lo tanto, cuando hablamos de servicios o microservicios que consumimos en este libro, lo que hacemos es hablar de la combinación entre el *Load Balancer* y las instancias, pues de cara al consumidor es completamente transparente.

Algoritmos de *Load Balancing*

La forma en la que un *Load Balancer* elige a qué servidor va a enviar la *request* se basa en uno de los siguientes algoritmos, y cada uno de ellos cumple con diferentes características y beneficios para cubrir diferentes casos de uso.

A la hora de elegir qué algoritmos queremos elegir tenemos de dos tipos: estáticos y dinámicos.

Los algoritmos estáticos tienen una serie de normas predefinidas, y son *Round Robin*, *Weighted Round Robin*, *Sticky Round Robin* y *IP Hash*.

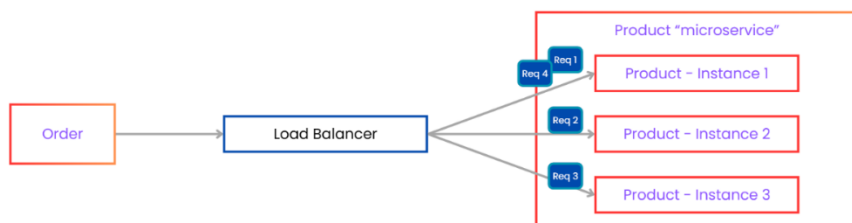
Mientras, los algoritmos dinámicos comprueban el rendimiento de nuestras instancias y las condiciones de estas para decidir a qué

instancia enviar la *request*. Estos son el algoritmo *Least connections* y *Least response time*.

La elección de qué algoritmo es mejor depende de cada caso de uso particular, ya que, por ejemplo, *Round Robin* funciona muy bien para las aplicaciones sin estado, mientras que los algoritmos dinámicos pueden ayudarte a mejorar la latencia o *throughput* del sistema.

Round Robin

Esta es la forma más simple de asignar las *request* con las instancias o servidores, ya que se hace en orden. Si tenemos tres instancias, la primera *request* irá a la primera instancia, la segunda a la segunda, la tercera a la tercera, la cuarta a la primera instancia y así.



Weighted Round Robin

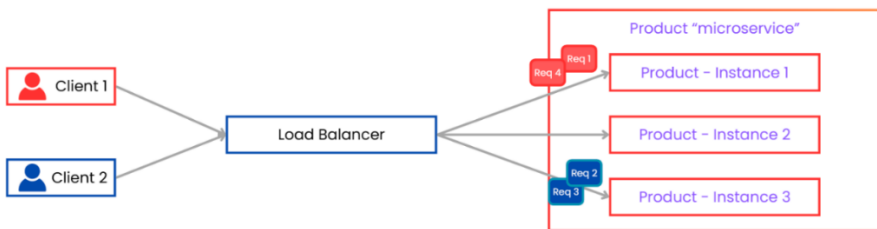
Similar al caso anterior, en esta ocasión configuramos de forma manual qué instancias tienen más preferencia, ya bien sea porque tienen más recursos, o porque son capaces de trabajar con una mayor carga. Como puedes imaginar, este algoritmo, en un *Load Balancer* con auto escalado,

no se suele utilizar ya que todas las instancias son iguales (normalmente).

En cambio, si hemos desplegado una instancia mucho más grande de lo habitual para cubrir una carga de trabajo estacional como las navidades, podemos indicar que esa instancia en concreto es el doble de potente, y el *Load Balancer* mandará a dicha instancia el número proporcional de *request* conforme el peso que le hayamos dado. En otras palabras, si tenemos tres instancias, dos de ellas son un 20% cada una, y la tercera es un 60%; el 60% de las llamadas irán a la tercera, mientras que las otras dos recibirán un 20%.

Sticky Round Robin

Una variación del *Round Robin* donde las *request* de un mismo cliente se mandan a la misma instancia.



Este tipo de algoritmo suele ser utilizado si tienes que precargar información en memoria, o tener información relacionada entre diferentes llamadas.

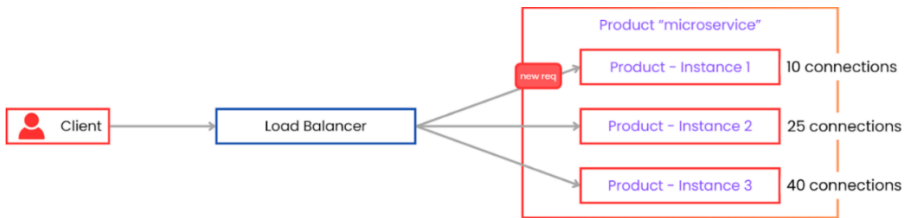
Un punto para tener en cuenta en este algoritmo es que las *request* de nuevos clientes se asignan de forma completamente aleatoria, lo que implica que, si tienes tres instancias, tendrás una al 90% de capacidad, otra al 70%, y otra al 20%.

IP hash

Cada cliente que realiza una llamada lo hace desde una IP, esa a esa IP se le hace un *hash* y dependiendo del valor de dicho hash se enlaza a una instancia u otra.

Least Connections

Como el nombre indica, este algoritmo da prioridad a la instancia, con menos conexiones activas en ese momento, y lo que hace es prever una sobrecarga de las diferentes instancias.



Esto conlleva a que debemos de mantener una cuenta del número de conexiones actuales.

Least response time

Es similar a *Least connections*, pero en este caso distribuimos a la instancia que menos tiempo tarda de media en completar una *request*.

Igual que en el caso anterior, debemos mantener una monitorización constante del tiempo medio de respuesta de cada *request*, lo que añade complejidad.

CAPÍTULO 4

Comunicación Síncrona

Llegados a este punto, hemos visto que unos microservicios dependen unos de otros, por lo tanto, se van a tener que comunicar de alguna forma. Aquí es donde entran los diferentes tipos de comunicación que podemos tener.

La realidad es que no hay unas formas que sean mejores ni peores que otras en general, pero sí que las hay que son más adecuadas en diferentes situaciones.

Por norma general, cuando migramos a microservicios, lo más sencillo es utilizar comunicación síncrona, por lo menos de primeras, ya que la comunicación síncrona no es más que una llamada directa en la red que espera una respuesta.

Una de las características principales de la comunicación síncrona es que nos aseguramos de que todos los procesos se ejecutan en orden. Pero puede hacer que ciertos flujos sean algo lentos cuando tenemos que consultar múltiples microservicios en cadena. Por ejemplo, en nuestra web de una tienda, si abrimos un envío vamos a ver a que pedido hace referencia y ese pedido contendrá la información de los productos.



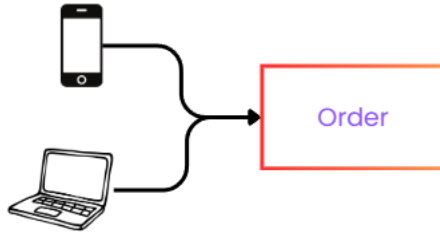
Lo que implica que nuestro servicio de envíos hará una llamada a pedidos que, a su vez, consultará productos, montará la información y la devolverá a envíos para que la imprima por pantalla.

Al trabajar de forma síncrona en ciertos escenarios, podemos hacer llamadas en paralelo para que la respuesta al usuario final sea más rápida, pero cuando tenemos llamadas que actúan en cadena, no podemos hacer mucho para aligerar dicha latencia. Y sí, utilizar microservicios es más lento que si trabajamos en un monolito donde puedes crear una consulta o un *store procedure* donde haces un *join* de las diferentes tablas. Quiero dejar este punto claro, porque mucha gente se opone al cambio y uno de los motivos es ese, pero como todo en programación es un *trade off*, es un precio que pagamos por tener otros beneficios.

A la hora de comunicarnos tenemos varias formas de hacerlo, la más común es una llamada HTTP utilizando REST, pero tenemos otros como GraphQL o gRPC y nos comunicaremos a través de una API (*Application Programming Interface*) y el objetivo de dicha API es simple, permitir la comunicación entre diferentes dispositivos.

API REST

Dentro de las formas o métodos que tenemos de comunicarnos, la más común, con diferencia, es REST (*Representational State Transfer*), y el motivo de ser la más común es simple, la gran mayoría de webs y aplicaciones móviles utilizan este protocolo para comunicarse con el servidor, o el *backend*.



De hecho, si vas a incluir servicios de terceros dentro de tu aplicación lo más probable (bueno en un 99.9% de casos) es que esos servicios de terceros expondrán una API, la cual será un API REST, ya que ha sido el estándar en las últimas décadas.

Uno de los motivos principales de que REST se use tanto es su facilidad a la hora de ser implementado, no solo en el lado del servidor, sino en el lado del cliente; donde todos los lenguajes de programación tienen librerías y funcionalidades para hacer dichas llamadas. Además, es el protocolo que utilizan los navegadores para llamar a tu aplicación por lo que, si quieres servir una web, necesitas poder utilizar REST.

Cómo funciona REST

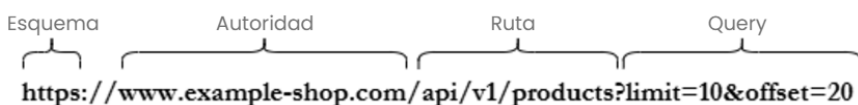
Una de las características principales de REST es que es *Stateless*. Esto quiere decir que ni el servidor ni el cliente deben mantener información del otro para las subsecuentes llamadas, lo que significa que cada llamada que se realiza es completa, o puede ser completada al 100% por el *backend*, sin necesidad de esperar por mensajes previos. Por este motivo, el estado de la sesión se hace en el lado del cliente.

La forma en la que REST funciona es muy sencilla. Una API REST expone recursos a través de una URI, la cual es única para cada recurso y estos recursos están agrupados por nombre.

Quiero pensar que todo el mundo sabe lo que es una URL, pero por si acaso vamos a hacer un pequeño resumen. Las URLs son la forma de

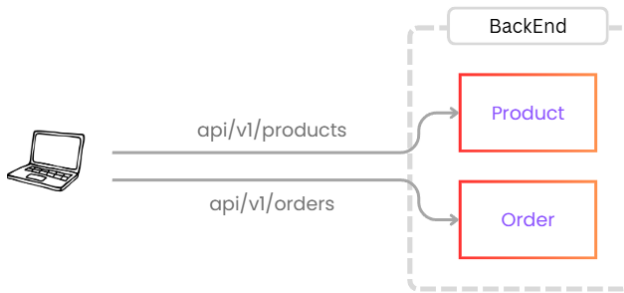
identificar recursos en internet y radica de múltiples partes:

- **El esquema:** la primera parte indica cómo acceder al recurso, cuando usamos API será HTTP o HTTPS, pero también puede ser otros como ftp.
- **Autoridad:** generalmente es un dominio web, aunque también puede ser una dirección IP, esto es básicamente la web, como `www.example-shop.com`.
- **Ruta:** ubicación específica de dicho recurso en el servidor; por ejemplo `/products`, para acceder a los productos. En el caso de las APIS solemos tener una ruta un poco más larga donde indicamos que es una API y la versión de dicha API, `/api/v1/products`.
- **Query:** parámetros opcionales que pueden servir para filtrar recursos, si estamos haciendo la consulta a `api/v1/products`, nos debería de poner todos, pero podemos incluir el *offset* y el *limit* para incluir una paginación.



Esto quiere decir que en una API REST bien diseñada accederemos a `/products` para acceder a los productos y a `/orders` para acceder a los pedidos.

Técnicamente debemos utilizar la URL completa, pero para simplificar, en los diagramas, se utilizará solo la ruta. Además, si exportas esto al mundo empresarial, la empresa puede ser vendida, o cambiar de nombre, lo que implica que la autoridad va a cambiar, pero la ruta, que es el acceso a cada uno de los recursos, seguirá siendo la misma, ya que es independiente.



En este ejemplo tenemos un ordenador portátil que da a entender que es una web, pero puede ser tanto un cliente móvil, como un cliente cualquiera que tenga acceso. De hecho, también podemos utilizar REST para la comunicación entre los diferentes microservicios.

Al acceder a estos recursos lo estamos haciendo por la URL, pero no es tan sencillo como eso. Cuando hacemos una *request* en un navegador a través de la caja superior, estamos utilizando el método GET de HTTP, que significa coger, pero no es el único, y depende de la acción que queramos ejecutar en nuestra API debemos utilizar unos u otros, ya que es la combinación entre el método HTTP (también llamado verbo), junto con la ruta del recurso, lo que invoca al servidor para saber qué acción generar.

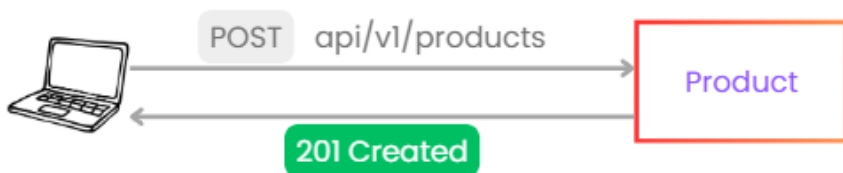
Método HTTP	Efecto	Ejemplo
GET	Obtiene datos de un recurso	GET /api/v1/products/ 1
POST	Envía datos para crear un recurso	POST /api/v1/products
PUT	Envía datos para actualizar un recurso	PUT /api/v1/products/ 1
DELETE	Indica que recurso vamos a eliminar	DELETE /api/v1/products/ 1

De estas cuatro definiciones, es de donde viene el acrónimo CRUD. Y aunque también tenemos los métodos HEAD, OPTIONS y PATCH, estos no se utilizan apenas en el mundo real.

Conviene saber que *head* es como un *get* que no devuelve un *body*. *Options* devuelve los métodos HTTP soportados en un recurso, y PATCH es como PUT pero que solo actualiza parte de un recurso.

Además de la URL y el verbo, cuando hacemos una llamada, tenemos la opción de añadir un cuerpo a la *request* (http body) que, normalmente, será el objeto para crear y está definido por el servidor.

En nuestro caso, es tan sencillo como en el POST enviamos toda la información del producto a crear, normalmente utilizando Json, y, si todo funciona como es debido, el servidor nos devolverá que todo ha funcionado bien, con un código de estado correcto:



Pero ¿qué es este estado correcto? Ese estado correcto es la respuesta del servidor y, tanto si el proceso ha creado el recurso como si no, nos devolverá un estado, idealmente diferente y si tenemos algún error debería mostrarnos diferentes estados dependiendo que tipo de error tenemos, aunque en la práctica eso no siempre pase.

Los códigos de estado HTTP los podemos agrupar en cinco grupos diferentes.

Códigos de estado Informacionales

Del 100 al 199 son códigos de estado informacionales. Si, por ejemplo, tienes un proceso que lleva tiempo, pongamos 10 minutos de ejecución, los clientes pueden preguntar en qué estado está dicho proceso, y tu respondes con un 102 que significa procesando.

Normalmente los 1xx no se ven mucho por no decir nada, ya que el más común sería el 102, pero en el mundo real se utiliza el 202 que veremos ahora.

Códigos de estado de éxito

El segundo de los grupos es de 200 a 299, y son los códigos de estado exitosos, cuando una *request* ha funcionado con normalidad. El más común es el código 200, que significa OK, y es el que se devuelve en las peticiones GET en la web cuando estas son correctas. Pero no hay que olvidar que tenemos más, como puede ser el 201, para la creación de recursos, o el mencionado 202, que se utiliza cuando tenemos procesos de larga duración.

Códigos de estado de redirección

Como su nombre indica, son códigos que le indican al navegador que un recurso ha sido movido, y van del 300 al 399. Es común si cambias de dominio ver cómo *example-web* redirecciona a *new-web* y el código de estado que ves en la primera llamada es un 301.

Códigos de estado de errores del cliente

Utilizaremos los códigos de estado entre el 400 y el 499 para indicarle al cliente que, ya bien sea el recurso que está consultando, o el cuerpo del mensaje son erróneos. Por ejemplo, el 401 se utiliza cuando el cliente está intentando acceder a un recurso para el que no tiene las credenciales necesarias.

El 404 es cuando intentas acceder a un recurso que no existe, por ejemplo, `/api/v1/products/1234` si ese producto no está creado, el error debería ser un 404.

Los códigos de estado 404 son los más extensos, ya que cubren una gran variedad de situaciones con las que podemos indicarle al cliente, desde el lado del servidor, qué es lo que está mal con la petición que han realizado.

Códigos de estado de errores en el servidor

Del grupo entre el 500 y el 599 tenemos a los códigos de estado que indican que algo va mal en el servidor. Estos suelen suceder cuando tenemos un problema inesperado, ya que no son errores normales. Si, por ejemplo, estás utilizando .NET y te salta una excepción de que la base de datos no está disponible, el servidor devolverá un estado 500.

Hay que tener cuidado con los errores de parte del servidor, ya que muchos clientes tendrán una póliza de reintentos en el lado del cliente y tener una aplicación parcialmente caída puede multiplicar el uso de esta, ya que los clientes no pararán de consultar y puede llegar a ser un problema en el caso de las creaciones de recursos.

Esto es debido a que podemos generar un 500 después de la creación de un recurso, pero el cliente no sabe que ese recurso está creado, así que

vuelve a intentarlo en un bucle y podemos acabar en la situación donde un recurso se ha creado múltiples veces.

GraphQL

Otra forma de comunicarnos con el servidor, o con las aplicaciones, es a través de GraphQL, el cual es un lenguaje de consultas, a nivel de aplicación, que permite a los usuarios o clientes especificar exactamente qué información quieren recibir de un recurso.

Cuando utilizamos REST y consultamos un producto, la API nos va a devolver toda la información de un producto, pero en GraphQL no es así, porque en la propia consulta le podemos indicar qué propiedades de dicho producto queremos que nos devuelva. Esta funcionalidad puede ser muy interesante porque nos permite ahorrar ancho de banda, que en conexiones lentas o de móviles puede ser crucial. Si, por ejemplo, estás listando todos los productos y la API de productos nos devuelve el id, el nombre, la descripción, el stock, valoraciones, análisis, imagen principal, imágenes, etc., al final es mucha información que va por la red que no vamos a utilizar, pero con GraphQL podemos indicarle que nos devuelva el id el nombre y la imagen principal. De esta forma ignoramos el resto de las propiedades y la llamada es mucho más rápida.

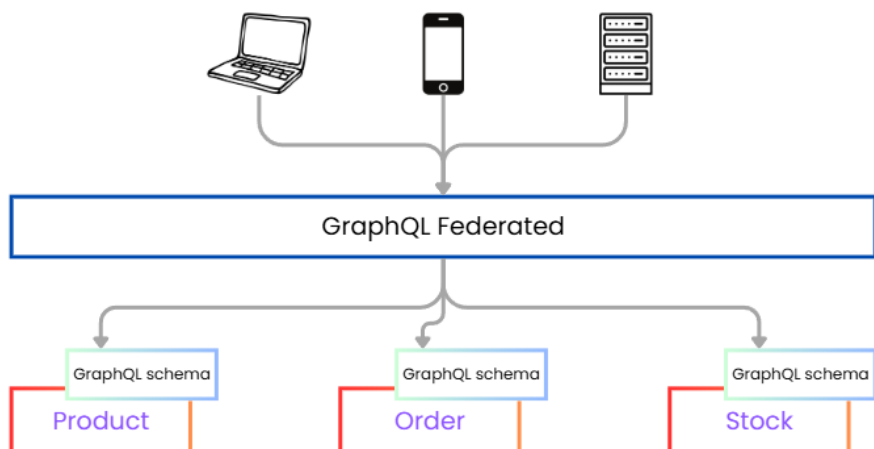
Como cosas a saber de GraphQL hay que mencionar que soporta tanto consultas (*Query*), como actualizaciones o creaciones a través de las mutaciones. Personalmente creo que hacer mutaciones con GraphQL es un error, que para eso ya está REST, pero este tema no es para este libro.

Lo importante es que cuando añadimos GraphQL a una API y creamos la *Query* y la mutación, se nos genera un fichero llamado esquema, el cual

va a ser público y contiene la definición de la API de GraphQL, tanto de los tipos o propiedades como de las relaciones entre ellas. Esto es importante. El esquema de GraphQL es un esquema fuertemente tipado, que puede ser utilizado en lenguajes débilmente tipados y lo que hace es forzar a dicha API a tener tipos. Por lo que te asegura que, en la capa de entrada a tu sistema, todas las *request* que vayan a ser ejecutadas, van a ser validadas contra el esquema y así podemos asegurar que los datos que vienen son válidos.

GraphQL Federated

La gracia de utilizar GraphQL viene si utilizas *GraphQL Federation*, que es un patrón de arquitectura que permite combinar múltiples *endpoints* de GraphQL en uno solo; y al trabajar con microservicios es una funcionalidad muy poderosa, ya que los clientes van a realizar las llamadas a un único *endpoint* y ahí, en el cuerpo del mensaje, pueden especificar toda la información que necesiten, porque igual que en los ficheros de esquema individuales, la capa de federación tiene los tipos y las relaciones entre los mismos.



Cada uno de nuestros servicios generan un esquema GraphQL, al cual llamaremos *SubGraph*, y es específico de cada uno de los microservicios. El *GraphQL Federated* integra todos los *SubGraphs* y crea un *SuperGraph*. La magia viene cuando recibes una llamada, ya que es el propio *GraphQL Federated*. Ya bien sea tuyo o tengas un servicio de terceros, sabe dónde tiene que ir a consultar cada propiedad consultada por el cliente devolviendo únicamente la información necesaria.

Pero la integración no acaba ahí. ¿Recuerdas que hemos dicho que GraphQL conoce las relaciones entre los diferentes objetos? Esto se aplica a nuestro *SuperGraph*. Desde la interfaz podemos mandar una consulta para que nos devuelva la información básica de un producto y, además, nos devuelva cuánto stock tenemos. Será la propia capa de *GraphQL Federated* la que se encargue de llamar a ambos microservicios, sabiendo que ID tiene que pasarle para recibir el stock. Y si, por ejemplo, la consulta que nos viene desde el cliente no contiene el stock, únicamente haremos la llamada al microservicio de producto.

GraphQL contado así parece una maravilla, pero es a su vez un arma de doble filo. El motivo es que, al tener tantas relaciones, podemos tener enlaces entre diferentes entidades en bucle y, somos, por defecto, vulnerables a ataques, ya que en este escenario tendríamos consultas siendo ejecutadas entre nuestros servicios sin parar. Como todo en programación, tiene pros y contras y, si a ti te sirve, y te ayuda a producir valor para la empresa, adelante.

gRPC

Vamos a hablar sobre gRPC, que, si vamos a su descripción, es un *framework* universal de *Open Source* para RPC de alto rendimiento. Si desglosamos esta frase podemos entender lo siguiente:

- Es de **alto rendimiento** por la forma en la que serializamos y desrealizamos la comunicación o las llamadas.
- Es un **framework universal** porque es agnóstico al lenguaje de programación. Funciona tanto con C#, Java, C++, Python, etc. Cualquier lenguaje que sea maduro tiene soporte para gRPC.
- RPC son las siglas de **Remote Procedure Call** (llamada a procedimiento remoto), que viene a significar la solicitud de un servicio o programa a que otro realice una operación. Si comparamos con REST, es lo mismo que una *request*.
- ¿Y la g? De forma oficial, la web dice que la G es como la L de Linux, que la *g* significa gRPC. Pero, gRPC fue desarrollado por Google, así que, que cada uno saque sus propias conclusiones.

gRPC puede parecer muy similar a REST si lo miramos a nivel de arquitectura o diseño, pero su implementación es muy diferente, ya que cuando utilizamos gRPC vamos a tener una comunicación binaria, y esto, trae consecuencias.

El efecto más claro y evidente es que los humanos, por norma general, no podemos leer binario, así que esta comunicación está pensada para que dos máquinas se comuniquen entre sí, sin la intervención o supervisión humana.

Debido a que estamos utilizando binario, el tamaño de los paquetes que se envían por la red son lo más pequeños posible; lo que hace que sea mucho más rápido y barato si ejecutas tu sistema en un proveedor de servicios.

A la hora de implementar en las aplicaciones gRPC, no es tan sencillo como en REST. Con REST es una llamada normal y corriente y será el servidor quien valide si una llamada es correcta o no. En el caso de gRPC, la cosa cambia.

Arquitectura de gRPC

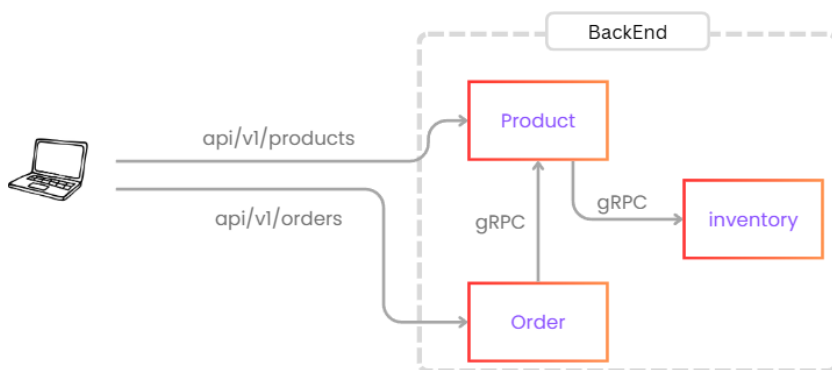
La comunicación en gRPC se basa en contratos. Esto quiere decir que para poder serializar y deserializar correctamente la información, tanto del lado del cliente como del servidor, ambos necesitan tener el mismo contrato, o la deserialización fallará.

Estos contratos son ficheros que definen los tipos, objetos y métodos permitidos por el servidor, y contienen la extensión “:proto” y debemos mantenerlos actualizados cada vez que realizamos un cambio en el código, ya que, bueno, el cambio en realidad lo harás en el fichero proto y de ahí lo aplicarás a tu código sea del lenguaje que sea.

Para comunicar entre diferentes aplicaciones necesitamos usar HTTPS (sobre HTTP/2) así que toda la comunicación es no solo rápida y eficiente, sino también segura.

Para finalizar, gRPC soporta o tiene la capacidad de enviar y recibir datos de forma simultánea; no es necesario que una *request* termine para empezar a devolver información. Esto puede ser una característica muy potente si estás trabajando, por ejemplo, en una aplicación de vídeo donde todos los participantes envían y reciben vídeo de forma constante.

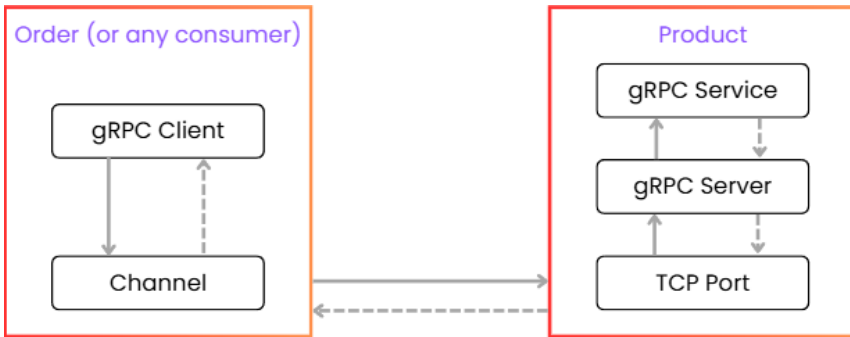
Aun así, el número de casos de uso donde transmitimos vídeo en la vida real son muy limitados; la gran mayoría de empresas no tienen esta necesidad. El mejor caso de uso para gRPC tampoco es entre la comunicación del *FrontEnd* con el *BackEnd*, donde brilla de verdad es en la comunicación entre servicios, ya que hace que las llamadas entre dichos servicios sean muy rápidas.



En esta imagen tenemos nuestros servicios que tienen, tanto acceso por una API REST, para ser invocados desde el *FrontEnd*, como acceso a través de gRPC para ser utilizados por otros servicios internos. Además, si se da el caso de tener servicios a los que los clientes no tienen que acceder, podemos tener dicho servicio sin acceso por API REST y únicamente utilizar gRPC.

Por detrás, la forma en la que gRPC funciona, es un poco más compleja. Al trabajar con contratos podemos decir que tenemos una serie de componentes o procesos que definen la forma en la que vamos a trabajar. En el lado del servidor tenemos nuestro servicio, que contiene el servidor gRPC y, en dicho servidor, tendremos un puerto abierto (normalmente TCP) para aceptar la comunicación.

En el lado del cliente, tenemos un cliente gRPC (que normalmente será una librería creada por el equipo responsable del servicio), y un *channel*. Este *channel* es un objeto de gRPC el cual se encarga de administrar la comunicación entre el cliente y el servidor. Por lo tanto, es el propio objeto *channel* el que envía la *request* a través del puerto TCP. La *request* va al servidor y, de ahí, va a redirigirla al servicio correspondiente; el cual contiene el código con la implementación de la lógica. Cuando termina, hacemos el camino inverso para devolver la respuesta al consumidor.



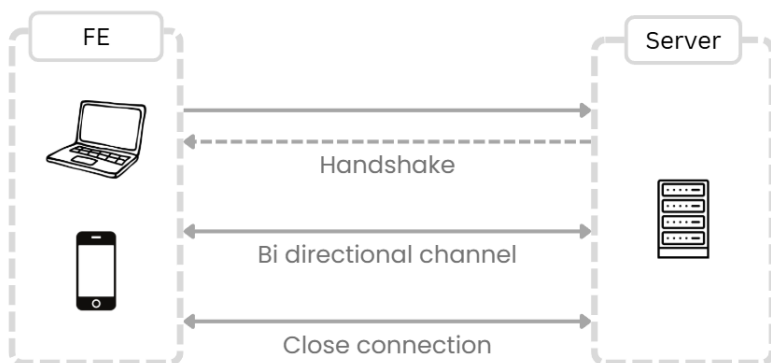
Y, aunque parece un proceso muy complejo, en verdad no lo es tanto, porque tanto el *Channel*, como el servidor de gRPC, como el puerto, y la comunicación entre el *Channel* y el servidor nos lo da la propia librería de gRPC. Obviamente debemos incluir la configuración en el cliente para hacer la llamada a la URL y puerto correcto, y configurar el puerto en el lado del servidor. Pero desde un punto de vista práctico, lo único que necesitamos mantener es el cliente de gRPC y el servicio, y ambos se pueden generar automáticamente con el fichero “:proto”, que hemos mencionado anteriormente.

Websockets

Aunque no es normal la comunicación a través de *websockets* dentro de los sistemas distribuidos, entre las diferentes aplicaciones de *BackEnd*, el uso de *websockets* es muy común cuando se trabaja con aplicaciones con información en tiempo real, para la comunicación entre el cliente y el servidor.

Cuando utilizamos *websockets*, tenemos que abrir una conexión entre el cliente y el servidor. Para ello, hacemos una llamada HTTP. En este punto, convertimos la llamada HTTP y, utilizando el socket TCP, el servidor le indica al cliente que debe actualizarse a un *websocket-socket* y dejar el canal abierto, a esta acción se le llama *handshake*. Desde ahí, tenemos

un canal bidireccional donde tanto el cliente como el servidor puede mandar mensajes.



Ese canal seguirá abierto hasta que uno de los dos, ya bien sea el cliente, o el servidor, lo cierren.

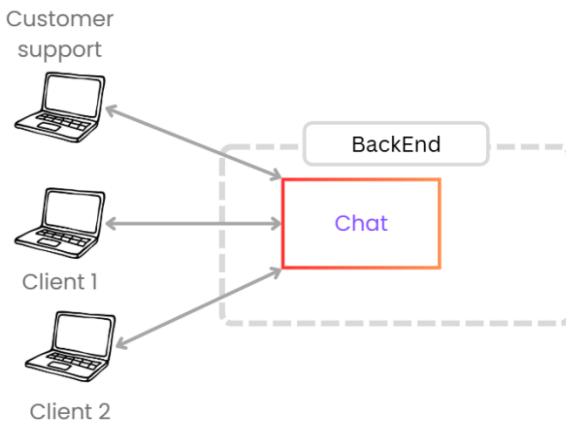
El uso de *websockets* es específico para ciertos casos de uso, principalmente debido al coste que puede llegar a traer, primero en términos de mantenimiento o infraestructura. No todos los navegadores soportan *websockets*, así que dependiendo de qué tipo de aplicación y quiénes sean nuestros clientes, no nos sirve. Del lado del servidor, podemos tener problemas con *proxys* y *firewalls*, es cierto que hoy en día no debería ser un problema, pero si tienes una versión vieja haciendo de *proxy*, quizá tengas que actualizar, lo que trae consigo un coste de personal y un riesgo.

Otro problema muy importante es que la conexión queda abierta, eso quiere decir que si la red falla, la conexión se cierra y el funcionamiento de una web y, por tanto, la experiencia de usuario puede verse afectada.

Finalmente podemos hablar de la escalabilidad. El uso de *websockets* mantiene una conexión abierta entre el cliente y el servidor, que, a gran escala, requiere muchos recursos.

Aún con todas las desventajas aparentes, el uso de *websockets* tiene cabida en una arquitectura para cierto tipo de aplicaciones. Un ejemplo es todo el tema de juegos, donde refrescamos la información de la interfaz, sin volver a cargar la ventana principal, lo mismo aplica para cualquier web en tiempo real.

Pero el ejemplo del que vamos a hablar es una aplicación de chat. Ya sea un chat entre dos personas, o un chat grupal, el funcionamiento es el mismo, todas las partes se conectan al servidor.



El servidor tiene una conexión abierta con cada uno de los clientes y cuando recibe un mensaje de uno de ellos, lo propaga al resto, actualizando en tiempo real en todas las terminales. Es la aplicación *backend* la que se encarga de propagar el mensaje únicamente a aquellas máquinas que no lo han enviado, y la implementación depende del lenguaje que estemos utilizando, pese a que la idea y arquitectura que tiene detrás sean la misma.

Comunicación asíncrona

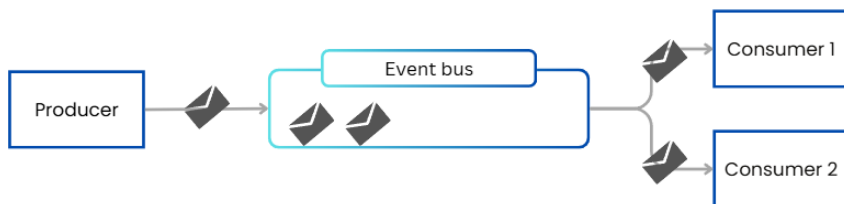
Hasta ahora todo lo que hemos visto ha sido relacionado con microservicios, pero estos son únicamente la punta del iceberg cuando hablamos de los sistemas distribuidos.

La realización de llamadas entre microservicios para obtener información o enviar comandos es una opción, pero no es la única, especialmente si no necesitamos tener una respuesta inmediata.

El patrón Productor Consumidor

Otra técnica muy común en los sistemas distribuidos es utilizar el patrón productor consumidor (*producers/consumers*), el cual es un patrón que se basa en generar mensajes, también llamados eventos, y consumirlos en otro servicio. Estos eventos no se mandan de un sistema a otro de una forma directa, sino que la aplicación productora genera un evento, el cual va a un *bus* que almacena dicho mensaje de forma temporal, y los consumidores están suscritos a dicho *bus*, por lo que dicho evento se propaga entre los consumidores.

Los eventos no dejan de ser un mensaje generado por una aplicación (o sistema), en el que podemos poner la información que queramos.



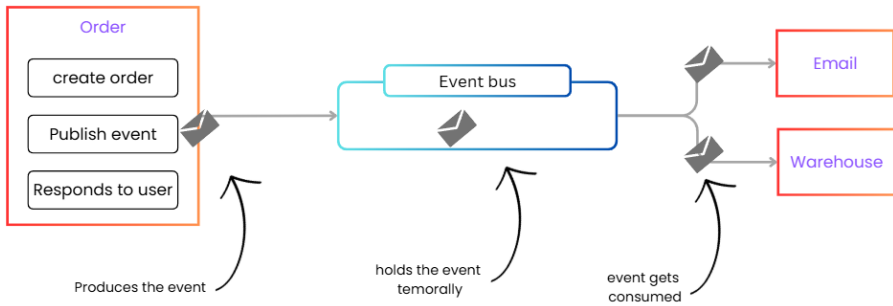
Podemos explicar su funcionamiento de una forma muy sencilla con un ejemplo.

Estamos en nuestra tienda, donde estamos vendiendo productos. Cuando alguien hace un pedido lo que hacemos es enviar un email. Pero ¿cómo funciona este email?

Si estuviéramos en una arquitectura de microservicios tendríamos una llamada directa a dicho servicio de email, lo cual tiene sus complicaciones, ¿qué pasa si el servicio de email está caído? ¿Paramos el proceso? Obviamente no, gracias a la comunicación asíncrona.

Cuando nuestro microservicio de pedidos crea el pedido, generamos un evento el cual es publicado a un *Event Bus* específico, a la vez, devolvemos al usuario que todo ha funcionado bien.

En este momento, el usuario puede ver por pantalla que su pedido se ha generado correctamente, pero, en realidad, su pedido únicamente ha sido añadido dentro del servicio de pedidos, ni se ha enviado el email, ni el sistema que almacena sabe qué tiene que hacer ni nada. Pero gracias al patrón productor consumidor en paralelo al usuario recibir la confirmación, vamos a estar procesando dicho evento en cada uno de los consumidores.



Que en nuestro supuesto es el sistema de email, el cual mandará un email, y el sistema de almacén, el cual generará la orden a los empleados del almacén para empaquetar el producto.

Algo para tener en cuenta es que generamos un único evento, pero lo consumimos con dos consumidores diferentes. Desde el punto de vista de quien produce nos da igual qué es lo que los consumidores hagan con ese evento.

Dicho esto, cada servicio puede publicar tantos mensajes como quiera, y cada consumidor puede consumir tantos como quieran, lo único que tienen que saber es la dirección del *Event Bus*, o de las colas.

Hay varias características que vamos a ver sobre la comunicación asíncrona que vamos a tratar durante este libro, pero ahora vamos a hablar un poco de los eventos.

Eventos

La idea de los eventos es que sean ligeros y contengan información de que una acción ha sucedido en el sistema. Y a la hora de crear dichos eventos, tenemos dos opciones: crear eventos que contengan la información mínima, o eventos completos. ¿A qué me refiero con esto?

En nuestro caso, donde un pedido se genera, podemos tener dos tipos de objetos, uno que se llame *OrderCreated* y contenga el *OrderId*, o podemos tener el mismo evento *OrderCreated* y que contenga no sólo el *OrderId*, sino también qué productos han sido comprados, la información del envío, la información personal, de pago, etc.



La diferencia es que, en el primero de los eventos, cuando tenemos la información mínima, cada uno de los servicios que consuma nuestro evento, va a necesitar hacer una *request* a nuestro microservicio para obtener la información necesaria. Por ejemplo, el servicio de almacén necesita hacer una llamada a nuestro servicio de pedidos para saber qué productos ha comprado. El de email necesitaría consultar el email, etc. Lo más normal es enviar toda la información necesaria en cada evento, para evitar esta sobrecarga de llamadas.

Finalmente, en esta sección, tenemos el nombre del evento. Los eventos representan una acción que ya ha sucedido, por lo tanto, en el pasado, por lo que, el nombre del evento debe estar en pasado.

Además de los eventos, hemos hablado de los elementos clave en el patrón productor consumidor y en las librerías distribuidas, que son el *Event Bus* y las colas de los cuales vamos a ver y diferenciar ahora mismo.

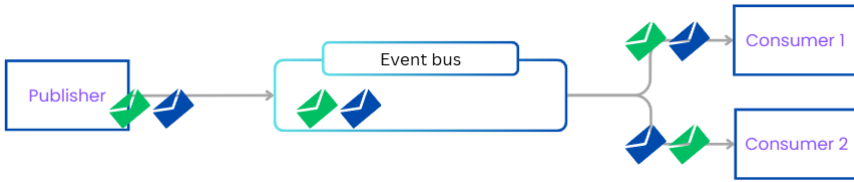
Event Bus

Un *Event Bus* es una pieza de infraestructura (una aplicación) que permite a múltiples sistemas comunicarse. Normalmente tendremos un productor de eventos y tendremos uno o varios consumidores de dicho evento. Si acomodamos esta descripción al ejemplo que acabamos de ver, el sistema de pedidos genera un evento y dos servicios responden a dicho evento.

Cuando utilizamos un *Event Bus*, debemos saber que ninguno de los elementos involucrados (productor o consumidores) es consciente de ninguno de los otros elementos. Esto quiere decir que ni al productor le importa quien consuma, ni los consumidores esperan mensajes que no saben que están ahí. Obviamente, para la lógica de tu sistema es importante, pero al consumidor le da igual quien genera el evento *OrderCreated*, desde su punto de vista lo único que hacen es consumir dicho evento.

Una característica clave dentro de los *Event Bus*, es que **no podemos garantizar el orden de procesamiento de los mensajes**. Y sí, normalmente están en orden, pero pongámonos en el supuesto de que dos pedidos se generan a la vez, Pedido A y Pedido B.

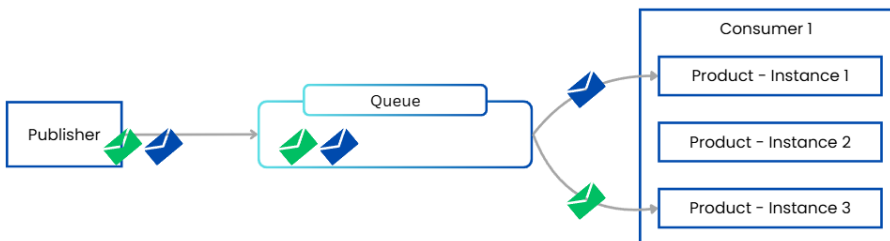
Cuando los eventos llegan al consumidor pueden llegar tanto en el orden Pedido A - Pedido B como en el orden Pedido B - Pedido A.



Al final, lo importante de un *Event Bus* es que todos los mensajes lleguen a todos los consumidores. Pero, si un mensaje llega al *Event Bus* y no hay ningún consumidor, ese mensaje se va a perder, esto es un funcionamiento esperado de los *Event Buses*.

Colas

Cuando hablamos de *Colas*, hablamos de comunicación asíncrona pero que tiene una relación 1:1, esto quiere decir que un servicio produce los mensajes, y únicamente un servicio lee dicho mensaje. Esto es importante ya que es una gran diferenciación con respecto a lo que hemos visto con los *Event Bus*. En un *Event Bus* el mensaje se consumía por todos los consumidores, mientras que aquí solo lo hace con uno.

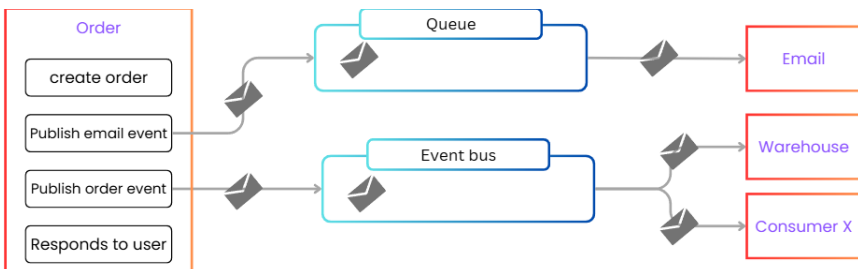


Esta funcionalidad aplica con las instancias de un servicio, donde un mensaje va a ser consumido por una única instancia. Pero también aplica si tienes mal la configuración de un servicio donde un mensaje en una

cola es consumido por un consumidor que no toca, haciendo que el consumidor original pierda dicho mensaje y, posiblemente, liando una gorda en la empresa, porque ahora tu sistema tiene datos incorrectos.

Otro punto importante de las colas es que estas almacenan los mensajes. Hemos dicho que, en el *Event Bus*, los mensajes, si no hay nadie consumiendo, desaparecen, aquí no, aquí lo que sucede es que se quedan en la cola hasta que son consumidos o eliminados después de cierto tiempo, si configuras el TTL (*time to live*). Pero no solo eso, una cola puede ser configurada para que los mensajes se procesen en orden de llegada con las colas FIFO, o en orden inverso con las colas LIFO, y al no desaparecer el consumidor puede ir consumiendo como mejor le venga.

Y aquí nos viene perfecto el caso de uso que teníamos antes, donde cuando se crea un pedido se genera un evento al cual vamos a tener múltiples servicios escuchando. Puede parecer ideal, pero la realidad es que, si hacemos eso, nuestro servicio de email va a estar acoplado (que depende) de nuestro servicio de pedidos cuando esto no debería ser así. Lo mejor en este caso es generar un evento indicando que hemos generado un pedido y, a su vez, generar otro distinto el cual será exclusivamente para los emails.

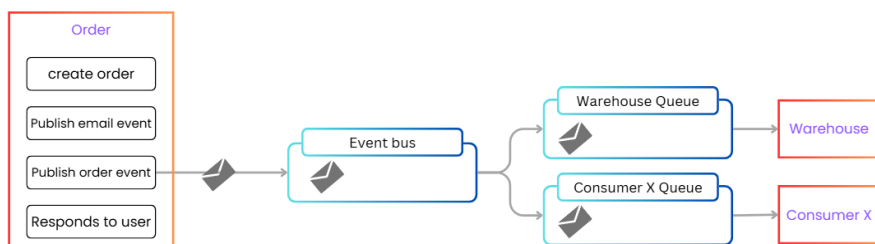


De esta forma todas las aplicaciones que envían emails, lo que van a hacer es enviar un evento a la cola de emails para que así el servicio de emails los vaya procesando.

Combinar *Event Buses* con *Colas*

Desafortunadamente, esta solución tiene un pequeño problema. Vamos a imaginarnos que estamos en época del Black Friday, y donde antes teníamos una instancia de pedidos, ahora tenemos diez. Esto implica que vamos a generar diez veces el número de eventos tanto para los emails, como para el *Event Bus*. Lo que implica que debemos ser capaces de escalar nuestro microservicio, que está consumiendo los mensajes, de forma acorde con el número de mensajes que entran y asegurarnos de que la aplicación no se cae, porque recuerda, si un *Event Bus* no tiene consumidores el evento se pierde.

Aquí es donde entra la combinación de las *Colas* junto a los *Event Buses*. Lo que hacemos en esta situación es que en vez de suscribirnos con los servicios al *Event Bus* directamente, creamos una *Cola* entre el *Event Bus* y el servicio, la cual va a estar suscrita al *Event Bus*, y la aplicación va a procesar los mensajes de la *Cola*, y, como estos no se pierden, no tenemos la necesidad de escalar en tiempo real como lo haríamos si estuviéramos suscritos al *Event Bus* directamente.



Este escenario añade complejidad a la arquitectura del sistema, pero, a la vez, nos da una seguridad y tranquilidad, porque no vamos a perder información en el proceso. Obviamente tendremos que escalar los sistemas para ser capaces de coger una carga diez veces superior a la

normal, pero no tendremos que escalar x10, por lo menos no instantáneamente.

Streams

En una posición intermedia entre las *Colas* y los *Event Buses* tenemos a los *Streams*.

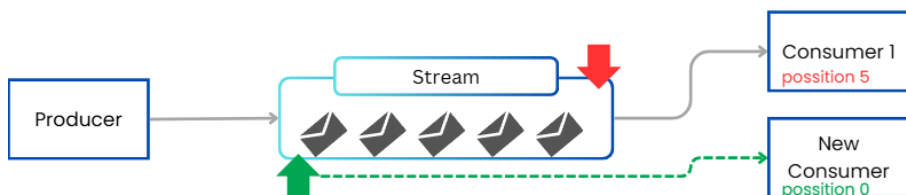
La idea general es exactamente la misma, un productor publica eventos a un *Stream*, los cuales son procesados y leídos por un consumidor. La diferencia en este caso es que el consumidor, cuando se suscribe, va a empezar a procesar los mensajes desde el principio y va a ir procesándolos uno a uno.



Estos mensajes tienen un TTL, y, mientras ese TTL no se cumpla, los mensajes no van a ser borrados. En aplicaciones como Kafka, puedes tenerlos almacenados en el *Stream* de forma indefinida, lo que implica que cuando una nueva aplicación se suscribe a dicho *Stream*, lo que hará será procesar todos los mensajes que se han publicado desde el principio de los tiempos.

Así que los *Streams* son como una cola porque mantiene los mensajes, y es como un *Event Bus* porque permite tener múltiples suscriptores para el mismo *Stream*. Pero, a diferencia de los anteriores, es el consumidor quien tiene que mantener la información sobre qué mensaje ha sido el último que ha leído.

Por ejemplo, tenemos el *consumer 1* está al día, por lo que tiene almacenada la información sobre cuál ha sido el último mensaje que ha leído. Pero cuando llega un nuevo consumidor, este debe empezar por el principio del *Stream* procesando todos los eventos.

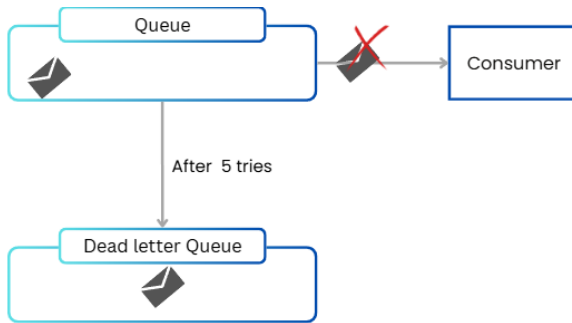


Esta funcionalidad es muy útil para rellenar con información los nuevos servicios que se van creando en la empresa, o para volver a procesar mensajes cuando tenemos cualquier problema.

Dead Letter Queue

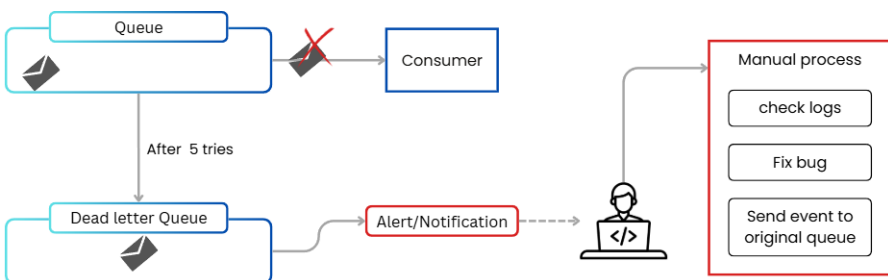
Hasta ahora hemos visto lo que sucede en el lado del consumidor si algo falla. En la versión de las *Colas* hemos indicado que el mensaje vuelve a estar disponible después de 10 o 30 segundos. Y sí, esto es cierto, pero qué sentido tiene procesar un mensaje infinitas veces si sabemos que va a fallar.

Aquí es donde entran las *Dead Letters Queue* (DLQ). Este tipo de *Colas* existen para almacenar los mensajes que han fallado una cantidad de veces, evitando, de este modo, que el consumidor los procese de forma indefinida.



En este escenario tenemos varias opciones. Actualmente, tenemos nuestro consumidor funcionando y un evento en la Dead Letter Queue. Este evento de la Dead Letter Queue no se va a procesar de forma normal, sino que lo más normal es que lance una alerta donde nos indique que algo ha ido mal, pero no sabemos el qué.

Por lo que ahora entra un proceso manual donde debemos revisar los logs de la aplicación; ver cuál puede ser el problema, arreglarlo si es un *bug* y volver a enviar el mensaje a la cola original para volver a ser procesado, lo cual es un proceso manual.

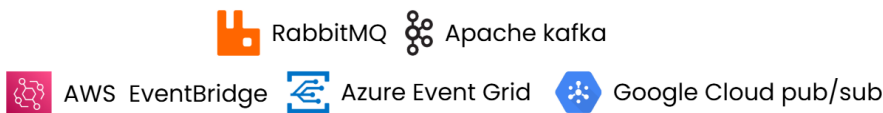


Y este proceso va a ser así con todos los eventos que lleguen a la *Dead Letter Queue*. Como puedes imaginar, es bastante trabajo. Pero un *bug* no es la única causa posible, también puede ser que el contrato haya cambiado y tengas que actualizar tu sistema, o incluso que el propio

mensaje esté mal formado y tengas que notificar a quien genera dichos mensajes.

Aplicaciones comunes para la comunicación asíncrona

Hay muchas aplicaciones que hacen de *Event Bus*, pero puedes leer en sitios que se les llama *message bus*, *event hub*, *service bus* o, incluso, *event grid*. Yo, comúnmente, las llamo, o bien *Event Bus*, o bien *Service Bus*. Todos estos son prácticamente iguales, y lo que hacen es expresar una capa de transporte para el patrón productor consumidor.



El más popular es posiblemente Kafka, seguido, por lo menos dentro del mundo de .NET, de RabbitMQ, además de, por supuesto, las propias implementaciones por parte de Azure, Google, o AWS. Su implementación puede ser un poco diferente en cada uno de ellos, pero el concepto es el mismo.

Por ejemplo, al publicar un mensaje en RabbitMQ técnicamente lo mandas a un Exchange dentro de un *broker*, el cual está conectado a *Colas* y puedes estar configurado como *fanout*, *header*, etc.

En el caso de Kafka, cuando publicas un mensaje lo haces en un *topic*, el cual es un *Stream*, que tiene diferentes *Consumer Groups* suscritos, por

lo que el *topic* propaga el mensaje a todos los Consumer Groups en un modo *fanout*.

Cualquier sistema que permita el patrón productor consumidor puede ser llamado un *Event bus*, y estos tienen reglas específicas de su implementación, pese a que el objetivo final es el mismo: mover un evento del servicio que publica a los que consumen.

Por ejemplo, cuando tú publicas un evento en Kafka, este va a un *topic*, y no desaparece del sistema hasta que el consumidor lo lee. En el caso de RabbitMQ, una vez va al Exchange, si no tiene ningún consumidor (que son las *Colas*), el evento se pierde para siempre, aunque hay técnicas dentro del propio RabbitMQ para enlazar el evento a una cola y dicho evento persistirá en dicha cola.

Como vemos, la idea es la misma, pero la implementación es distinta. Además, casos como Kafka tienen más funcionalidades, como hacer de *Event Streaming*. O Redis siendo popular por ser una caché distribuida en memoria soporta el patrón productor consumidor.

La arquitectura de un *Event Bus*

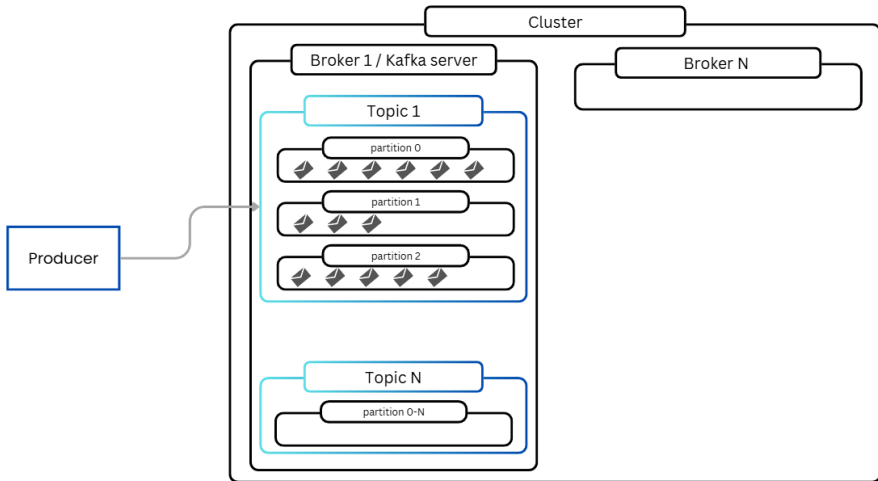
Los sistemas distribuidos se basan principalmente en el patrón productor consumidor, que, a su vez, se basa en los *Event Buses*. Llegados a este punto, veo muy importante explicar cómo funcionan, y también explicar algunas de las palabras o conceptos clave que hemos nombrado en este capítulo, pero no explicado.

Así que lo que vamos a hacer es explicar tanto Kafka como RabbitMQ, ya que son los más populares, y para ser sinceros, RabbitMQ es muchísimo más simple que Kafka. Pero claro, desde un punto de vista del productor o del consumidor da igual cual estés implementando, lo que importa es que el mensaje llegue desde el sistema productor al sistema consumidor.

Kafka en detalle

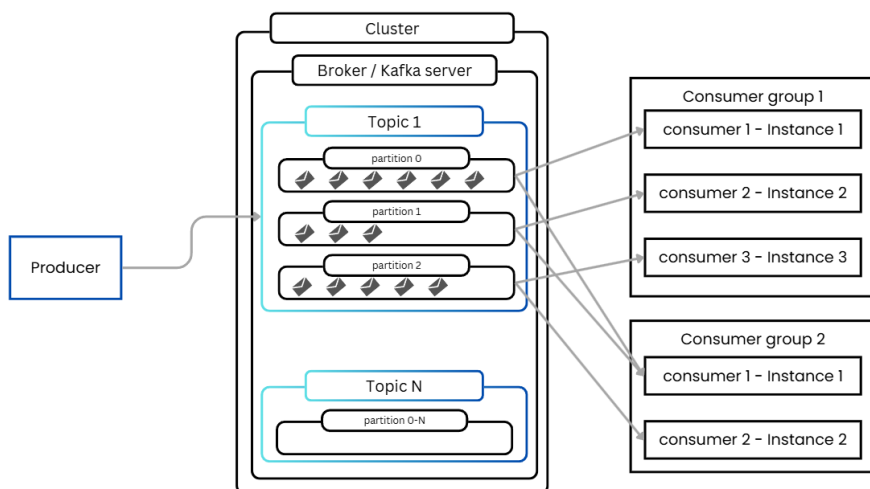
Como hemos visto hasta ahora, vamos a publicar mensajes desde nuestra aplicación *productor* al *topic*, si bien esto es cierto, y en la gran mayoría de entrevistas es todo lo que necesitas saber, está muy simplificado porque técnicamente Kafka está corriendo en un servidor al cual llamaremos *bróker*, y este *broker* está corriendo en un *cluster*, que es una agrupación de *brokers*.

Es cierto que cuando decimos que vamos a publicar a un *topic* estamos simplificando que lo que se hace es ir a una partición que está dentro de un *topic* que a su vez está dentro de un *broker*. Pese a dicha simplificación es muy importante saber los términos y cómo funciona para entender Kafka, ya que las particiones son donde físicamente se almacenan los mensajes que están en Kafka, y cada una de estas particiones en Kafka tiene lo que se denomina un *offset*, que no es más que metadatos en Kafka que representa la posición de un mensaje en una partición en particular. Al ser este *offset* individual de cada partición, significa que cada una de estas particiones puede tener un número diferente de eventos en cada una.



En el lado del consumidor tenemos una situación similar. Hemos mencionado que en Kafka los mensajes son consumidos por todos los consumidores. Esto funciona gracias a los *consumer groups*, donde estos hacen referencia a cada una de tus aplicaciones o microservicios.

Podemos entender que cada microservicio es un *consumer group* donde los consumidores son cada una de las instancias de nuestro servicio. Algo muy importante es que dentro de un único *consumer group* no podemos tener dos consumidores de la misma partición dentro del mismo *consumer group*.

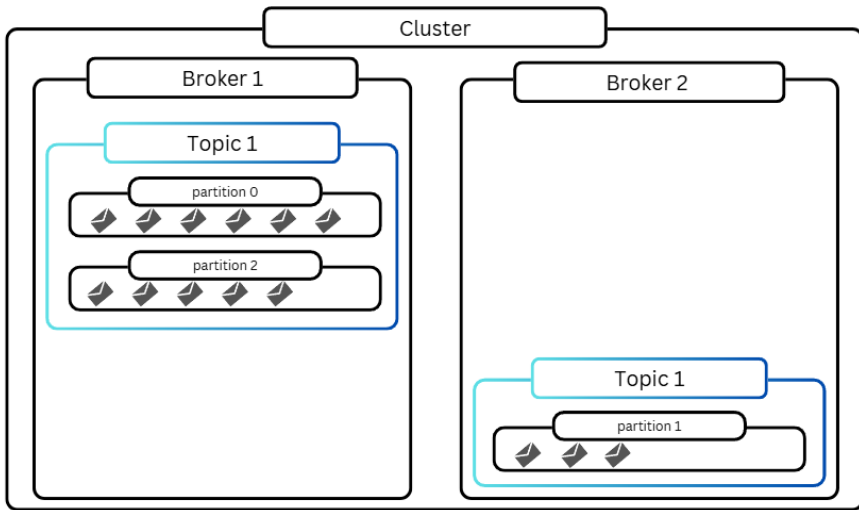


Lo que quiere decir es que si tenemos más consumidores que particiones vamos a tener consumidores que no van a consumir nada, y, en caso de tener menos consumidores que particiones, lo que va a suceder es que cierto consumidor va a consumir más de una partición.

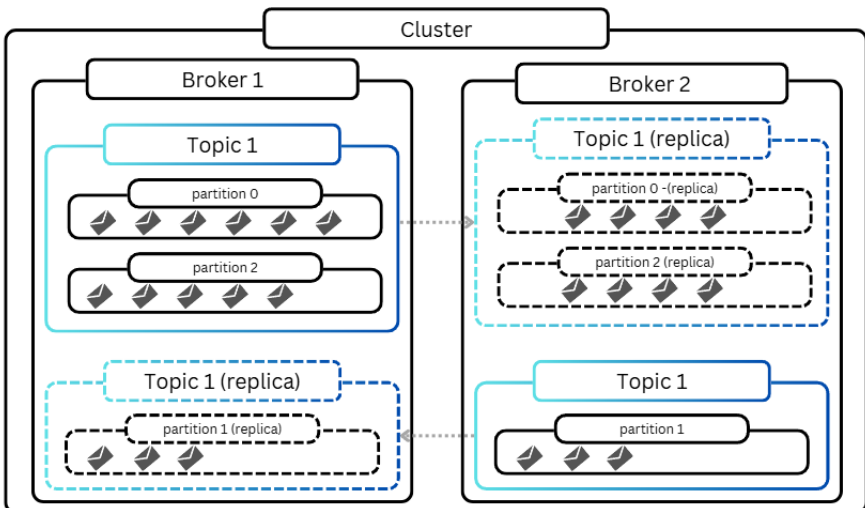
Tolerancia a fallos en Kafka

Al diseñar un *cluster* en Kafka hemos visto que podemos tener múltiples *brokers*, los cuales son un servidor individual para cada uno. La idea de estos brokers es que sean servidores individuales, de nada nos sirve tener múltiples brokers en un mismo servidor. Para eso tienes uno y ya está.

Aquí es donde entra la tolerancia a fallos, ya que podemos tener un *topic* redistribuido entre múltiples *brokers* (**partition sharding**) y, de cara, tanto al consumidor, como al productor de los eventos, funciona de forma transparente, por lo que les da igual si está todo en un servidor o en cincuenta.



Igual que el caso de antes, aquí tenemos el *topic 1* repartido entre dos brokers diferentes. Pero no solo eso, porque de igual manera podemos tener réplicas de nuestros *topics* y particiones, los cuales funcionan como maestro/esclavo, aunque en el mundo de Kafka se le suele llamar *fallover*.



Nota: normalmente Kafka distribuye las particiones de *topics* en de la forma más igualitaria posible; si tienes tres particiones y tres *brokers*, por defecto pondrá una partición en cada *broker*, aunque, obviamente, esto puede ser sobrescrito de forma manual.

Administración de los metadatos en un cluster de Kafka

Te puedes preguntar quién o qué sistema es el que controla toda esta información.

Al utilizar Kafka, necesitaremos un sistema que nos permita coordinar todos los elementos que tenemos dentro de un *cluster*, detectar cuando algo falla, mantener información sobre dónde está cada partición, replicar los mensajes entre el maestro / esclavo, hacer el cambio si algo falla, etc.

Para almacenar y administrar toda esta información tenemos dos opciones: utilizar Zookeeper, que hasta 2021 era la única opción, o, ahora, Kafka puede ser configurado con KRaft, que es una versión nativa desde Kafka, pero aquellas empresas que usen Kafka desde hace muchos años posiblemente no puedan adaptar pues no es totalmente compatible.

Cada vez que leemos un evento con un consumidor dentro de un *consumer group*, Zookeeper/KRaft mantiene la información al respecto en una variable llamada *committed offset*.

consumer	Consumer group	topic	partition	Committed offset
Consumer 1	Consumer group 1	Topic 1	Partition 0	5

Ese 5 en el campo *committed offset* significa que ese consumidor ha leído cinco de los seis mensajes que tiene partición, y debe de leer el último.

El motivo por el que mantenemos esta información en administrador es porque, en caso de que un *consumer* se caiga, será este administrador (KRaft/Zookeeper) quien asignará la partición que se ha quedado sin consumidor a otro consumidor para que este siga procesando los mensajes, y como es el propio administrador quien contiene toda la información del procesamiento de los datos, no habremos perdido nada.

Eventos en Kafka

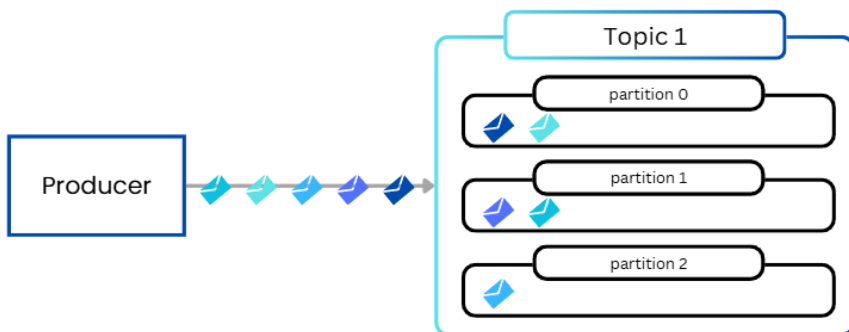
Cuando publicamos un evento en Kafka tenemos que publicarlo sí o sí con una cierta información:

- **Key:** es un Identificador, pese a que no tiene por qué ser único. En mi opinión debería ser único para cada mensaje, por lo menos único en cada *topic*, aunque dependerá sobre todo el caso de uso.
- **Value:** el contenido del evento que queremos enviar y que el consumidor recibirá.
- **Topic:** el *topic* al que estamos mandando un Evento. Si enviamos el evento a un *topic* diferente lo que pasará es que, o bien se perderá, o saltará como un error en los consumidores.
- **Partition:** Partición a la que irá el evento.

En muchos casos y empresas trabajamos con abstracciones que abstraen tanto la *Key* como la *Partition*, así que lo que siempre vas a tener que especificar en algún sitio es el mensaje y el *topic*; en otras palabras, el *evento* y dónde lo quieres publicar.

La *partición*, pese a que puede ser especificada de forma manual, lo normal es dejarla en blanco, y lo que sucede es que hace un *hash* de la *Key* y, basándose en el resultado del *hashing*, asignará una *partición*.

En el improbable caso de que tanto la *Key* como la *partición* no estén especificadas, los eventos son asignados a las diferentes particiones utilizando un algoritmo Round Robin, como el que vimos en la explicación del *Load Balancer*.



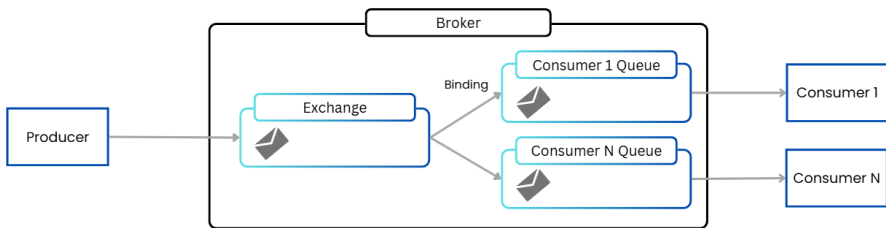
Finalmente, hay que mencionar que Kafka tiene un ***Pull Based approach***, esto quiere decir que es el consumidor el que va a estar haciendo peticiones al servidor de Kafka para ver si tiene algún mensaje nuevo constantemente. Esto tiene una implicación muy importante, y es que permite pedir más de un mensaje cada vez; por ejemplo, puedes consumir mensajes de 10 en 10 o de 100 en 100. A esto se le llama ***Batching***.

RabbitMQ en detalle

Ahora vamos a ver RabbitMQ, donde su lógica interna es algo más simple, aunque, por dicho motivo, es algo más limitada.

La idea general de RabbitMQ es igual que la de Kafka, mandar el mensaje del productor a él o los consumidores. Como hemos mencionado antes, RabbitMQ funciona conectando un *Exchange* a una *cola*, por lo tanto, sigue la misma lógica que hemos visto en el apartado de “combinar un Event Bus con colas”, y el vocabulario es el mismo, al grupo de *Exchanges* se les llama *Broker*, que viene siendo el servidor de RabbitMQ.

La diferencia es que el productor publica los eventos en el *Exchange*. Este *Exchange* hace las veces de un *Event Bus* y lo publica en los distintos consumidores que son las *colas*. A esta vinculación entre el *Exchange* y las diferentes colas le llamamos *binding* y las *colas* envían los mensajes a los diferentes consumidores.



Igual que pasa en cualquier otra cola, los mensajes persisten hasta que son enviados a un consumidor, en el caso de RabbitMQ, es el propio RabbitMQ el que envía los mensajes al consumidor, ya que cada consumidor se tiene que suscribir a cada una de las colas. Por lo tanto, RabbitMQ tiene un **Push Based approach**, y la forma en que los consumidores se conectan al servidor es utilizando el protocolo AMQP (*Advanced Message Queuing Protocol*).

Por defecto, RabbitMQ envía los mensajes uno a uno tan pronto como están disponibles en la cola, pero no envía un segundo mensaje al mismo consumidor hasta que este devuelve un ACK indicando que ha terminado de trabajar con el mensaje anterior y todo ha sido correcto. Esta

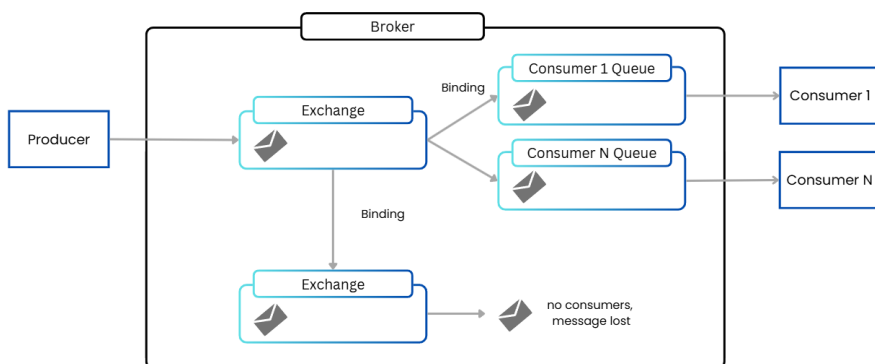
configuración se puede cambiar para especificar cuántos mensajes simultáneos quieres ser capaz de procesar.

Y en el caso de tener varias instancias suscritas a una cola, por defecto, RabbitMQ distribuye los mensajes utilizando el algoritmo *Round Robin*.

Vinculación entre colas y Exchanges

A la hora de vincular (*binding*) colas y Exchanges lo podemos hacer de múltiples formas, o siguiendo múltiples reglas.

La primera de todas es que un *Exchange* puede estar vinculado a otro *Exchange* el cual puede seguir reglas completamente diferentes, al fin y al cabo, ese *Exchange* puede estar pendiente de otra parte del dominio del sistema.



Dichos *Exchanges* actúan como un *Event Bus*, no mantienen los eventos que pasan por ellos y en caso de que no tengan consumidores los eventos simplemente se pierden.

Tipos de Exchanges

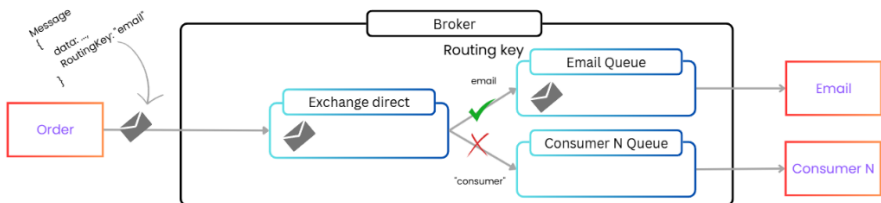
Dentro de RabbitMQ podemos configurar los *Exchanges* de diferentes formas, los cuales van a definir el tipo de *binding* que va a ser generado entre los diferentes elementos y también van a definir ciertos campos, dentro de cada uno de los eventos que el productor envía al *Exchange*.

Esta configuración se basa, principalmente, en un campo llamado *Routing Key*, el cual podemos entender como una llave de dirección, es una propiedad que enviamos al mensaje, junto con los propios datos, la cual va a ser comprobada por los diferentes *binding* para saber que tiene que hacer en cada uno de ellos.

Esta *Routing Key*, en la gran mayoría de situaciones, va a hacer referencia, o bien a la parte de dominio del sistema donde va destinado dicho evento, o bien a la parte de dominio de sistema donde ese evento se genera.

Exchange tipo Direct

Este tipo de Exchange está ligado directamente a la *Routing Key*, esto se debe a que la *Routing Key* del mensaje tiene que hacer un *match* exacto con la *Routing Key* que tendremos entre el Exchange donde publicamos el mensaje y la cola que escucha a dicho Exchange.



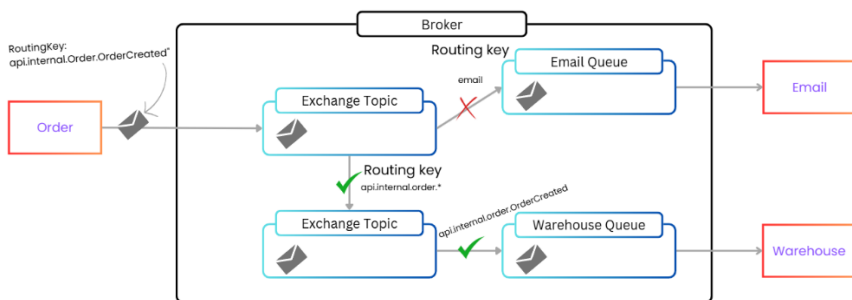
En caso de que el *match* de la *Routing* no sea exacto, el mensaje NO se propaga.

Exchange tipo Topic

Cuando usamos el tipo *Topic* vamos a hacer un match utilizando *wildcards* (similar a una expresión regular), las cuales permiten añadir configuración extra.

Como hemos explicado antes, esta Routing Key suele hacer referencia al dominio de la empresa, por lo tanto, es muy común tener un *Exchange*, el cual hace referencia a cada uno de los dominios de la empresa y de ahí se enlaza con más *Exchange* o colas.

En el caso de pedidos, tenemos que escucharlos desde el dominio de almacén por lo que RabbitMQ va a hacer las veces de modo de transporte entre dominios.



Hay dos tipos de *wildcards* posibles:

- Asterisco (*): Que cubre una única palabra en la Routing Key.
- Hash (#): Cubre cero o N palabras en la Routing Key.

En el ejemplo de antes, el mensaje envía la *Routing Key* *Api.Internal.Oder.OrderCreated*

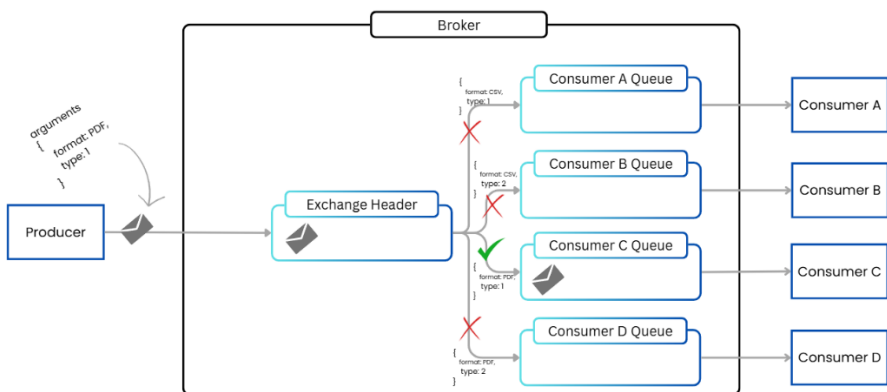
Si tenemos un binding con una wildcard que sea “*.Order.OrderCreatd”, NO haría un match, ya que la routing key enviada que se está comparando tiene dos palabras.

Pero si el binding fuera “#.Order.OrderCreated”, Sí haría un match, ya que el hash cubre ambas palabras.

Exchange tipo Header

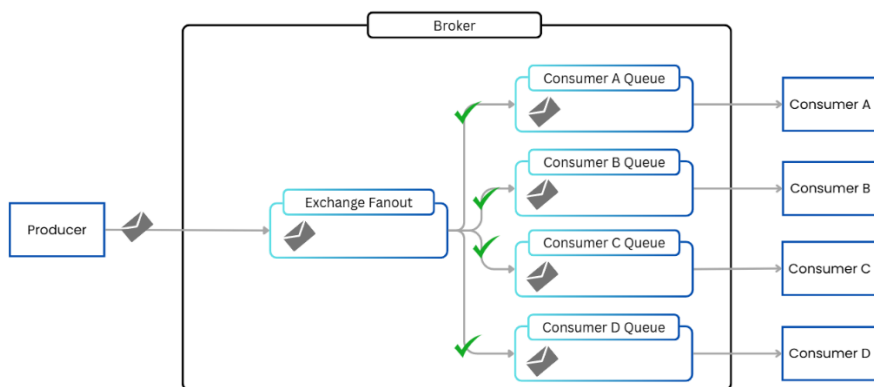
En este caso ya no utilizamos la *Routing Key*, sino que utilizamos la propiedad llamada *header*, que, a diferencia de la *routing key*, puede contener más de un argumento, de hecho, podemos poner tantos como queramos.

Vamos a ponernos en el supuesto de que tenemos un servicio que envía ficheros en diferentes formatos y esos ficheros tienen que ser parseados y leídos. Asumiendo que tenemos varias variaciones, por ejemplo, formato A, B y tipo de fichero CSV y PDF, tendremos un escenario como el siguiente:



Exchange tipo Fanout

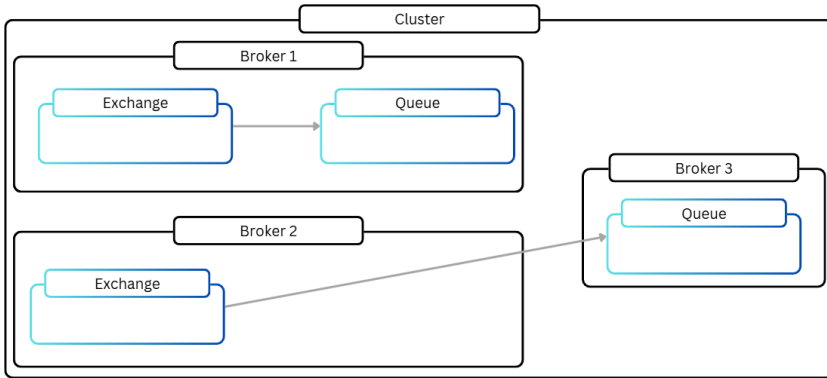
Finalmente, tenemos el Exchange tipo Fanout, el cual propaga los mensajes por todos los binding, sin importar el Routing Key, o ninguna otra configuración.



Clustering en RabbitMQ

Cuando un único servidor, o bróker, se nos queda pequeño llega el tiempo de ampliar, ya bien sea el tamaño de dicho servidor, donde no tendremos que hacer nada más, o levantar un nuevo servidor y enlazarlo al que ya teníamos. A esta acción se le llama **clustering**.

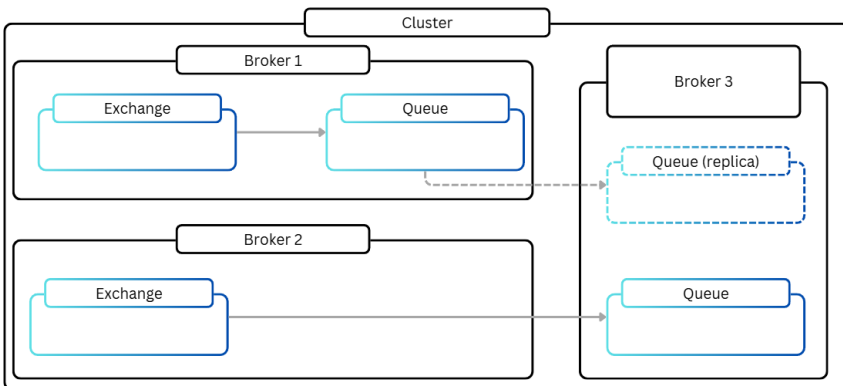
Algo que hay tener en cuenta es que cada broker (server) actúa de forma independiente con sus Exchanges, colas y binding, pero estos están disponibles desde otros brokers.



La comunicación y los metadatos de los diferentes *brokers* del *cluster* se almacenan internamente en RabbitMQ, teniendo cada *broker* un nombre único ya que es como se identifican y relacionan entre ellos.

Cuando tenemos un nuevo *broker* para unirse al *cluster*, lo que hacemos es configurarlo para que tenga la información de un nodo existente y este nodo replica la información del resto de nodos existentes.

De igual forma que tenemos múltiples servidores, podemos tener colas duplicadas. Estas colas se utilizan únicamente como réplica en caso de que, si un nodo falla, los mensajes siguen estando disponibles.



Entrevista de diseño Junior

A lo largo de este libro vamos a ver diferentes niveles de posibles entrevistas. Al final son ejemplos prácticos reales que van a ayudarte a conseguir experiencia en situaciones reales y prepararte para una posible pregunta de entrevista real.

Normalmente, para puestos *Junior* no solemos tener una entrevista única de diseño, principalmente porque no tiene sentido. Lo que sí tiene sentido es hacer pequeñas preguntas o supuestos cortos donde podemos ver qué es lo que de verdad sabe la persona a la que estamos entrevistando. Este tipo de preguntas o mini entrevista de diseño suele estar dentro de la entrevista de una hora que te hará la empresa, y, lo normal, es que te pregunten solo cosas que pones en tu CV que sabes, así que, si no pones que sabes sobre sistemas distribuidos o pub/sub, es muy probable que no te lo pregunten.

En cualquier caso, vayamos al lío.

Diseño de un sistema de reportes para usuarios de venta.

“En el supuesto actual estamos en una empresa donde únicamente creamos aplicaciones internas, esto significa que todos los usuarios de nuestra aplicación son trabajadores de la misma empresa en la que estamos.

Dada esta información, diseña un sistema capaz de generar reportes para los usuarios de ventas con los pedidos y los productos realizados dentro de un rango de fechas proporcionado por los usuarios.”

Este puede ser el supuesto que te den. Lo que se espera aquí no es que te pongas a diseñar como un loco sin cabeza, eso te puede servir para un primer boceto, pero lo que se espera de verdad es que realices una serie de preguntas que te permitan entender el problema y cuáles son los límites que tienes que aplicar.

Cuando estás en una entrevista, el entrevistador sabe que no puedes leer la mente, así que se siente libre para realizar cualquier pregunta. Debes saber que ellos quieren ayudarte tanto como tú quieres que te ayuden.

Candidato: ¿Cómo obtenemos los datos? ¿La empresa dispone de un sistema monolítico, uno de microservicios, o tenemos una base de datos con su propio sistema separado donde puedo consultar la información?

Entrevistador: Tenemos un sistema de microservicios con los límites de cada aplicación muy bien definidos. Para conseguir la información de los productos tienes que ir a través del microservicio de productos, lo mismo para los pedidos.

Candidato: ¿De qué forma acceden los usuarios al sistema?

Entrevistador: Tenemos una web interna donde los usuarios inician sesión con su usuario y contraseña. Toda la información de los usuarios

está en un servicio separado y usamos un *bearer token* para la autorización.

Candidato: ¿Cuál es el volumen de pedidos que tenemos?

Entrevistador: Somos una empresa líder en el sector, donde vendemos cientos de pedidos por hora. En épocas concretas como el Black Friday pueden llegar a ser miles. Se puede decir que tenemos un gran volumen de pedidos y generar un reporte pueden ser varios minutos.

Candidato: ¿Hay alguna otra característica requerida en la implementación que el equipo de ventas haya pedido?

Entrevistador: Debemos ser capaces de filtrar por tipo de envío entre otros campos, pero eso está fuera de lo que se pide para este problema, así que no.

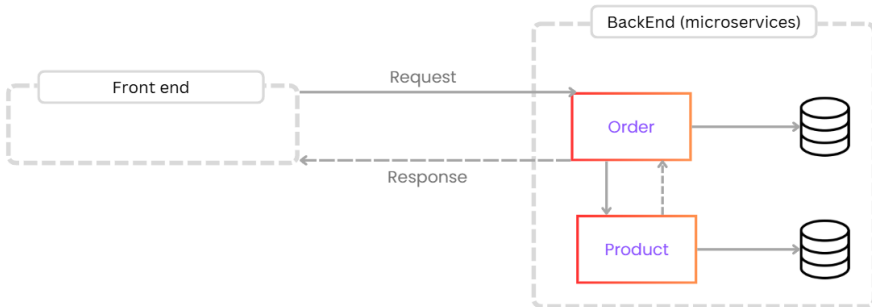
Con estas cuatro preguntas que se hacen en menos de un minuto hemos pasado de tener una ligera idea de lo que nos pide el problema, a comprender cuál es el sistema actual de la empresa, por lo que la solución va a ser acorde al sistema actual lo que es ideal.

A la hora de empezar a responder con nuestro diseño se puede hacer de varias formas, o, mejor dicho, tú puedes empezar con una solución básica y a partir de ahí ir iterando, haciendo preguntas al entrevistador.

En cualquier caso, recuerda ir pensando en voz alta, para que el entrevistador te escuche. Pensar en voz alta sirve para dos cosas: la primera, que el entrevistador comprenda tu forma de pensar. La segunda, si te equivocas, te van a corregir o añadir una anotación para que vayas por el buen camino.

Una solución al problema podría ser lo siguiente:

Desde el *FrontEnd* realizamos una llamada al *BackEnd*, al servicio de pedidos donde leemos todos los pedidos con sus productos y montamos el reporte en el *FrontEnd*.

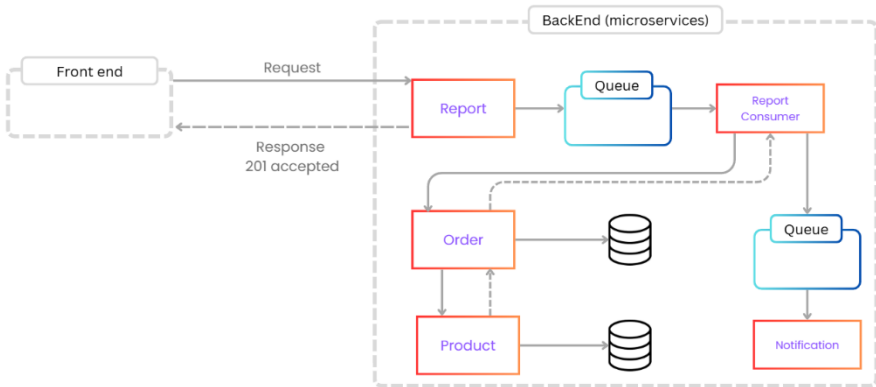


Entrevistador: Si bien esta solución podría funcionar tiene una experiencia de usuario terrible ya que los reportes pueden tardar varios minutos. Además, ¿qué es lo que sucede si el usuario cierra la pestaña o el navegador? Por ejemplo, se va a hacer todo el trabajo, pero se va a perder, por lo que es un gasto de recursos.

Candidato: Entonces lo que hacemos es recibir la petición del usuario, ejecutar el reporte por detrás y devolverlo cuando esté listo utilizando una cola.

Entrevistador: Exacto.

Esta pequeña conversación implica que vamos a tener que utilizar comunicación asíncrona, ya que vamos a devolverle al usuario una respuesta inmediata, pero realizando el proceso por detrás. En una entrevista da igual si usas o dices cola, *Event Bus*, o, incluso, el nombre de la aplicación. Quizá depende de cómo vayas de tiempo, te hagan alguna pregunta al respecto, pero con que sepas lo general, que sirve para mandar un mensaje de un punto a otro, sobra.



Candidato: En esta implementación lo que estamos haciendo es devolverle al usuario instantáneamente que hemos aceptado la petición, habiendo comprobado si es un usuario de ventas pidiendo un reporte de ventas.

Mandamos esa petición a una cola, la cual tiene un consumidor que es quien genera el reporte. Cuando termina, generamos un evento llamado *ReportGenerated* con la información necesaria para que el servicio de notificación lea el evento y notifique al usuario.

Entrevistador: En ese evento que mandas a la cola de notificación, ¿qué información estás mandando?

Candidato: La información de la petición del reporte con los filtros, el usuario que lo ha solicitado y el propio reporte.

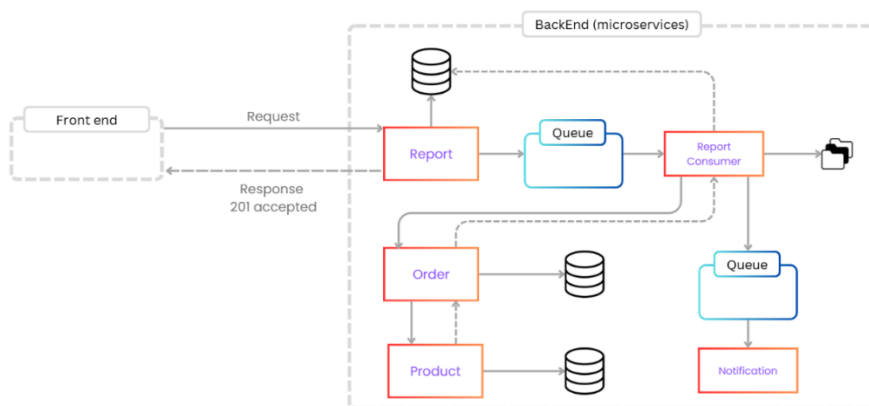
Entrevistador: Los reportes pueden ser muy grandes, lo ideal es almacenar el reporte en un lugar físico, en vez de ir mandando en los eventos ya que los eventos grandes generan problemas de rendimiento, latencia, escalabilidad, almacenamiento etc. Pero bien. Última pregunta. ¿Qué es lo que sucede cuando un usuario pide el mismo reporte cinco veces?

Candidato: En la solución actual, se genera el mismo reporte cinco veces.

Entrevistador: ¿Puedes cambiar el diseño para solucionar este problema? incluye también lo que hemos mencionado de no enviar el reporte en el propio evento.

Para esta solución lo que necesitamos es una base de datos y un sistema de ficheros adicional.

En la gran mayoría de casos estas bases de datos, y el sistema de ficheros, son los propios y existentes ya por parte de la empresa, pero en una entrevista especialmente *Junior*, una explicación sobre si es una base de datos compartida, o exclusiva para los reportes no es necesaria, a no ser que te la pregunten. Más adelante en el libro veremos este punto.



En la solución final tenemos el sistema de reportes conectado a una base de datos, la cual contiene la información de los reportes solicitados al sistema, donde, como clave, tenemos el tipo de reporte y los filtros que se han solicitado. Además, mantenemos varios campos adicionales, uno con el estado del reporte, otro con la ubicación del fichero generado, y otro con la fecha y tiempo de la última actualización.

Cuando una nueva solicitud de reporte llega al sistema, creamos un nuevo registro, con un estado de “pendiente” y generamos el evento para que el consumidor genere dicho reporte. La diferencia es que, si una solicitud igual llega, lo que hacemos es seguir devolviendo el código de estado 201, pero esta vez, comprobamos primero en la base de datos si dicho reporte ha sido solicitado o no, por lo que no generamos reportes adicionales.

El consumidor de reporte lee el evento de la cola, e inicia el proceso de generación de dicho reporte, lo primero será actualizar el valor en la base de datos a “Generando” junto con la fecha y hora actual.

Cuando el proceso termina, almacenamos el fichero en una carpeta en el sistema de ficheros físico de la empresa, donde los usuarios con acceso pueden acceder. El usuario accede directamente a ese fichero sin pasar por ningún microservicio adicional y dependiendo que tipo de sistema de ficheros tengas (uno propio de la empresa, el de AWS, Azure o Google), esa configuración se hace de una forma u otra, en cualquier caso, no es responsabilidad del sistema de reportes.

Después de almacenar el fichero, cambiamos el estado a “Generado” y actualizamos la fecha y la hora, así como generar el evento *ReportGenerated*, el cual contendrá el enlace al fichero será leído por el sistema de notificación.

Aquí tenemos una cosa importante, si nos vuelve a llegar una petición para generar el mismo reporte, podemos devolver un 201 *accepted*, pero con un *header Location*, el cual contiene la ruta al fichero.

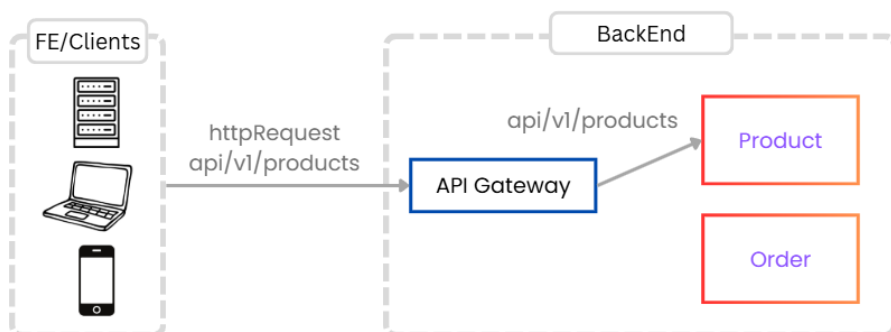
CAPÍTULO 7

API Gateway

Una API Gateway, o patrón API Gateway, es una capa que añadimos a nuestro sistema para que todas las *request* que vienen desde el *FontEnd*, o desde clientes externos, pasen por ella en lugar de ir directamente a los servicios del *BackEnd*.

Ahora veremos más funcionalidades, pero la funcionalidad principal de un API Gateway es actuar como un reverse *proxy*, y enrutar cada llamada que viene desde fuera del sistema hacia el microservicio correspondiente.

Esto significa que un API Gateway actúa como punto de entrada único al sistema. De cara a los consumidores externos, la API Gateway es el único punto de autoridad del sistema siendo la ruta de la URL lo único que cambia, aunque internamente sea completamente diferente.

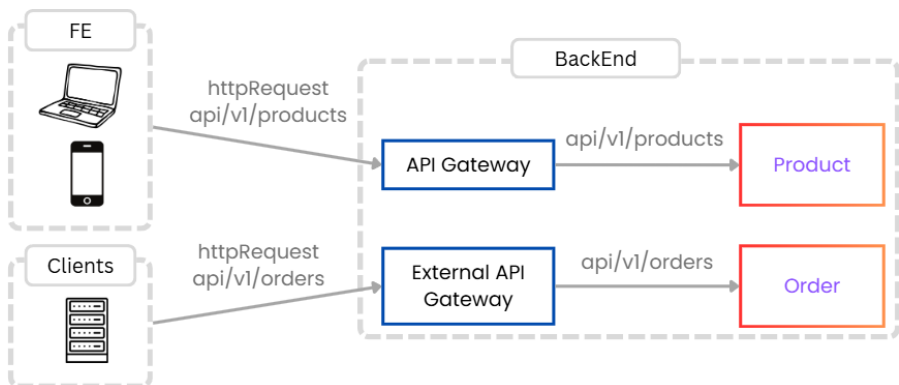


Como vemos, una HTTP Request llega a nuestra API Gateway y esta es capaz de identificar dónde tiene que ir.

Es muy común en empresas tener varias API Gateway, donde una va a ser para todo el tráfico interno de la empresa y de las aplicaciones móviles o web, mientras que hay otra API Gateway que será donde cualquier cliente externo se conecte. La idea entre ellas es la misma, únicamente cambiará cierta configuración.

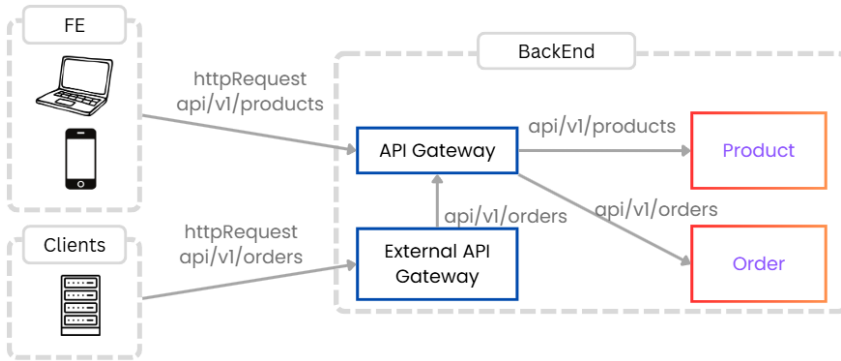
Técnicamente, podemos usar una única API Gateway, pero en casi todas las empresas que he estado utilizan una separada para los clientes externos para así poder realizar acciones sobre ellos sin tener que afectar al resto del sistema.

En mis años de experiencia he visto tanto tener dos API Gateway corriendo en paralelo de esta forma:



Así como tener dos API Gateway conectadas de forma secuencial, donde las aplicaciones internas llaman a la API Gateway y las aplicaciones

externas llaman a una API Gateway diferente la cual enruta a la API Gateway interna.



En ambos casos la idea es la misma, tener a los clientes separados para poder tener un control más granular de los mismos sin afectar al trabajo interno.

Al margen de en la configuración que quieras poner, donde ambas soluciones cambian es en el tema de la autenticación, ya que usando la primera opción, donde tenemos dos API Gateway podemos tener dos sistemas de autenticación completamente diferentes, mientras que, si la API externa llama a la interna, debemos de tener en cuenta que la API interna, o bien es compatible con la autenticación externa, o la autenticación externa es compatible con la interna.

Autenticación y autorización

Uno de los puntos más importantes dentro de una API Gateway es actuar como un portero de discoteca; parar a todo el mundo que no tenga credenciales para entrar. Y para ello utilizamos un sistema de

credenciales. Lo más común es utilizar un *bearer token* (OAuth/SAML), pero no es la única manera.

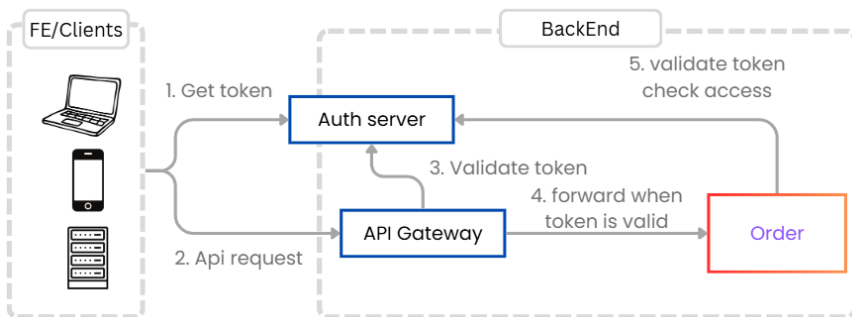
Tenemos dos formas de afrontar la seguridad en una API Gateway.

La primera es comprobar si el usuario, o cliente, tiene un token válido del sistema, pero no comprobaremos que tenga acceso a dicho recurso. Esta comprobación será en la propia API que devuelve el recurso.

La segunda es comprobar la autenticación y la autorización para dicho recurso en la API Gateway. De esta forma, si alguien no está autorizado para ejecutar dicho recurso, la llamada nunca llegará al servicio.

Como puedes imaginar, la segunda opción es mucho más compleja a la hora de administrar, ya que tenemos que sincronizar los permisos entre los diferentes servicios del *BackEnd* y la propia API Gateway, pero, a su vez, podemos reducir una gran cantidad de solicitudes.

La forma más común de estar autenticado es utilizando *bearer tokens*. Esto incluye tanto a los usuarios internos como a los externos. Aquí vemos lo que sería el flujo normal de autenticación:



En este escenario he puesto el *Auth server* en el propio *BackEnd*, pero en muchos casos puede ser un servicio externo y podría no solo estar fuera del *BackEnd*, sino en un sistema o empresa completamente distinto.

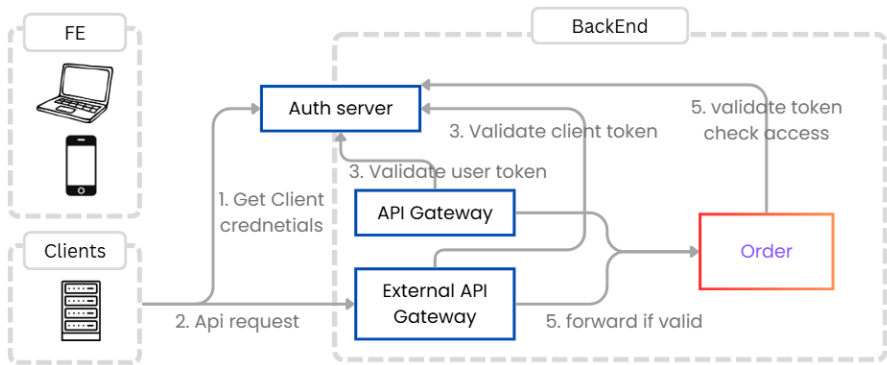
En este flujo, la API Gateway comprueba que los clientes realizan la llamada con un *token* válido generado por nuestro *Auth server*, la API Gateway valida dicho token y enruta la llamada al microservicio correspondiente y, de ahí, el microservicio vuelve a validar el token y comprueba la autorización en dicho token para comprobar que se tiene acceso a los diferentes recursos.

Credenciales de clientes

Cuando navegamos por Internet nos acostumbramos a tener que crear un usuario y una contraseña, los ponemos en un formulario de *login* y entramos en un sistema. Esto funciona bien para los humanos, pero qué sucede cuando trabajamos con máquinas.

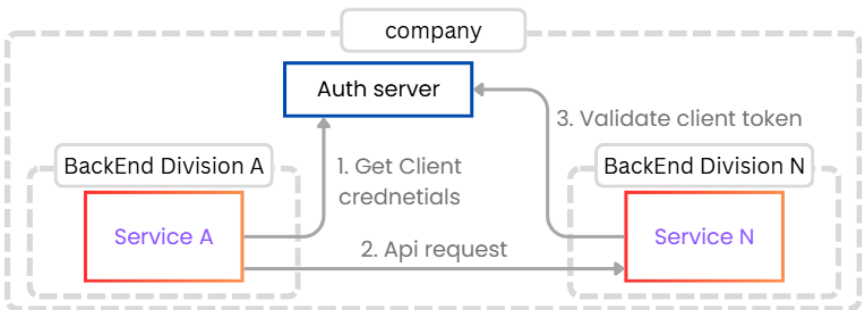
La idea pasa a ser similar pero no la misma. Cuando tenemos clientes externos, la comunicación es máquina a máquina y como es diferente a usuario-servidor deberíamos tener dos tipos diferentes de autenticaciones.

En el caso de los usuarios, estos escriben su usuario y su contraseña, mientras que, en el caso de la máquina, proveemos a nuestros clientes de unas credenciales de cliente, las cuales van a ser utilizadas para conseguir un Access Token; el cual puede ser utilizado por los clientes durante su tiempo de vida, que normalmente estará en un rango de entre 6 y 24 horas.



La diferencia es que, en este caso, desde la API Gateway interna estamos validando si el token es un token de usuario, mientras que desde la API Gateway externa validamos si es un token de cliente. En ambos casos solo enrutamos la llamada si es un token válido. Y, en ambos casos, el propio microservicio debe volver a comprobar las credenciales.

NOTA: depende del tamaño de la empresa y de cómo esté dividida internamente. Es posible que entre ciertos componentes del *BackEnd* también tengas que utilizar credenciales de cliente, ya que es posible que, aunque estéis trabajando a dos metros, tú seas de la División A y otra persona sea de la División B, y, si por lo que sea necesitas consultar información en dicha División, lo más probable es que haya una propia separación física de las aplicaciones, lo que implica que necesitas credenciales de cliente.



Dependiendo de la implementación de cada sistema, la *request* a la API debería ir a través de una API Gateway, o no, pero lo que pretendo ilustrar en esta imagen es el uso del *Auth server* para llamadas dentro de la misma empresa.

API KEY

Es común ver también el uso de API Keys, aunque cada día lo vemos menos, ya que su uso es similar al de las credenciales de cliente, pero con muchas más limitaciones que hacen que sea una opción menos atractiva en la mayoría de los escenarios.

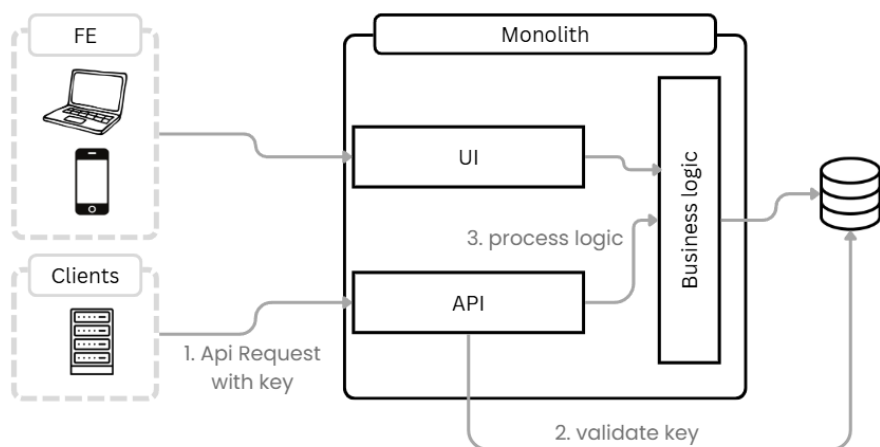
Un API Key es un **identificador único** que sirve para identificar y autenticar una *request* en una aplicación. Esta Key se utiliza para permitir el acceso a una API. Pero, a diferencia de las credenciales de cliente, esta Key no contiene información de autorización. Cuando una Key es válida lo es para todo el sistema donde permitimos Keys.

Por lo tanto, si tenemos una key válida, tendremos un acceso total a todos los recursos de dicha API.

Una de las características de las API Key es que es única y es enviada en cada request, lo que implica que para identificar al sistema que está realizando la llamada lo haremos con dicha API Key. Internamente debemos tener mapeadas, las Keys al cliente o al entorno que pertenece para poder hacer la identificación de una forma correcta.

En un sistema distribuido donde todo está separado en microservicios y con separaciones y configuraciones claras y sencillas, no solemos ver este tipo de Keys, pero sí son muy populares en dos situaciones adicionales.

En el mundo de los monolitos, tenemos un monolito enorme donde trabajamos con sesiones y ciertos clientes necesitan acceso para conseguir los reportes del día anterior, o información varia. En vez de cambiar completamente nuestro sistema, lo que muchas veces se hace es crear una serie de APIs dentro del propio monolito, las cuales aceptan API Key, y lo que hacemos es mandar a los clientes dichos *endpoints* y la forma para conseguir la API.



Otro escenario muy común es cuando tenemos servicios de terceros que son críticos en funcionamiento, pero no en datos, así como únicamente accesibles desde la red interna.

Aquí podemos incluir servicios como los utilizados para monitorización o logs del sistema.

Lo principal que hay que tener en cuenta con la API Key es que con ella se puede acceder a todos los recursos disponibles; no hay una diferenciación por permisos, lo que implica que, si el sistema crece mucho y quieres tener una separación de permisos con los clientes, este se vuelve difícil de mantener.

Otros servicios más famosos como ChatGPT o Youtube utilizan API Key. Esto es porque, una vez tienes acceso al sistema, tienes acceso al sistema completo y no hacen una diferenciación entre diferentes roles de usuario, o permisos, lo cual, en la gran mayoría de aplicaciones empresariales, es completamente inviable.

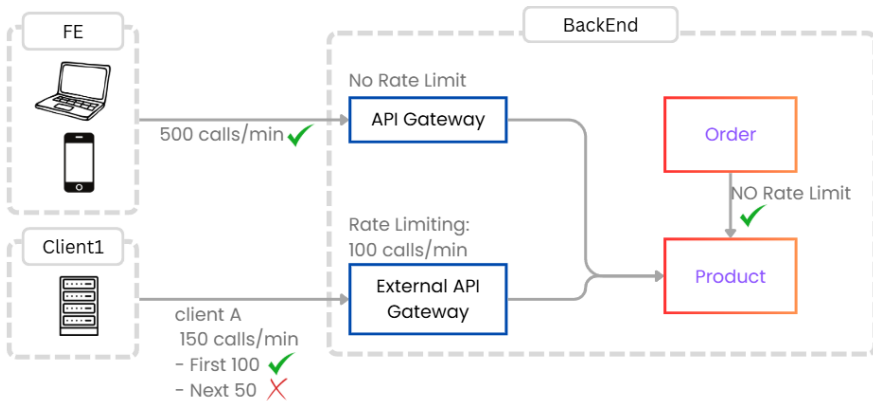
Esto lleva también a problemas muy serios si la API Key queda comprometida, ya que cualquier actor malicioso puede utilizarla a sus anchas hasta que uno de los miembros afectados se da cuenta y la bloquea o elimina del sistema.

Esto es debido a que la Key no tiene caducidad, funciona siempre y cuando no se elimine.

Rate Limiting

Otra de las características principales de una API Gateway es el Rate Limiting, o lo que es lo mismo, limitar el uso que los clientes hacen de nuestro sistema para evitar que se utilice más de la cuenta.

Técnicamente, podemos poner esta limitación en cada uno de nuestros microservicios de forma individual, pero esto no tiene mucho sentido, ya que, teniendo nuestro API Gateway como punto de entrada, lo podemos hacer directamente ahí. Además, es muy posible que el límite de uso que queremos dar a los clientes sea muy distinto al límite que queremos implementar para los usuarios de la web o del móvil, o, incluso, para el resto de los servicios de dentro de nuestro sistema que dependen del nuestro.



El motivo principal por el que necesitamos Rate Limiting es porque una vez tienes clientes en tu sistema, estos abusan del mismo. No lo hacen intencionadamente, o a malas, pero directamente lo hacen porque pueden y no piensan en las consecuencias que puede llegar a tener, lo cual puede llegar a ser un problema muy grande debido al uso excesivo sobre nuestra aplicación. Pudiendo llegar a causar una degradación parcial del sistema, o, incluso, que se caiga completamente.

Por ello, al implementar el Rate Limiting, estamos implementando un límite de uso de nuestra API, y cuando una request falla, el cliente sabe que falla por Rate Limiting ya que devolveremos el tipo 429, el cual significa *too many request* (muchas requests).

Tipos de Rate Limiting

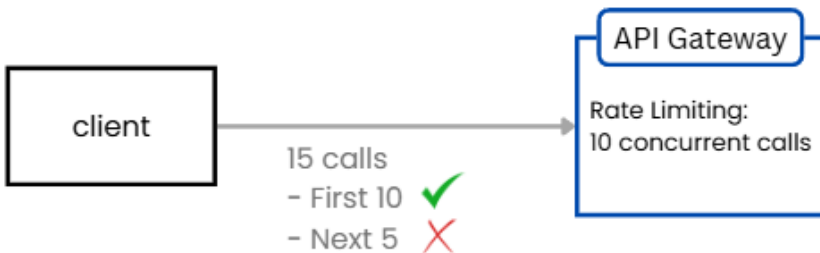
Dentro del Rate Limiting disponemos de varios tipos de configuraciones que podemos aplicar. Estas pueden ser tanto específicas, para un cliente en concreto, como para todos e, incluso, compartidas.

Por ejemplo: quieres que el Cliente A pueda hacer ciento cincuenta llamadas y el Cliente N pueda hacer cien. Otra opción es que todos los clientes puedan tener ciento cincuenta llamadas cada uno y, la última, es que entre todos los clientes puedan hacer mil llamadas por minuto.

Igual que puedes limitar por cliente, puedes limitar por recurso (un único *endpoint*), o de forma global. Y, si bien es cierto, que en la gran mayoría de sistemas puedes crear tus propios Rate Limitings, muchos sistemas vienen con una serie de tipos predefinidos por defecto que son los siguientes.

Límite de concurrencia

El primero es el más claro y sencillo. Podemos configurar cuantas request simultáneas queremos poder aceptar. Si permites diez llamadas simultáneas, las diez primeras se ejecutarán y la once dará un error.

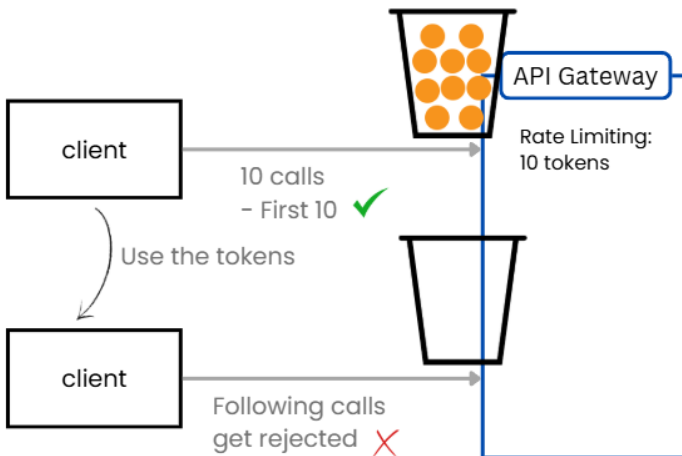


Eso sí, si la once llega después de que una de las diez llamadas originales ha terminado, la once también progresará sin problemas.

Límite de depósito de tokens

El nombre viene por su representación en el mundo real. En este escenario tenemos un límite de *tokens* y cuando los gastamos todos ya no podemos utilizar más el servicio.

Para nuestro ejemplo, vamos a simplificar donde cada *token* es una *request* a la API, por lo tanto, si tenemos diez *tokens*, y, después de gastarlos todos ya no tendremos más disponibles.

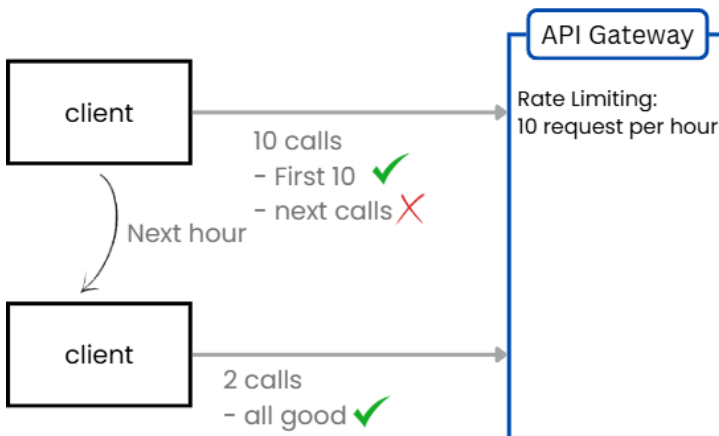


Este límite es muy común en aplicaciones prepago, donde adquieres cierto número de *tokens* antes de utilizar el servicio. Otro uso común es donde no queremos sobrecargar el sistema, permitiendo una regeneración de los *tokens* a un ritmo fijo. Por ejemplo, un *token* cada *x* minutos, con el máximo de diez y es el cliente quien decide si los quiere usar todos de golpe, o ir usándolos cuando se regeneran.

Límite de ventana fija

En este escenario definimos una ventana de tiempo, que puede ser minutos, horas, o, incluso, días, y un límite de solicitudes permitidas dentro de esa ventana.

Por ejemplo, si aceptamos diez peticiones por hora, cualquier petición que venga después la rechazamos hasta que pase la hora.

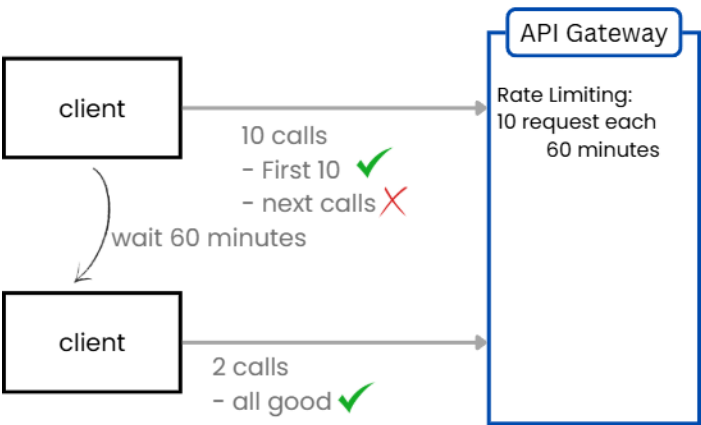


Este límite puede ser abusivo si un cliente realiza todas las llamadas posibles al final de una ventana y vuelve a realizar todas las llamadas posibles al inicio de la siguiente, ya que, técnicamente, las realiza todas en un corto periodo de tiempo.

Límite de ventana deslizante

Es similar al caso anterior, pero en vez de tener bloques fijos de tiempo, estos se calculan para considerar las solicitudes en un intervalo dinámico.

Para esta solución podemos definir un Rate Limiting también por segundos, minutos u horas. En nuestro caso de ejemplo, definimos diez llamadas cada sesenta minutos.

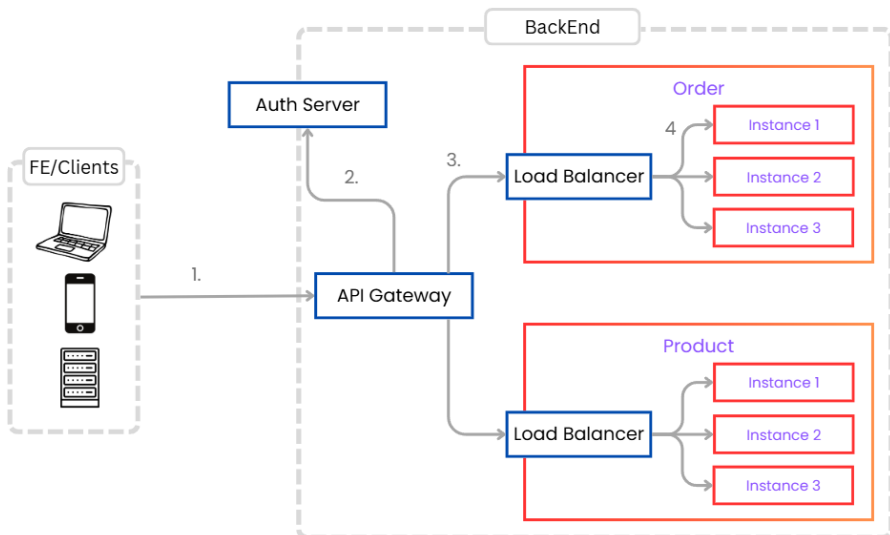


Diferencia entre API Gateway y un Load Balancer

Una pregunta muy común en las entrevistas de diseño es la diferencia entre una API Gateway y un Load Balancer. Como estás viendo en este libro las diferencias son bastante considerables.

Un Load Balancer se encarga de elegir qué instancia dentro de un mismo microservicio va a ejecutar la *request*, mientras que una API Gateway realiza muchas más acciones; enrutamiento, autenticación, Rate Limiting o, incluso, caché y monitorización de los clientes.

Ello quiere decir que en nuestro sistema vamos a poder trabajar con ambas tecnologías sin ningún problema.



En este ejemplo vemos el siguiente flujo:

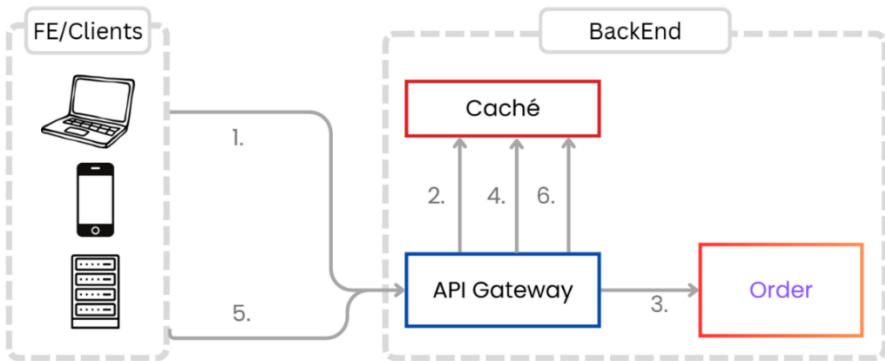
1. El cliente realiza una *request*, la cual llega a la API Gateway.
2. La API Gateway comprueba que el *token* enviado por el cliente es un *token* válido.
3. La API Gateway enruta la petición del usuario, con el servicio correspondiente, apuntando al punto de acceso de dicho servicio, en este caso el Load Balancer.
4. El Load Balancer selecciona, basándose en el algoritmo especificado, a qué instancia va a enviar la petición.

Como vemos, son dos herramientas completamente complementarias que no se pisan entre ellas.

Caché

Otra de las funcionalidades de una API Gateway es la posibilidad de incluir una caché. Lo que hacemos en esta situación es, cuando recibimos la llamada, devolvemos la información directamente desde la caché.

Esta funcionalidad nos puede ayudar a reducir el número de llamadas a los servicios del *BackEnd*, pero claro, solo va a funcionar con llamadas repetidas. Ojo, que no es tan extraño recibir la misma llamada una vez detrás de otra. No es ideal, pero puede pasar.



En esta imagen vemos como la segunda llamada se devuelve directamente de la caché.

1. La aplicación cliente realiza una llamada, la cual va a la API Gateway.
2. La API Gateway comprueba si dicha llamada y respuesta está en la caché.
3. Al no estar en la caché, recibimos la información del microservicio correspondiente.
4. Guardamos la información en la caché y devolvemos la respuesta al usuario.
5. El cliente realiza la segunda llamada.
6. Comprobamos si está en la caché, en este caso está disponible, así que devolvemos la información desde ahí.

La solución ideal en este escenario es una combinación de dos acciones:

1. Implementar un Rate Limiting, para que, si un cliente llama sin parar, eventualmente al cliente le de error.
2. Educar al cliente para que no realice la misma llamada una vez detrás de otra de forma constante.

Desafortunadamente, muchas veces hay contratos firmados, y esta acción simplemente no es posible, o es muy costosa en la parte del cliente, y la solución puede tardar doce meses, o incluso más. Así que, para proteger nuestro sistema, podemos incluir una caché por cierto tiempo.

Si, por ejemplo, tenemos diez clientes con cien pedidos, y consultar la información de los pedidos y los productos en un bucle una vez cada uno por minuto, significa mil llamadas por minuto. Si ponemos una caché de diez minutos, vamos a evitar que nueve mil llamadas cada diez minutos lleguen al *BackEnd*, y, lo más probable, es que no tenga un impacto real en el cliente.

Por supuesto, cada caso de uso es diferente, y debemos saber cuándo una caché es aceptable o puede ser un problema.

¿Por qué harán esto, te preguntas? Como he indicado antes, porque pueden. Normalmente no es con mala fe.

Monitorización de clientes

Una parte crucial de cualquier sistema distribuido es la monitorización, de la cual hablaremos más adelante. Por ahora, vamos a comentar como mantener un control de los clientes.

Hemos explicado, en el apartado de la caché, una situación donde tenemos diez clientes y cada uno realiza cien llamadas por minuto de

forma repetida. Ese será un escenario simplificado con un problema ya definido e identificado.

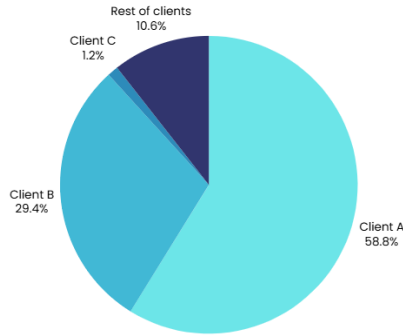
Pero claro, ¿qué es lo que sucede si llevamos poco tiempo con el sistema montado y nos levantamos un día por la mañana y vemos que el microservicio de productos y el de pedidos tienen una carga diez veces superior a la normal?

De hecho, no serán solo esos microservicios, cualquier otro servicio o piezas de infraestructura, como la base de datos, también necesitarán escalar para cubrir la demanda.

Así que, son las 6 am, no tienes usuarios en la web, pero tienes una carga diez veces superior a la normal. ¿Cómo encontramos el problema de raíz?

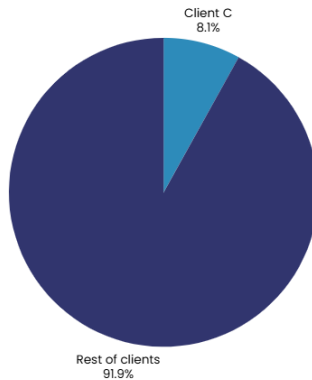
Aquí es donde viene la monitorización de los clientes. Cada vez que recibamos una llamada, debemos mantener una lista actualizada de qué cliente ha realizado dicha llamada, e, idealmente, deberíamos almacenar también el tiempo de respuesta y el cuerpo de la llamada. Pero para este ejemplo, nos sirve únicamente con el cliente.

Recuerda que en el apartado de autenticación y autorización vimos que cada cliente tiene un *client token* el cual contiene, o, debería contener, el identificador de dicho cliente. Pongámonos el supuesto anterior de tener diez veces más carga de la necesaria. Podemos ir al sistema de monitorización y ver algo así:



Tenemos un gráfico donde vemos como casi el 58.8% de las llamadas viene desde el cliente A, el cliente B copa un 29.4% de las llamadas, y un mero 11.8% lo ocupan entre el resto de los más de cien clientes de la empresa.

Lo primero que vas a hacer es comparar con el día o varios días anteriores, y te das cuenta de que hasta hace dos días el cliente C tenía una carga superior al resto por mucho, pero era únicamente del 8.1%, y ese 8.1 se ha convertido en un 1.2%, y siendo que su número de llamadas está estable, los otros dos han crecido exponencialmente.



Ten en cuenta que esta situación le está costando a tu empresa dinero, ya que, para cubrir la carga, necesitas muchos más recursos de los esperados.

Con la monitorización actual que tenemos, nos damos cuenta de que el problema viene directamente del Cliente A y del Cliente B.

Aquí es donde viene el tema de qué solución implementar, pero para implementar una solución necesitas saber qué uso le están dando a la API externa. Así que vas a la monitorización de los *endpoints* concretos y ves lo siguiente:

CLIENT	DATETIME	URI	STATUSCODE	RESPONSE TIME
CLIENT A	01/01/2025 00:00:01	GET orders/api/v1/orders/984857379	200	1500 ms
CLIENT A	01/01/2025 00:00:01	GET orders/api/v1/orders/745403785	200	1200 ms
CLIENT A	01/01/2025 00:00:01	GET orders/api/v1/orders/457473469	200	1100 ms
CLIENT A	01/01/2025 00:00:01	GET orders/api/v1/orders/984857379	200	1900 ms

De esta tabla podemos sacar varias conclusiones: la primera es que, sin haber filtrado por cliente, tenemos el mismo cliente continuamente delante nuestro, la segunda es que podemos ver el tiempo, y todas las llamadas se realizan en el mismo segundo. Es más, dos de ellas parecen ser exactamente iguales, ya que, si observamos la URL de la primera y la

última en la tabla, vemos que son la misma, y son peticiones GET. Eso quiere decir que muy probablemente estén llamando en bucle a nuestro sistema para ver el estado de los pedidos que tienen con nosotros.

El último detalle en el que nos fijamos es el tiempo de respuesta, donde vemos que varía desde 1100ms a 1900ms, e, internamente, sabemos que lo normal este tipo de llamadas suele tardar menos de 500ms. Esto nos da una idea de que tenemos una degradación en el sistema.

Aquí es donde llega el punto de encontrar una solución. En este supuesto en concreto nos puede servir con incluir una caché, pero esta no tiene por qué ser la solución siempre.

Es posible que en otros escenarios tengamos que llamar al cliente por teléfono para entender lo que están haciendo y por qué.

Ya que nos encontraremos en situaciones donde, simplemente, el cliente ha crecido muchísimo y como somos una dependencia suya, su dependencia en nosotros aumenta y por tanto incrementa el uso de nuestro sistema.

Yo he llegado a ver que un cliente abusara del sistema y, después de hacerle entrar en razón, se llegara a bloquear dicho cliente, para eso también sirve la API Gateway. Es un punto de entrada único, por lo que le puedes poner una lista de clientes a los que les vas a denegar el acceso.

Esto es únicamente un ejemplo de cuando algo nos va mal, pero una monitorización del uso de nuestra API externa, nos puede dar una idea de cómo nuestros clientes están utilizando el sistema y basándonos en eso, podemos hacer cambios a dicho sistema para mejorar el rendimiento general.

API Composition

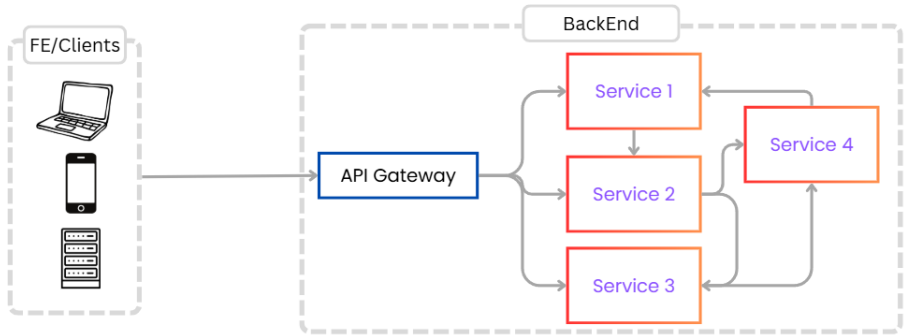
Hasta ahora hemos visto cómo, para devolver llamadas al *FrontEnd* o la app móvil, lo que hacemos es enviar peticiones desde el *FrontEnd* al *BackEnd*, y el *BackEnd* construye la información necesaria para devolver todo lo que necesita el *Front*.

Por lo tanto, los servicios se van a estar llamando entre sí cuando necesitan información del resto de servicios, creando un acoplamiento entre ellos, similar a lo que teníamos en la arquitectura monolítica.

Esto es debido a que cuando, por ejemplo, en el caso de los pedidos, tenemos una lista de pedidos con productos, únicamente tenemos el ID de los productos. Si necesitamos el *stock* necesitamos comprobar con el ID del producto el *stock*.

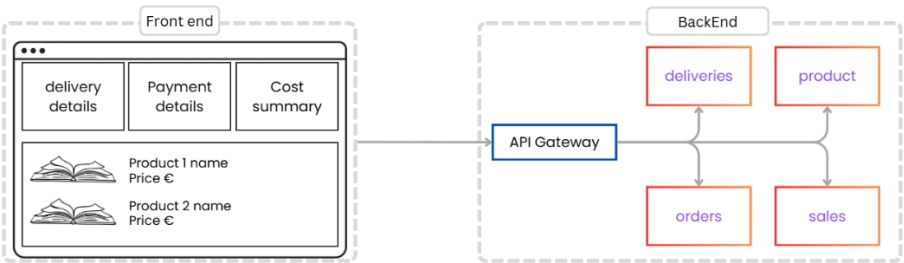
Si estamos visualizando un producto y queremos coger reseñas, debemos consultar con el servicio de reseñas, etc.

Y, al final, lo que sucede es una maraña de servicios donde todos se llaman entre sí.



Una forma de solucionar este problema es mover esa lógica al *FrontEnd*, si estamos cargando un pedido para ver los detalles, desde el *FrontEnd* hacemos múltiples llamadas, las cuales nos devolverán toda la información que necesitamos.

En nuestro caso, necesitamos, por supuesto, los elementos del pedido, con los nombres, la dirección de envío, el método de pago y el coste del pedido, con un desglose de los precios e impuestos.



Este proceso es más complejo de lo que parece, porque en este caso tenemos múltiples llamadas que las podemos poner en el siguiente orden:

1. Con el *orderId* cogemos los detalles del envío.
2. Con el *orderId* cogemos los detalles del pago.
3. Con el *orderId* cogemos el resumen del pedido.

4. Con el *orderId* cogemos los productos.

Aquí es donde viene la parte importante. Con estos cuatro puntos, cargas casi toda la información. Y digo “casi” porque una vez recibes el pedido, lo que recibes son los IDs de los productos, no tienes ni el nombre, ni el precio, porque el precio debe mostrar el coste de los productos cuando los compraste, por tanto, ahora tienes que hacer dos llamadas extra.

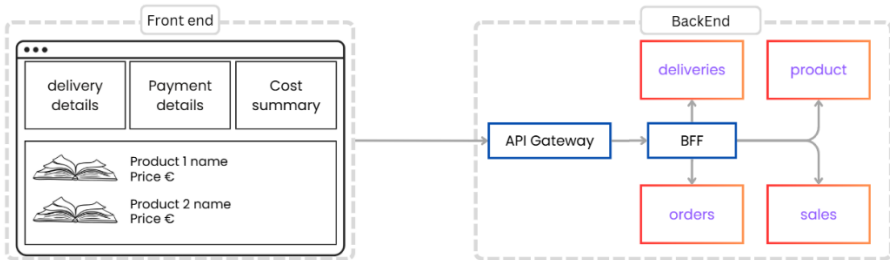
1. Con la fecha del pedido y los IDs de los productos, debes recibir el coste del producto en el momento de la venta.
2. Con el ID de los productos, debes devolver el nombre.

Como vemos, para mostrar esta información, la cual es muy básica, hemos tenido que realizar seis llamadas HTTP en dos bloques diferentes, lo cual afecta a la experiencia de usuario.

Aquí es donde entra el patrón API Composition, el cual es un patrón donde vamos a combinar múltiples llamadas API en una, para, como su nombre indica, componer una *request*. A este patrón se le llama también *BackEnd for FrontEnd* (BFF). Porque lo que hacemos es combinar todas las llamadas que el *FontEnd* necesita en una sola para mejorar la velocidad.

Además, de esta forma, únicamente devolvemos al cliente, en este caso, la interfaz de usuario, la información que necesita.

Por lo tanto, nuestro pequeño diagrama de arquitectura va a tener una capa extra:



Para este diagrama he puesto la capa de API Composition la cual responde a la interfaz, pero depende de que implementaciones tengan tus clientes externos. También puede darse el caso donde tengas un *BFF* para los clientes, aunque en ese caso suele llamarse *Aggregator layer*, o *Composition layer*. Lo importante es que la idea y funcionamiento es la misma, que el cliente únicamente tenga que realizar una única llamada a nuestro *BackEnd*.

BackEnd for FrontEnd

Esta capa, sea para el *FrontEnd*, para la aplicación móvil, o para clientes externos, es una capa que es completamente nuestra, la cual va a tener que ser mantenida por nosotros y creada desde cero por nosotros. Esto es debido a que está creada para cubrir las necesidades de los servicios que consumen el *BackEnd*, así que depende de en qué empresa estés o cómo tengan montado el sistema. Puedes ver diferentes funcionalidades en esta capa.

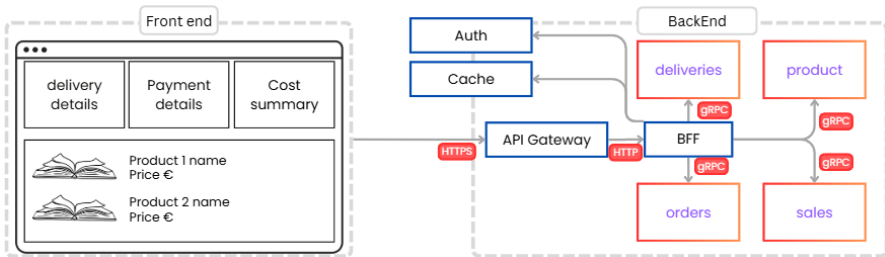
Hay empresas que únicamente agrupan la información que necesitan devolver.

En el caso de nuestro diagrama tenemos un *endpoint* que es */order*, y este *endpoint* tiene la lógica para llamar a los diferentes servicios propagando

el *header* de autenticación a los mismos, y son los propios servicios quienes se encargan de devolver la información correcta, o un error si algo no va bien.

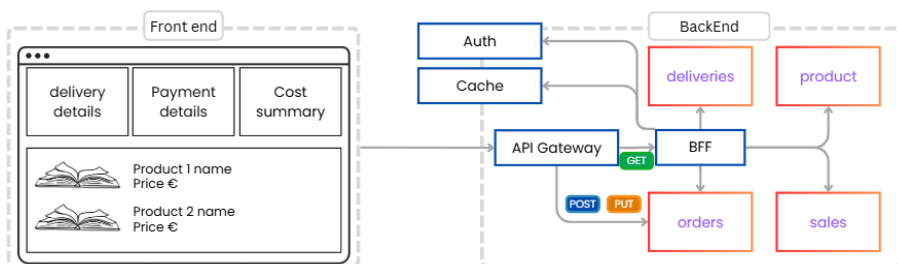
Esta sería la implementación más básica.

Al ser una capa completamente propia, podemos configurarla como queramos, lo que implica que podemos poner caché si lo deseamos, comprobar credenciales, o, incluso, ser esta la única API “accesible” por http y a partir de aquí hacer todo a través de llamadas gRPC, las cuales son mucho más rápidas.



En este escenario le darías una responsabilidad extra a la capa de BFF, que sería la del acceso a los recursos, y asumirías que todo servicio que llama a uno de los servicios gRPC tiene permisos para hacerlo.

En mis años trabajando también he visto situaciones donde el BFF es utilizado únicamente para leer datos, mientras que el resto de las acciones se hacen las llamadas directamente (a través de la API Gateway) a los servicios.



Finalmente, tenemos el modelo en el cual combinamos la API Gateway junto al *Backend for FrontEnd* y agrupamos todas las funcionalidades en una sola aplicación. Hay empresas, como Netflix, que hacen esto, pero necesitas muy buenos ingenieros, ya que una de las grandes ventajas de la API Gateway es que es muy rápida y ligera para cubrir toda la demanda de peticiones, mientras que en un *Backend For FrontEnd* podemos tener varios problemas, como vamos a ver a continuación.

Desafíos de utilizar Backend for FrontEnd

Por supuesto, esta capa trae una serie de desafíos con los que tenemos que lidiar. Unos son más sencillos que otros, pero al final hay que tenerlos a todos en consideración.

El primero, y más importante, es que muchas empresas tratan el BFF como un punto de entrada único al sistema. Esto quiere decir que en vez de tener un BFF por sección del Front End, como a mí personalmente me gusta, tienen una única aplicación, y claro, el BFF deja de ser una capa ligera y pasa a ser una capa tan enorme como un monolito donde todo el mundo lo toca, pero, a la vez, todo el mundo está nervioso al tocarlo, no sea que lo rompa y no funcione nada en todo el sistema.

Como he mencionado a mí me gusta tener un BFF por cada sección del *FontEnd*. Es muy fácil de ver si tenemos micro *FontEnd*, cada micro *FrontEnd* debe tener acceso únicamente a su propio BFF, sin realizar llamadas a otros BFF o a otros servicios de forma directa. Pese a que esta idea puede parecer muy buena, trae consigo un acoplamiento extremo ya que el BFF se vuelve muy específico al *FontEnd*, y no a todo el mundo le gusta.

Además, si realizamos un cambio en el *FontEnd*, debemos hacer sí o sí, un cambio en el BFF para acomodar dicho cambio, lo que implica que para desplegar un cambio en el *FrontEnd* tengamos que realizar los siguientes tres pasos:

1. Cambio en el servicio que realiza la acción.
2. Cambio en el BFF.
3. Cambio en el *FrontEnd*.

Este cambio extra, es un trabajo extra del 50%, lo que para muchos equipos pequeños puede ser un gran inconveniente.

Finalmente, no nos podemos olvidar de todos los problemas que puede traer el tener un servicio extra, como son el mantenimiento de dicho servicio, monitorización, la infraestructura a su alrededor y lo que es más importante, la escalabilidad. El servicio debe ser capaz de escalar de forma correcta conforme necesitamos más carga lo que añade un costo extra a la empresa.

Debido a todos estos retos, la gran mayoría de empresas acaban teniendo un sistema híbrido, donde ciertas acciones van a través de un BFF, pero otras muchas se hacen llamando directamente a los servicios y, a no ser que tu empresa tenga una gran cantidad de personas para mantener todas las capas, es el enfoque más aconsejable.

Componentes clave para la coordinación de servicios

Toda esta comunicación que estamos viendo entre las diferentes capas de la infraestructura, o, incluso, entre los propios servicios no pasa por arte de magia, debemos tenerla definida en algún sitio. Aquí es donde entran los elementos de infraestructura que no es que estén olvidados, pero se dejan de lado en las entrevistas para ahorrar tiempo.

Los tres componentes de los que vamos a hablar son técnicamente opcionales y el uso, o no, de cada uno de ellos, va a cambiar la implementación de los otros. En otras palabras, si existen, dependen unos de otros, pero si no existen, podemos funcionar sin ellos.

Gestión de la configuración

Toda aplicación que diseñamos necesita cierta configuración, desde el nombre de la propia aplicación para los metadatos de la observabilidad al nombre de la *cola*, *Exchange* o *topic* donde vamos a publicar un evento.

Toda esta configuración debe estar especificada y no solo eso, lo más probable es que cambie en cada entorno en el que desplegamos la

aplicación. Lo que quiere decir que la configuración entre entornos va a ser diferente.

Como tantas otras partes de nuestro sistema, tenemos múltiples opciones para especificar la configuración.

Ficheros de configuración

Podemos tener, en el propio repositorio, un fichero para cada uno de los entornos en los que estamos desplegando la aplicación.

Esto significa que tenemos un fichero padre y, dependiendo de en qué entorno estamos sobrescribimos dicha configuración.

Esto sería la estructura en un proyecto escrito en C#.

```
appsettings.json  
appsettings.dev.json  
appsettings.staging.json  
appsettings.prod.json
```

Podemos comprobar cómo tenemos un fichero *appsettings*, el cual contiene cierta configuración para que todo funcione de forma local. Por ejemplo, en el fichero por defecto, puedes estar apuntando a una base de datos local, mientras que en dev es la base de datos de dev, en staging es la de staging, en prod es la de prod.

Lo mismo sucede para un microservicio del que dependemos. Puedes estar probando una nueva funcionalidad entre el servicio 1 y 2 y los ejecutas de forma local, pero en el resto de los ficheros tienes la URL especificada de su ubicación en el servidor.

Tener toda la configuración de los entornos en los propios ficheros no suele ser la mejor de las ideas, ya que cualquier cambio de configuración, como puede ser una URL, implica que tienes que cambiar el contenido de los ficheros, lo cual, puede ser un problema muy grande si tienes decenas o cientos de servicios.

Configuración en tiempo de despliegue

Una de las alternativas más populares es calcular la configuración en tiempo de despliegue.

Esta acción implica que, mientras desarrollas la aplicación, apuntas a tu máquina local pero no te tienes que preocupar (por ahora) del resto de entornos, será cuando tengas que desplegar cuando tengas que ir a tu orquestador y configurarlo para que, en tiempo de despliegue, reemplace los valores necesarios.

Vamos a suponer que estás desarrollando el servicio A para que llame al servicio B como una nueva dependencia. Tú, en tu fichero, especificas algo similar a lo siguiente:

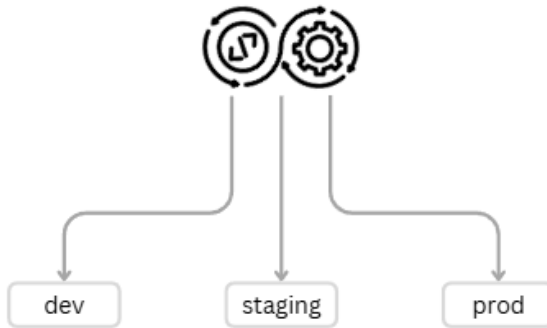
“ServiceB.URL”: “http://localhost:5000”

Simple y sencillo. Ahora, lo que tienes que hacer es, en el orquestador, añadir dicha variable para que funcione en todos los entornos. Depende un poco de qué tipo de sistema vayas a utilizar, pero todos funcionan de la misma manera; tendrás una tabla que te permite añadir dichas variables y su valor, así como el entorno sobre el que va a funcionar.

Key	Valor	Entorno
ServiceB.URL	https://dev.mycompany.com/serviceB	dev
	https://st.mycompany.com/serviceB	staging
	https://prod.mycompany.com/serviceB	production

Por supuesto, necesitarás un registro para cada uno de los entornos. En caso de no existir, tendrás el valor que tiene cuando trabajas de forma local.

La forma en la que este tipo de servicios funciona es muy sencilla. Al desplegar una aplicación en cada uno de los entornos, comprobar qué variables tiene configuradas que deban ir a dicho entorno y lo que hace es buscarlas en el fichero de configuración y reemplazarlas por las que tenemos en el orquestador.



De esta forma, al ejecutar la aplicación, tenemos únicamente un fichero de configuración con todas las variables bien configuradas.

Este tipo de sistemas nos van a permitir tener variables comunes o compartidas a través de librerías o sets.

Por ejemplo, tú puedes definir una librería o un set que se llame “ServicesInTheCompany” y tú ahí defines las variables comunes, en nuestro ejemplo, tendríamos la siguiente tabla dentro de dicha librería.

Key	Valor	Entorno
ServiceB.URL	https://dev.mycompany.com/serviceB	dev
	https://st.mycompany.com/serviceB	staging
	https://prod.mycompany.com/serviceB	production

Y luego, desde el orquestador de tu aplicación, lo que haces es invocar a la variable de dentro de la librería “ServicesInTheCompany”.

Key	Valor	Entorno
ServiceB.URL	<code>#{ServicesInTheCompany.ServiceB.URL}</code>	dev, staging, production

Por supuesto, esto aplica a cualquier variable en cualquier set o librería, lo que hace que, si tienes que cambiar una URL, lo hagas únicamente en un único lugar ya que el resto de los servicios están referenciando a dicha librería.

A su vez, si una variable es una contraseña, podemos configurar dicha variable como tal y no la podremos ver a través de la interfaz o la API, pero a la hora de desplegar la aplicación tendrá dicha contraseña.

Este enfoque es, sin ninguna duda, el más popular en las empresas.

Configuración como servicio

Si bien esta opción no es tan popular, existe y es posible hacerla, así que yo creo que merece la pena discutirla.

En este escenario, absolutamente toda la configuración está en la nube; en un servicio específico para ello. Su funcionamiento es similar al caso anterior, tenemos una tabla donde tenemos la Key con el valor y el entorno, pero, a diferencia del caso anterior, también tendremos la aplicación a la que hacen referencia.

Nombre	Valor	Entorno	Applicatio n
ServiceB. URL	https://dev.mycompany.com/serviceB	dev	ServiceA
	https://st.mycompany.com/serviceB	staging	ServiceA
	https://prod.mycompany.com/serviceB	productio n	ServiceA

Igual que en el supuesto previo podemos utilizar librerías o sets que nos proveen de la configuración compartida, o de la misma forma para ocultar las credenciales.

La diferencia radica en cómo esta información es pasada a la aplicación. En el caso anterior durante el despliegue se realiza una sustitución. En nuestro caso actual, es al inicio de la aplicación, durante el tiempo de ejecución, cuando la aplicación consulta al servicio de configuración y cuál es la configuración para el entorno en el que está siendo ejecutado. A su vez el servicio responde con la configuración en un formato que es entendible por la aplicación.

En algunos casos la información de la configuración pasará a las variables de entorno, mientras que en otros pasará a un fichero, o a ser un objeto *singleton* en memoria.

La gran pega de este enfoque es que, si tienes que realizar algún cambio local, o estás realizando testeos locales, puedes estar afectando al resto de compañeros que estén trabajando en la misma aplicación, porque, pese a trabajar en local, la configuración de la aplicación es compartida.

Service Registry & Service Discovery

Conforme nuestra empresa crece y los servicios aumentan, gestionar la configuración y la ubicación de estos servicios se vuelve más complejo. Saber dónde están ubicados y cómo acceder a ellos pasa a ser una parte crucial del sistema.

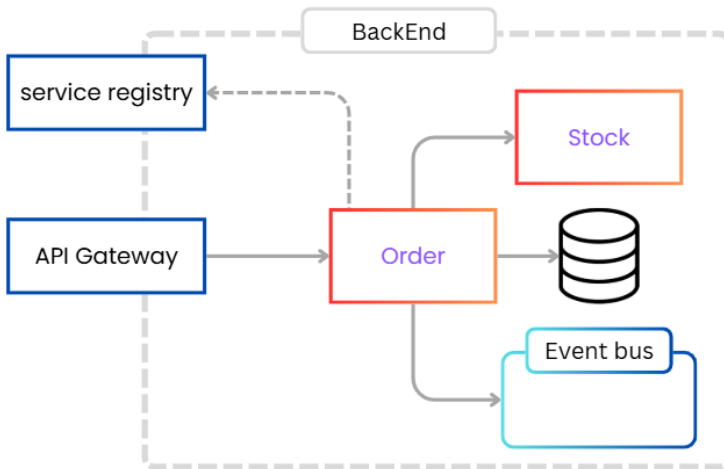
Descubribilidad de los servicios

Una forma de manejar esta configuración es inyectar las direcciones de los servicios directamente desde un orquestador. Sin embargo, existe un enfoque más dinámico: **utilizar un servicio dedicado para el registro y descubrimiento de servicios**, a esta acción de consultar, donde está cierto servicio se le denomina Service Discovery.

Pero la información está ubicada en un servicio al que llamamos Service Registry. Este servicio centraliza la información de ubicación y permite a los servicios obtener dinámicamente la información que necesitan.

Imaginemos que nuestro *BackEnd* de pedidos necesita:

1. La dirección de la base de datos.
2. La URL del servicio de stock.
3. La localización del *Event Bus* para publicar eventos.



Será el Service Registry el que contenga la información de todos ellos.

Lo mismo sucede para el sistema de productos, o cualquier otro. En resumen, necesitamos la ubicación de todos los servicios, ya sean internos o externos, de los que dependemos.

Esto significa que cuando tenemos que consultar la base de datos o cualquier otro servicio debemos consultar a dicho Service Registry.

Hay que tener en cuenta que, por norma general, la URL no va a cambiar nunca o casi nunca, especialmente cuando son privadas. Así que lo que se suele hacer es almacenar la información devuelta por dicho servicio en memoria, y mantenerla durante el tiempo de vida de nuestra aplicación.

¿Pero qué sucede si una de nuestras llamadas falla? En este caso, y guiándonos por el código de estado que devuelva la llamada, podemos identificar si ha fallado porque hay un error en la aplicación o porque tenemos la URL errónea. En ese caso podemos volver a consultar la URL y actualizarla en memoria, lo cual es un proceso que se hace de forma automática y casi instantánea, mientras que, si lo hiciéramos con configuración, tendríamos que hacer un despliegue.

La forma en la que el Service Registry funciona es muy sencilla, cada servicio tiene asignado una Key y de esa Key devuelve el valor.



El Service Registry actúa como una especie de base de datos. Le pasamos el nombre de un servicio, y este nos devuelve una URL junto con otra información útil, como:

- El puerto donde opera.
- Metadatos asociados al servicio.
- Estado operativo (si el servicio está activo o no).

En muchos casos, el propio sistema de Service Registry realiza comprobaciones periódicas (*health checks*) para asegurarse de que los servicios están funcionando correctamente.

Registro de servicios

Claro que para que un servicio esté disponible en nuestro Service Registry, debemos registrarlo de alguna manera.

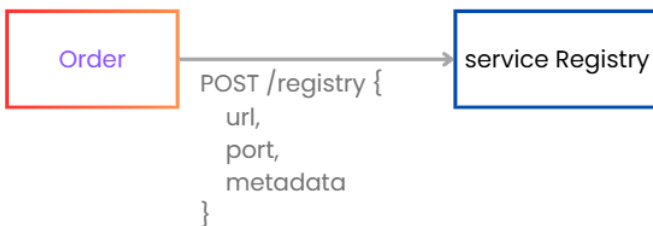
Aquí podemos hacer varias implementaciones dependiendo de la arquitectura de nuestro sistema actual.

Registro manual

No tiene mucho misterio. Cuando necesitas añadir un nuevo valor al registro, lo añades manualmente. Es sencillo y fácil de hacer, pero se hace de forma manual, lo cual es tedioso, puede tener fallos, etc.

Registro automático

Los servicios se registran de forma automática al iniciarse, simplemente se realiza una llamada al Service Registry con la información de la URL, puertos, metadatos, etc.



Por supuesto, para enviar la URL o puerto, el propio servicio debe ser capaz de obtener dicha información, por lo que, en este caso, necesitamos hacer más trabajo en el lado de la infraestructura del sistema. Aunque ese trabajo se hará una única vez.

Patrón sidecar

La tercera opción es desplegar cada una de nuestras instancias con lo que denominamos *sidecar*. Un *sidecar* no es más que una aplicación (o múltiples) que se despliega conjunto a nuestra propia aplicación.



En este escenario, la aplicación que registra el servicio es el *sidecar*, que, técnicamente, es una aplicación diferente al servicio que contiene la lógica. Simplemente corren en paralelo en la misma máquina y están conectados entre sí.

No viene al tema, pero lo mismo aplica para *logs*, *metricas*, *health checks*, etc, podemos excluir toda esa configuración de la aplicación donde está la lógica de negocio y ponerla en el *sidecar*.

Personalmente, no soy el mayor fan de los sidecares porque son una capa extra que hay que mantener y configurar, y, particularmente, prefiero tener una librería o un único SDK para hacer dicha configuración. Lo que sí es cierto es que, si tienes microservicios en múltiples lenguajes, puedes ahorrar trabajo, ya que el *sidecar* es agnóstico al lenguaje con el que la aplicación está escrita y, básicamente, actúa como un *proxy* entre la aplicación y el servicio al que quieres invocar.

Gestión de credenciales

De igual manera que tenemos un servicio para almacenar la información de dónde están ubicados otros servicios, podemos hacer exactamente lo mismo con credenciales y contraseñas. A este tipo de servicios los llamaremos un *Secret manager*.

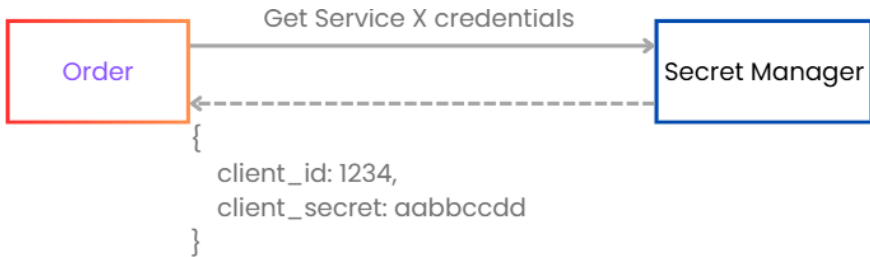
Este tipo de servicios tienen la característica de que la información que contienen es confidencial, por lo que tiene que ser almacenada de forma segura.

Cuando hemos visto la configuración de una aplicación en este mismo capítulo, hemos mencionado que podemos tenerla toda en ficheros, uno para cada tipo de entorno y, aunque parezca que no, esta práctica era muy común antiguamente, pero conlleva un problema muy grande, y es que las credenciales están expuestas a todo el mundo, lo cual es un problema obvio.

Debemos configurar nuestras aplicaciones para que las contraseñas no estén visibles, ya sea utilizando un orquestador, como hemos mencionado antes, o un servicio para leerlas en tiempo de ejecución.

La lógica es sencilla. Tenemos un servicio donde realizamos una llamada HTTPS y nos devuelve los datos.

Ojo, en muchos casos las credenciales pueden ser simplemente un *string*, pero en otros puede ser un objeto más complejo, ya que, en muchos escenarios, para conseguir acceso a un sistema, necesitamos un usuario y una contraseña, o para conseguir un *access_token* necesitamos un *client_ID* y un *client_secret*. Ambos elementos van a estar configurados dentro de nuestro gestor de credenciales.



La gran ventaja de utilizar un gestor de credenciales frente a gestionarlas en tiempo de despliegue es a la hora de tener que rotarlas.

En muchas empresas no se hace, pero es recomendable rotar cualquier credencial cada tres meses. Ello implica que, en nuestro escenario, la credencial se reemplazará automáticamente, ya que podemos añadir configuración para que, si la llamada al otro servicio falla, con un código de error de *Unauthorized* vuelva a consultar la credencial.

En cambio, si estamos aplicando estas credenciales en tiempo de despliegue, tendremos que volver a desplegar la aplicación, lo que requiere una ejecución manual, así como tests, etc.

CAPÍTULO 10

Arquitectura de eventos

Hemos visto el funcionamiento de microservicios junto a las colas, *Event Bus* o *Streams*. Combinamos ambas arquitecturas, o patrones, y tenemos lo que comúnmente llamamos como arquitectura de eventos.

El motivo es simple: cada vez que realizamos una acción en el sistema, generamos un evento que indica que dicha acción ha sucedido.

Cada vez que generamos un pedido, creamos un evento llamado *OrderCreated*. El nombre del evento es importante ya que, simplemente leyéndolo, sabes qué es lo que ha sucedido. Estos eventos representan una única acción, lo que quiere decir que si lees *OrderCreated* sabemos que el pedido se ha generado en el sistema, pero se utiliza un *OrderDispatched* cuando el evento ha sido enviado desde el almacén, o un *OrderFullfilled*, cuando todos los productos de dicho pedido están cargados en el camión. Cada evento representa un estado único que ha sucedido en el sistema, por lo tanto, el verbo estará siempre en pasado.

Dentro de la arquitectura de eventos tenemos varios patrones y arquitecturas que podemos elegir y saber en qué situación utilizar cada uno de ellos, o, mejor dicho, cómo utilizar cada uno de ellos, lo cual es crucial a la hora de diseñar sistemas.

Uso de la base de datos

Quiero empezar por el punto final de un proceso, el almacenamiento de los datos en una base de datos. A mí, personalmente, me gusta ver las bases de datos como almacenes de información, las cuales funcionan detrás de una interfaz en el código, y me olvido de ellas.

Pero a la hora de diseñar, debemos tenerlas en consideración.

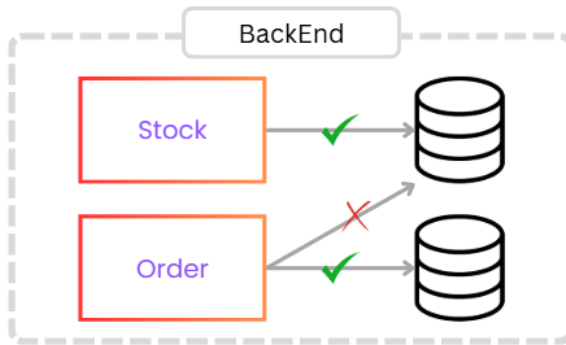
Hay dos preguntas que son muy comunes a la hora de diseñar nuevos sistemas.

¿Necesitamos una base de datos por microservicio?

La primera es saber si vamos a utilizar una base de datos por microservicio, o vamos a tener una base de datos global donde todo el mundo pueda acceder.

Esta pregunta es una cuestión muy muy importante porque migrar una base de datos, y el uso que hacemos de ella es mucho más complicado que migrar aplicaciones. Así que la decisión que tomemos va a persistir en el sistema, posiblemente para siempre.

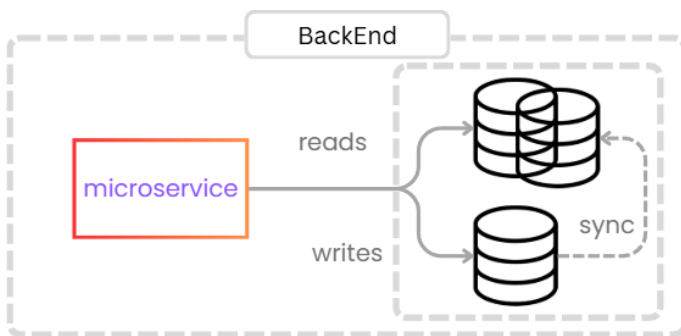
De primeras, y si somos estrictos a la hora de crear microservicios, la respuesta es que sí. Necesitamos utilizar una base de datos por microservicio.



Y esta situación es muy normal, me atrevería a decir que, en la mayoría de las empresas que utilizan microservicios, lo tienen así.

Esta separación es muy buena porque si tú quieres tener los datos completamente separados y evitar que el microservicio de Pedidos acceda a los datos de Inventario directamente, en la base de datos no tienes que hacer nada, porque no tiene acceso, se ve obligado a acceder a través de una API o recibiendo la información con un evento.

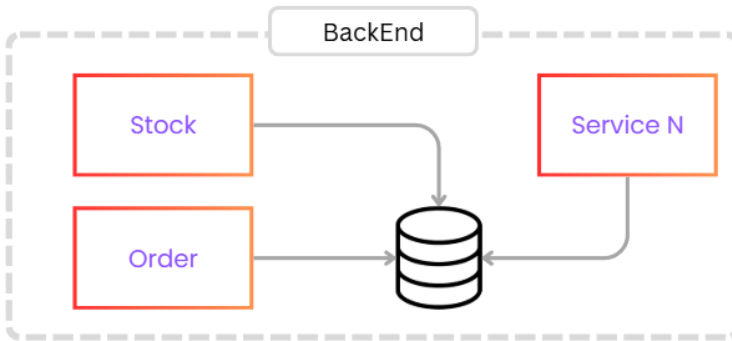
Ahora debemos tener en mente que, en muchas empresas, tendremos réplicas de las bases de datos y, además, las tendremos configuradas como maestro/esclavo o lectura/escritura.



Lo más normal es que maestro/esclavo funcione de forma transparente a los desarrolladores, pero, al final, no dejan de ser más elementos de infraestructura que se deben configurar y mantener, así que merece la pena mencionarlo.

De hecho, no solo eso, en muchos casos, ¡tendremos más de una base de datos por microservicio! Y no me refiero a maestros/esclavos, que esas están abstraídas, sino a diferentes bases de datos donde tendremos datos únicamente de lectura en una y otra, donde podemos leer y escribir. El tema es más complejo y hablaremos de ello en el apartado de CQRS (*Command query responsibility segregation*).

La alternativa a tener una base de datos por servicio es tener una base de datos común a múltiples (o a todos) los servicios de la empresa.



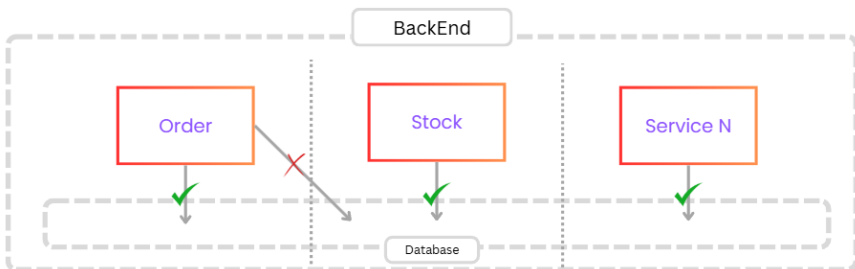
A muchos les puede parecer extraño, pero tener una única base de datos es muy común cuando estamos migrando sistemas desde uno monolítico a uno distribuido. Más adelante, en el libro, hablaremos de migraciones, pero sabed que, en muchos casos, la base de datos no se toca.

Obviamente, y en circunstancias ideales, debemos elegir el utilizar una base de datos por microservicio. Esta decisión trae consigo varias implicaciones, y todas son similares a las diferencias entre usar microservicios o no.

La primera de las implicaciones es muy obvia. Igual que sucede con los microservicios, si tenemos que escalar una base de datos, podemos hacerlo sin problemas; si únicamente tenemos una única base de datos, también podemos escalar por supuesto, pero será mucho más costoso.

De la misma forma, puedes elegir un tipo de base de datos que más se acomode a tus necesidades. Es posible que en ciertos servicios quieras utilizar una base de datos NoSQL, o incluso *ElasticSearch*, mientras que la norma en la empresa es utilizar bases de datos relacionales. Al no tener la limitación de la base de datos, las aplicaciones están mejor desarrolladas. Y en caso de tener que hacer un cambio, no vamos a depender de otro sistema para poder aplicarlo.

Igual que los microservicios, tiene la pega de que no puedes acceder al código que no está en el mismo microservicio. La misma lógica aplica a los datos, donde no puedes hacer *join* o transacciones. Pero claro, incluso la gran mayoría de empresas que tienen una única base de datos tampoco los pueden consultar, porque si bien es cierto que solo es una, la práctica más común es separar los dominios de forma lógica.



Lo que implica que, por un tema de permisos, tampoco podríamos acceder a los datos de otro dominio. Permisos que, una vez más, debemos mantener.

Con esto en mente, debemos saber que lo más común, y lo recomendado, es tener una base de datos por servicio.

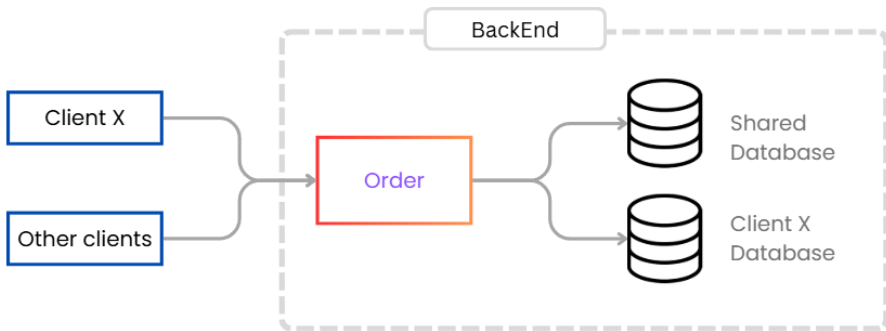
¿Necesitamos una base de datos por cliente?

La segunda de las preguntas más común es saber si necesitamos una base de datos para cada uno de los clientes o no. Si trabajamos en este tipo de aplicaciones las llamaremos aplicaciones *multi tenant*, y son el tipo de aplicación más común en el mundo empresarial.

A la hora de saber si necesitamos una base de datos por cliente, debemos de tener varias cosas en cuenta.

La primera, y más importante, es temas legales y de contrato. Depende de con qué empresas estemos trabajando pueden pedir que sea obligatorio el tener sus datos completamente aislados. A mí me ha pasado, donde tuvimos que cambiar un sistema para acomodar a un cliente. Básicamente puedes tener clientes importantes donde te dicen, “mi empresa está en X país y todo el procesamiento y almacenamiento de los datos tiene que realizarse en dicho país.”

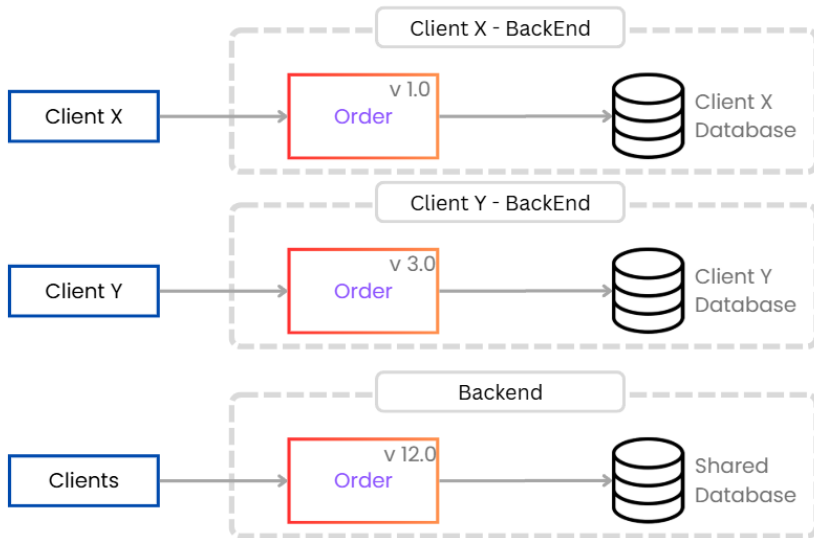
Si este caso existe, necesitas tener una base de datos diferente para ese cliente. Aún sin necesitarlo de forma legal, es posible que tú quieras hacer dicha separación, ya que trae consigo varios beneficios. El más importante es la propia separación de los datos, lo que evita el tener una posible contaminación de datos entre clientes.



Llegados a este punto, deberás averiguar si no sólo los datos, sino todo el sistema, tiene que estar en dicho país, por lo tanto, ya no tienes únicamente una base de datos para dicho cliente, sino que tienes un sistema entero.

Y aquí es donde la cosa se empieza a complicar mucho, ya que puede ser que tengamos el 98% de los clientes en el sistema común, pero tengamos dos que son completamente diferentes, y necesitan sus datos y procesamiento aislados.

En tal caso, necesitarás mantener tres sistemas de forma paralela e intentar que todos estén más o menos en las mismas versiones. Uno de los problemas que yo he vivido, y sufrido, con este tipo de implementaciones es que varios de los clientes no se actualizan en varios años, por lo que tienen un software antiguo y con errores, lo que causa muchos errores y puede ser un problema muy grande.



Este tipo de situaciones existe, yo lo he vivido y es un dolor, pero suele darse en empresas donde hay un número de clientes limitado. Por ejemplo, aerolíneas, instituciones bancarias, empresas de seguro, etc. Como ves, todo son empresas que tienen muchos datos de una cantidad de personas elevadas.

Para una aplicación donde tus clientes son talleres mecánicos (como ejemplo de un servicio muy común en todas partes), lo normal es tener un único sistema.

En cualquier caso, si tienes múltiples sistemas con múltiples bases de datos, deberás tener un equipo capaz de cubrir la demanda, ya que no es tarea sencilla ir replicando todo en todos los entornos, incluso con orquestadores es complicado y puede dar fallos inesperados.

Si vamos a la otra opción, donde el sistema es compartido por múltiples clientes, también debemos hacer una separación dentro de la propia

base de datos. Y, en este caso, lo que se hace es tener una columna, la cual contiene el ID de la empresa.

Esta columna va a estar en casi la totalidad de las tablas de la base de datos y, en cada consulta que realizamos, vamos a tener que comprobar dicho ID, ya que en caso de no hacerlo podemos tener contaminación de datos; lo que significa que un cliente puede estar leyendo datos de otro cliente.

Patrón CQRS

El patrón CQRS es un patrón de diseño de software, el cual nos muestra cómo separar la lógica de nuestras aplicaciones para separar las lecturas de las escrituras.

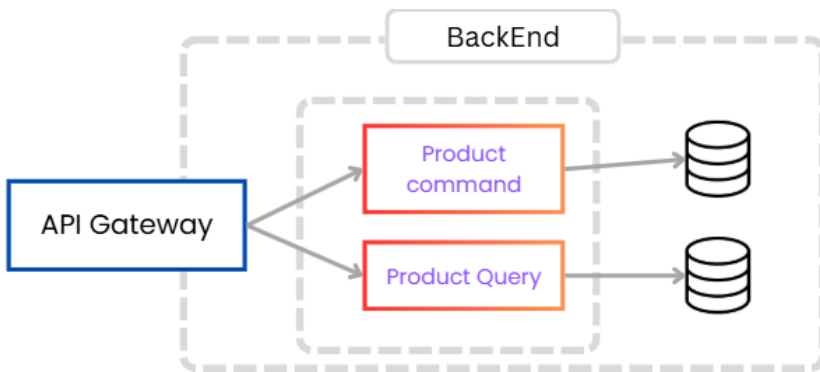
Puedes pensar que por qué estoy especificando un patrón de aplicaciones en un libro sobre sistemas distribuidos, y el motivo es por la implementación que muchas veces se hace de él en el propio código, ya que afecta a la infraestructura.

Las siglas CQRS vienen del inglés y significan *command Query Responsibility Segregation* y hacen referencia a la separación de responsabilidad entre las lecturas y las escrituras.

Este patrón no lo habíamos visto antes de la llegada de los microservicios, ya que en el mundo de los monolitos era bastante común leer y escribir en la misma invocación de código.

Este patrón trae consigo ciertas ventajas a nivel de aplicación, principalmente centradas en lo fácil que es seguir el flujo de trabajo del código. Pero aquí nos vamos a centrar en el tema de arquitectura.

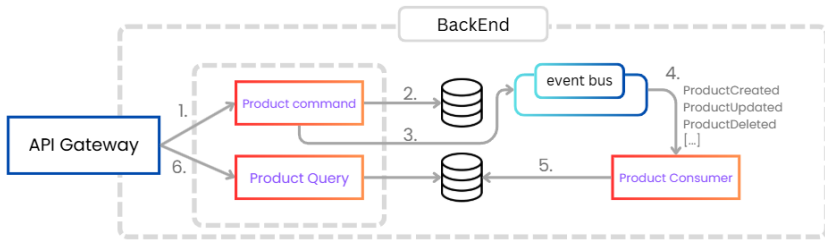
Si aplicamos este patrón a nivel de arquitectura, lo que estamos implementando son dos aplicaciones; una para crear y editar (*command*), y otra para leer (*Query*). Pero ojo, porque si hacemos esto, lo normal es no solo separar las aplicaciones, sino tener bases de datos independientes.



El cliente, o consumidor, desde fuera del sistema no lo sabe, porque está detrás del API Gateway, pero internamente nuestro servicio de Productos está separado en dos.

Llegados a este punto, puedes pensar que muy bien, pero ¿cómo funciona exactamente?, porque si estamos insertando datos en una base de datos, pero leemos de otra diferente, ¿cómo funciona el sistema?

Aquí es donde entra la comunicación asíncrona.



1. Creamos un producto a través de la aplicación de escritura.
2. Guardamos los datos en la base de datos de escritura.
3. Generamos un evento *ProductCreated*.
4. El consumidor, consume el evento *ProductCreated*.
5. Procesamos el evento *ProductCreated* y guardamos en la base de datos.
6. Las consultas llegan a la aplicación de lectura donde los datos están actualizados.

Lo mismo aplica para cualquier tipo de evento que pueda afectar a la información de los productos.

Una característica de esta implementación es que, en muchos escenarios, tenemos dos tipos de eventos. Los que se generan desde nuestra aplicación para ser leídos por otras aplicaciones de dentro de nuestro dominio lógico. A estos eventos los llamaremos eventos de dominio.

Y tenemos también los eventos que van a ser escuchados por aplicaciones en otros dominios lógicos, a este tipo de eventos lo denominamos eventos de integración.

Si se implementa esta solución, una práctica muy común es tener la información en la base de datos de lectura lista para ser leída,

completamente estructurada y accediendo por un ID único, por lo que la velocidad de lectura es muy rápida.

¿Necesitamos CQRS en la arquitectura?

La pregunta más obvia es ¿para qué necesitamos dicha implementación?, pues a la vista está que es una implementación muchísimo más compleja que la anterior, cuando escribimos y leemos de la misma base de datos.

Y aquí entra el tema del nivel de granularidad a la que queramos llegar en nuestras aplicaciones. Para la gran mayoría de sistemas, una separación de CQRS a nivel de arquitectura de sistemas puede no tener sentido porque añade mucha complejidad al sistema, pero habrá en otras situaciones donde nos sea muy útil.

Por ejemplo, vamos a poner la semana del Black Friday como ejemplo más obvio. Durante esa semana se crean muy pocos productos, lo que implica que las instancias del servicio de escritura de productos se pueden mantener al mínimo. En cambio, vamos a estar consultando productos constantemente, muchas veces con una carga 10 o 20 veces superior a la normal.

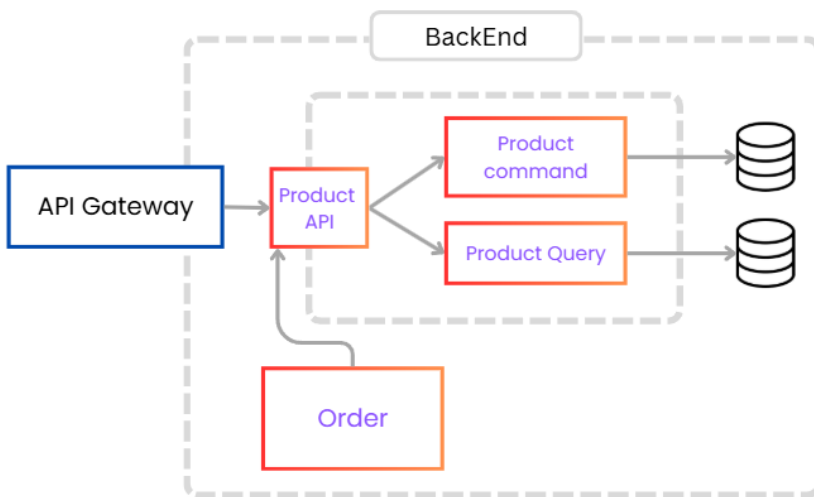
De esta forma podemos escalar únicamente la parte de lectura y dejar la de escritura sin tocar.

El hecho de tener una base de datos por servicio, o una compartida, es exactamente igual que la lógica que hemos aplicado a la hora de separar la base de datos en microservicios. Técnicamente, el servicio de lectura

es un microservicio completamente diferente al de escritura, por lo que deberían estar separadas. Aun así, están dentro del mismo dominio lógico del sistema, por lo que lo ideal sería buscar un balance apropiado.

Esta funcionalidad puede ser muy útil en algunos de nuestros servicios y un *overkill* en otros, por lo que saber dónde y cuándo debemos utilizar CQRS, a nivel de arquitectura, en nuestro sistema es muy importante.

Existe una variación a esta implementación donde tenemos una API delante de los servicios Query/Command.



En este escenario la API es la única que tiene acceso a los servicios de *command & query*, y actúa como un API Gateway para ese mini dominio; incluso si la llamada viene desde dentro del sistema o de otro servicio como el de pedidos deben de pasar por la API.

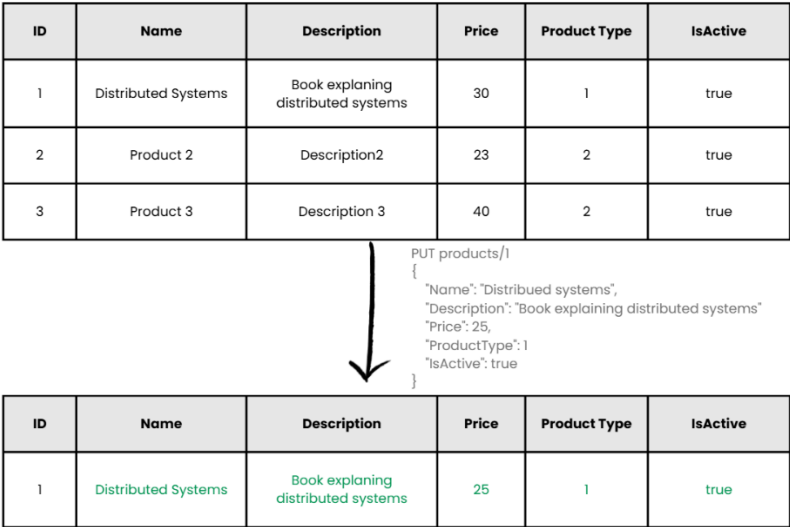
Patrón Event Sourcing

Cuando creamos sistemas utilizando el patrón CQRS, solemos crear los sistemas utilizando *event sourcing*, ya que están muy relacionados entre ellos.

Para comprender mejor qué es *event sourcing*, debemos comprender primero la situación en la que estamos actualmente.

Cuando queremos modificar un producto, tenemos una API o servicio que tiene un *endpoint* donde actualizaremos dicho producto, y en ese *endpoint* recibimos todo el objeto de producto, lo que significa que vamos a actualizar todo el registro en nuestra base de datos.

Una tabla con registros y queremos actualizar el precio (o cualquier campo) de uno de ellos, tendremos que realizar una llamada a la aplicación correspondiente enviando todos los datos para que estos se sobren escriban:



Y pese a que únicamente hemos cambiado el valor de uno de los campos, técnicamente los hemos modificado todos porque el *endpoint* recibe todos los valores.

Posteriormente crearemos un evento llamado *ProductUpdated*.

Lógica del patrón Event Sourcing

Para evitar esta situación, utilizamos el patrón Event Sourcing.

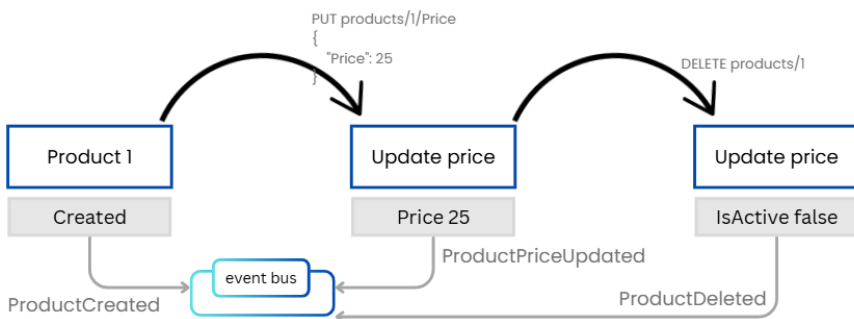
Event sourcing es un patrón de diseño donde no tenemos únicamente un registro con el estado actual, sino que tenemos la información de todos los eventos que han sucedido en el sistema.

Los eventos suelen estar relacionados, aunque no limitados, a las acciones de usuario, y cuando combinas todos eventos sucedidos para cierto objeto, llegas al estado actual.

Al diseñar aplicaciones con el patrón Event Sourcing vamos a crearlas de forma diferente, ya que vamos a agrupar los cambios lógicos que tengan sentido, porque así crearemos esos eventos específicos de los que hablamos.

Los eventos los almacenaremos en un almacén de eventos, comúnmente una base de datos, o un sistema similar. Kafka, al no borrar los mensajes, también nos puede servir como *event store*.

En nuestro caso, si queremos crear una situación como la anterior, donde queremos actualizar el precio de un producto, ya no vamos a crear un *endpoint* que sea para modificar un producto completo, sino que vamos a tener un *endpoint* en el sistema, el cual sea específico para la modificación del precio, otro para eliminar un registro, etc.



Es más, esa acción va a generar un evento único de esa acción, por lo que los consumidores de los eventos tienen una granularidad mayor.

Obviamente, no necesitas un *endpoint* para cada columna o valor, pero sí es importante agruparlos de forma lógica. Por ejemplo, el nombre y la descripción tiene sentido que vayan juntos en un único *endpoint*.

Finalmente, volver a hacer hincapié en los nombres y la atomicidad de los eventos. Es importante que los nombres sean claros y concisos, así como exclusivos de un dominio. Si tenemos un evento llamado *ProductDeleted*, no queremos otra parte del sistema que tenga un evento diferente que se llame igual. Hay que buscar nombres que nos permitan identificar a qué parte del sistema nos estamos refiriendo.

Event sourcing como *Source of truth*

Utilizar *Event sourcing* significa tener un registro de todos los eventos que han sucedido en el sistema, no solo eso, sino que los tenemos en orden. Debido a ello, tenemos información sobre cómo hemos llegado al estado actual. A esta información le llamaremos *Source of truth*, o fuente de la verdad, porque al contener todo lo sucedido por un sistema es fiable.

Un caso muy sencillo de ver en un dominio de una tienda puede ser útil para comprender el estado actual del inventario. Sin utilizar *Event sourcing* puedes ver una tabla como la siguiente:

ID	OnHand	Allocated	InTransit
1	140	10	50
2	20	60	3
3	349	1	50

Donde en el sistema de inventario tenemos una tabla con:

1. ID del producto.
2. Cuantos productos tenemos disponibles para que clientes los compren (*OnHand*).
3. Cuantos productos tenemos disponibles en el almacén, pero que están asignados a un pedido ya pagado, pero que no ha salido del almacén (*Allocated*).
4. Cuántos productos están siendo enviados a su destino (*InTransit*).

Y, si bien, esta información es la que más nos va a interesar leer, no tenemos un conocimiento directo de cómo hemos llegado hasta este estado.

Pero qué sucede si en el almacén hacen un recuento de inventario y te dicen que hay ciento sesenta y no ciento cuarenta del primero de los productos. Es más, quién te garantiza a ti, que tenemos cincuenta en tránsito y ese número no está mal. No tienes forma de comprobarlo de forma más o menos rápida.

Obviamente, hay formas de conseguir los valores exactos. Podrías ir pedido por pedido, ver los que están en tránsito y consultarlos uno a uno para ver que, de verdad, podrías coger el albarán de tu proveedor de productos, ver cuántos compraste, ir mirando todos los pedidos y hacer las cuentas a mano. Pero, esto se puede hacer como caso excepcional, si tienes que hacer esto mismo con cien o, incluso mil productos, es imposible.

Aquí es donde entra el patrón *Event sourcing*. Vamos a saber exactamente cómo hemos llegado a este estado, porque tenemos toda la historia de dicho producto. Pongamos un ejemplo en el primero.

El almacén te llama y te dice que hay veinte ítems más. Como es obvio, ellos van a coger y poner veinte ítems más en el campo *OnHand*, pero como es un error común decides investigar.

Lo primero que haces es ir a tu *event store* y ver los eventos que han sucedido sobre la entidad. Y la forma de visualizarlo es como una tabla.

EventId	DomainKey	EventName	TimeStamp	Content
869aa320-53c7-47c4-955c-8409334af1a3	1	ProductCreated	2025/Feb/25 10:00:00	{ productid: 1 }
98f9f920-fe95-4548-b210-34ffbb68ad3c	1	StockAllocated	2025/Feb/25 10:02:00	{ productid: 1 stock: 150 }
309939c9-7255-40ed-bdf7-9b58764defb1	1	OrderCreated	2025/Feb/25 12:00:00	{ other information [{ productid: 1 quantity: 1 }] }

Esta tabla nos da toda la información que necesitamos:

1. *EventId*: el cual es un ID único para dicho evento.
2. *DomainKey*: un ID de dominio, que hace referencia a un identificador único para relacionar los eventos entre sí.
3. *EventName*: el nombre del evento.
4. *TimeStamp*: cuando ha sucedido el evento.
5. *Content*: el contenido del evento.

Mantener esta información es muy importante dentro del patrón *Event sourcing*, ya que nos permite volver a ejecutar todos los eventos desde el principio para volver al estado actual.

De hecho, podemos ver cómo estos eventos afectan a los valores almacenados:



Una vez tenemos reconstruida la tabla, tendremos dos salidas:

- 1 - Los datos en la tabla de lectura son correctos.
- 2 - Los datos en la tabla de lectura son incorrectos.

Si los datos de la tabla de lectura son correctos, lo más posible es que tengas algún problema en el proceso manual.

Por ejemplo, quien introdujo la cantidad de elementos en el producto por primera vez, pudo tener un fallo e introducir más de los que de verdad teníamos.

Pero si los datos en la tabla de lectura son incorrectos, es posible que algún evento se haya perdido, o que cierta parte del sistema tenga un *bug*.

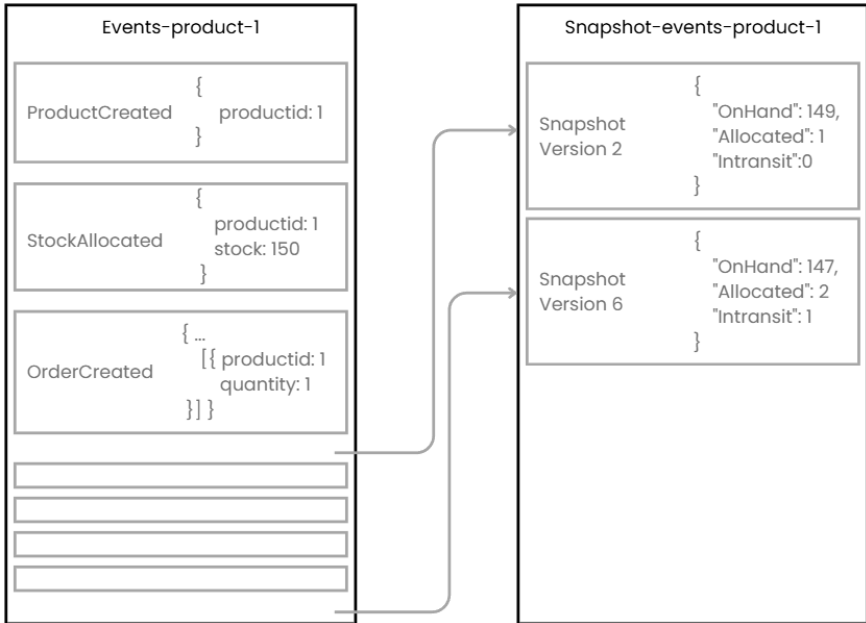
Es muy importante documentar qué es lo que sucede con cada evento, cuáles son las acciones que van a suceder, pues puede darse el caso de tener el sistema configurado para que cuando se cancele un pedido se añada el producto de *Allocated* a *OnHand*. Pero, si el pedido ya ha salido del almacén, el valor va a estar añadido ya a *InTransit*, por lo que necesitaremos un proceso para identificar si esta *InTransit*, moverlo a *Allocated* y de ahí, una vez pase por el almacén, moverlo a *OnHand*.

Comprimir eventos (Snapshots)

Llega un punto en el que tenemos una cantidad ingente de eventos, no hablo de cientos sino de miles o millones. Llegados a este punto no tiene mucho sentido leer todos los eventos de la base de datos para reconstruir el estado, porque es mucho tiempo. La solución que tenemos es hacer un *snapshot*, que no es más que una captura de nuestro estado actual.

Seguimos teniendo todos los eventos que han sucedido desde el día 1, pero ahora tendremos otra tabla, *stream* o servicio, para comprimir dichos datos, y cuando leamos, desde la aplicación, leeremos primero la tabla de *snapshot*, veremos dónde nos hemos quedado, y continuaremos con los eventos desde ahí.

Su funcionamiento es muy sencillo. Coges todos los eventos de un producto en concreto, y replicas el estado que generan estos eventos en un *stream* diferente. En nuestro caso sabemos cuál es porque tenemos nuestra *DomainKey*.



A esta *snapshot* le agregamos una versión, que hace referencia a la posición del último evento que hemos leído. Por lo que, a partir de ahora, si queremos reconstruir el objeto en el código, lo que hacemos es leer de nuestra *snapshot*, y de ahí leer los eventos que sean posteriores a la versión utilizada. De esta forma la velocidad de lectura desde la aplicación es mucho más rápida.

Idempotencia

Uno de los mayores miedos cuando trabajamos con sistemas distribuidos es que los eventos funcionen bien, que se manden, que se procesen y que, si fallan, tengamos un registro de ellos.

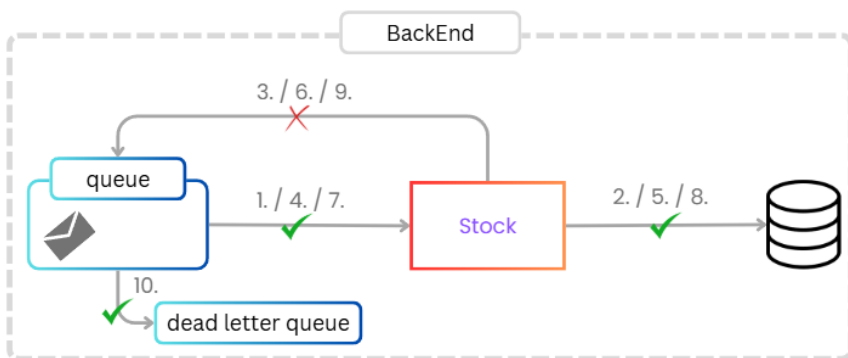
¿Pero qué sucede, si un sistema procesa eventos que ya han sido procesados con anterioridad?

Idealmente no debería de suceder nada, desafortunadamente, para que no suceda nada, debemos implementar idempotencia en nuestro sistema.

La idempotencia es una técnica que nos permite realizar una operación múltiples veces obteniendo siempre el mismo resultado.

Esto quiere decir que, si ejecutamos un evento dos veces, la segunda ejecución no debería alterar el resultado. En programación especificamos una propiedad en un evento, o en una *request*, para evitar que este se pueda procesar múltiples veces. A esta propiedad le llamamos *IdempotentId* o *IdempotentKey*, aunque depende de que IDs utilices en tus eventos puedes utilizar dichos IDs, ya que si es un UUID va a ser único.

Lo que queremos evitar es un escenario como el siguiente:



Tenemos un proceso que recibe un evento en una cola.

1. El consumidor lee el mensaje y lo procesa.

2. Actualiza los datos en la base de datos.
3. Comunica a la cola el ACK del evento.

Este último punto de comunicación falla, puede fallar, o puede ser que el proceso tarde cinco minutos en ejecutarse y tengamos los intentos desde la cola cada treinta segundos y, simplemente, el evento vuelva a estar disponible. El punto importante es que el proceso falla y el ACK nunca se manda, por lo que volvemos a procesar el evento varias veces, lo que puede causar un gran problema.

En este ejemplo sencillo vemos cómo vamos a procesar tres veces el evento *OrderCreated*, lo que implica que vamos a pasar de tener un stock disponible ciento cincuenta a ciento cuarenta y siete con tres en *Allocated*, cuando, en realidad, solo deberíamos tener uno.

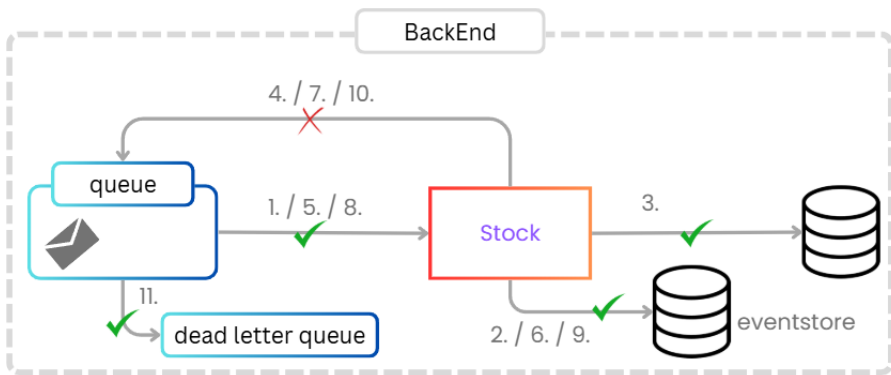


Este problema parece muy sencillo de arreglar, pero ahora imagínate que sucede durante varias horas o durante la noche, en un día muy ajetreado, y llegas a la mañana siguiente.

El problema puede ser mayúsculo, porque lo más probable es que no solo este sistema haya fallado sino algo más. Quizá almacén también tiene un problema similar y ha creado los pedidos cinco veces, por lo que van a empaquetar todo varias veces, o, en el peor de los casos, puede ser que el sistema de cobros también falle y vaya a cobrar a todos los clientes varias veces.

Un fallo así puede destruir la reputación de una empresa y hacer que se vaya a pique.

Para solucionar este entuerto hay que hacer un pequeño cambio en el código, y la verdad es que es muy sencillo. Cuando leemos un evento, debemos comprobar si hemos procesado ya dicho evento a través del *IdempotentId* o del *EventId* y descartarlo automáticamente.



Ahora el proceso es muy similar al de antes pero más seguro:

1. El consumidor lee el mensaje.
2. Comprobar si hemos procesado el evento y guardar los datos en el *event store*.
3. Procesar y actualizar los datos en la base de datos.
4. Comunicar a la cola el ACK del evento.

La diferencia principal radica en el segundo punto, donde vamos a guardar en el *event store* el evento que vamos a procesar, de esta forma solo procesaremos el evento una vez, aunque nos llegue repetido mil veces.

Este pequeño cambio minimiza el riesgo con los clientes, aunque de nuestro lado tengamos algún error, y quizá a la mañana siguiente debamos realizar algún proceso manual para verificar todo el sistema, es un costo mucho menor que afectar a los clientes y al costo reputacional.

Finalmente, hay que mencionar que no siempre tenemos que almacenar todos los eventos en todos los sistemas, depende mucho en cómo están estructurados ya que si son eventos que vamos a utilizar para almacenar información del estado actual de un registro. Por ejemplo, el nombre y la descripción de un producto, pues bueno, volverlo a procesar no es un problema mayúsculo, especialmente si no somos los dueños de dicho dominio.

Incluso en ese escenario, es recomendable mantener un registro de los eventos que han sucedido por un periodo de tiempo. Entre 24 horas y 7 días suele ser un intervalo aceptable para estos eventos que son efímeros.

Patrón *outbox* y Consistencia eventual

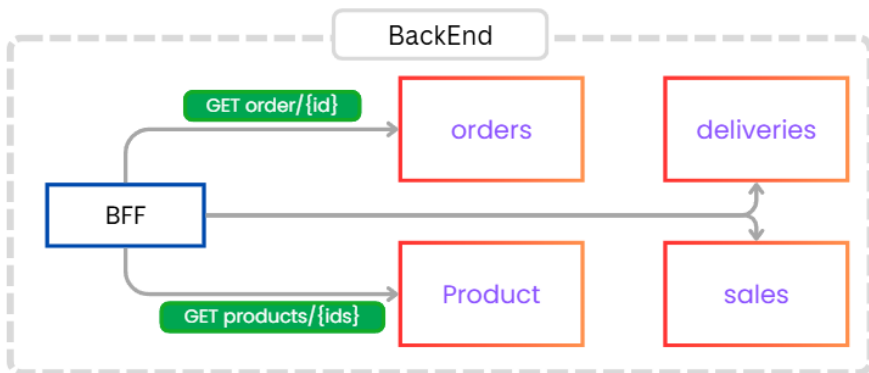
Al crear sistemas, lo podemos hacer de forma síncrona a través de llamadas entre servicios, o podemos hacerlo de forma asíncrona a través de los eventos. Pero no solo eso, podemos mezclar ambas, que es lo más normal.

La clave viene en saber si necesitamos saber dichos valores de forma correcta siempre, o podemos tener dichos valores almacenados en una tabla en nuestra base de datos. Este patrón es conocido como el patrón *outbox*.

Y está directamente relacionado con la consistencia eventual (*eventual consistency*), ya que los datos serán correctos, eventualmente, tanto en la tabla original, como en la nuestra con la información duplicada.

Ojo, eventualmente no significa mañana o en unos minutos, normalmente serán unos milisegundos y, en el peor de los casos, segundos.

Si volvemos a nuestro ejemplo de los pedidos y los productos, cuando leemos los pedidos estamos haciendo una llamada al servicio de productos para leer la información del nombre y de la imagen.



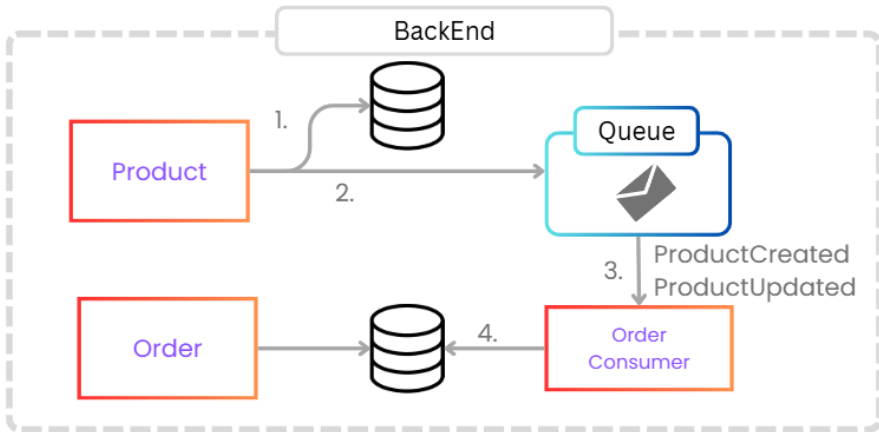
Y da igual si esta llamada la hacemos desde el BFF o desde el propio servicio de pedidos, para sacar la información completa de un pedido no nos sirve con sacar únicamente el ID.

Lo mismo sucede para otros servicios como envíos o ventas si son necesarios.

Esta es una de las opciones, realizar todas las llamadas desde la capa de agregación donde siempre vamos a tener los datos actualizados.

La otra opción es utilizar la comunicación asíncrona a través de los eventos para propagar la información.

En el siguiente esquema podemos ver una clara representación:



1. Creamos o modificamos un producto existente y lo guardamos en la base de datos.
2. Publicamos un evento de sistema o integración.
3. El *consumidor* de los pedidos consume los eventos de *ProductCreated* o *ProductUpdated*, ya que estos eventos son los que contienen la información que necesitamos (nombre, imagen).
4. El consumidor actualiza la base de datos en el lado de pedidos con la nueva información.

Este proceso nos hace indicar que, a partir de ahora, tendremos la información duplicada en ambos servicios, y esto es cierto y es así, pero el *Source of truth*, o fuente de verdad, será siempre el servicio originario.

En muchos escenarios es más rentable, en temas monetarios y de rendimiento, tener la información duplicada que tener que hacer llamadas entre servicios. Esta es la implementación del patrón *Outbox*.

Es un patrón muy útil sobre todo en aquellos datos que se repiten mucho, como puede ser el nombre de un producto. Todos los pedidos con ese producto van a tener dicho nombre, por lo que, si podemos tener la información duplicada una vez en la base de datos, es mejor que miles de llamadas.

La gran pega que tiene el uso del patrón *outbox* es que el funcionamiento correcto del sistema depende de tu cola, *Stream* o *service bus* ya que si realizas un cambio en el nombre de un producto y tu sistema de comunicación está caído o colapsado, y va lento, la información no va a ser actualizada, por lo tanto, el nombre que el usuario lea cuando abra su pedido va a ser diferente al nombre que aparece cuando cargas el propio producto en la interfaz.

Este es el motivo por el que, personalmente, me gusta agrupar la explicación del patrón *outbox* junto con la consistencia eventual (*eventual consistency*), ya que ambos están fuertemente relacionados. Porque los datos son los mismos, eventualmente, comúnmente rápido, pero puede darse el caso de que tarde un rato.

El uso del patrón *outbox*, o no, depende no solo de la arquitectura de tu sistema, sino de la arquitectura de cada sección de tu sistema.

En este ejemplo está claro, la información de los productos es obvia, y posiblemente podamos sincronizar también la de ventas para los precios. Por el contrario, la información de los envíos no nos merece la pena, la carga es 1:1 junto con la lectura de los pedidos. Pero no solo eso, los envíos contienen información personal que hace que el cliente pueda ser identificado, lo cual requiere seguridad extra con los datos. Así que, si mantuviéramos una copia de lectura de estos datos, tendríamos que

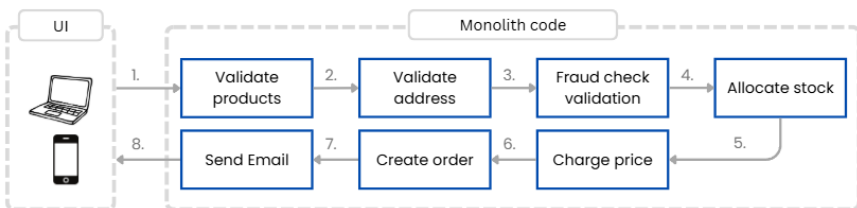
tratarlos igual que hace el servicio envíos, lo que implica un mayor mantenimiento.

Patrón SAGA

Con la llegada de la arquitectura de eventos, donde un servicio genera un evento, el cual es procesado por otro, nos damos cuenta de que el procesamiento de transacciones no es tan fácil, porque tenemos procesos a través de múltiples dominios.

Para facilitar el funcionamiento correcto de nuestro sistema, utilizaremos el Patrón SAGA, ya que nos asegura la consistencia en nuestras transacciones, cuando estas contienen múltiples servicios que se comunican de forma asíncrona e indirecta a través de eventos.

Por ejemplo, en un mundo monolítico si un usuario crea un pedido tenemos el siguiente proceso.



1. El usuario crea la *request*.
2. Validamos productos.
3. Validamos la dirección del envío.

4. Ejecutamos el *Fraud check*.
5. Cambiamos la información del inventario, moviendo un elemento de *OnHand* a *Allocated*, aquí comienza una transacción.
6. Cobramos el dinero al usuario.
7. Creamos el pedido y finalizamos la transacción.
8. Enviamos un email.

El proceso, a primera vista, se ve muy sencillo y seguro, podemos tener varios puntos problemáticos, las validaciones son obvias, en muchos casos son 'ifs' uno detrás de otro. Pero la parte de asignar el stock es cuando puede empezar a fallar. Puede darse el caso de que el usuario quiera comprar diez, pero solo tengamos siete, así que eso dará un error. Devolvemos un error al usuario sin problema, al fin y al cabo, no hemos cambiado nada.

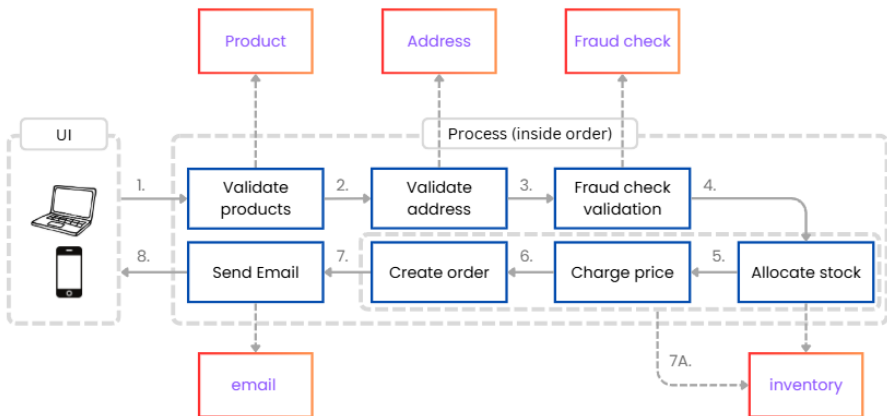
Ahora, el siguiente paso es pagar, y suele requerir una entrada manual por parte de los usuarios. Aquí el problema puede ser diferente: que el usuario escriba mal la tarjeta de crédito, o que el usuario directamente cancele. Si ese fuera el caso, la asignación de productos la tenemos que deshacer. Pero ese puede no ser un problema, ya que, si se está ejecutando dentro de una transacción de la base de datos, al cancelar el proceso las modificaciones de la base de datos no suceden.

En el caso de que falle a la hora de crear el registro del pedido en la base de datos, si pudiera ser un problema, y ver en cada caso que es mejor, si devolver el dinero al usuario para que vuelva a empezar o cobrarle y añadir el pedido a mano después. Pero vaya, que es muy extraño que este paso falle.

Finalmente, en el caso del email, al completarse la transacción enviamos un email.

Este es un proceso normal y típico, dentro de un monolito donde tenemos todas las partes del sistema acopladas entre sí.

Si pasamos al siguiente nivel, el cual es un sistema de microservicios, tendríamos el proceso completo dentro del servicio de pedidos, y, como vemos, es similar, pero algo distinto, ya que ahora involucramos los servicios que contienen la información de los elementos que necesitamos:



1. El usuario crea la *request*.
2. Validamos productos con el servicio de productos.
3. Validamos la dirección con el servicio de dirección.
4. Posiblemente tengamos un servicio distinto para el *Fraud check*.
5. Cambiamos la información del inventario, esta vez utilizando el servicio de inventario.

6. Cobramos al usuario. Igual que antes, la diferencia es que si el usuario cancela o el pago falla, debemos de tener esto en cuenta a la hora de escribir el proceso, porque tendremos que ejecutar 7A.
7. Creamos un pedido.
 - A. Volvemos a poner el inventario como estaba si el pago ha fallado, o incluso volver a devolver el dinero si lo que ha fallado es la creación del pedido.
8. Enviamos un email.

Este proceso parece ser similar al anterior, pero tiene dos diferencias clave.

- Utilizamos microservicios, y, por tanto, no nos podemos salir de las limitaciones del servicio, así que si necesitamos comprobar que un producto es válido lo hacemos a través del servicio de productos.
- Ya no tenemos transacciones, por lo que si tenemos una parte del proceso que puede afectar a otro, debemos mantener la lógica del estado en dicho servicio, para poder deshacer los cambios que hayan sucedido.

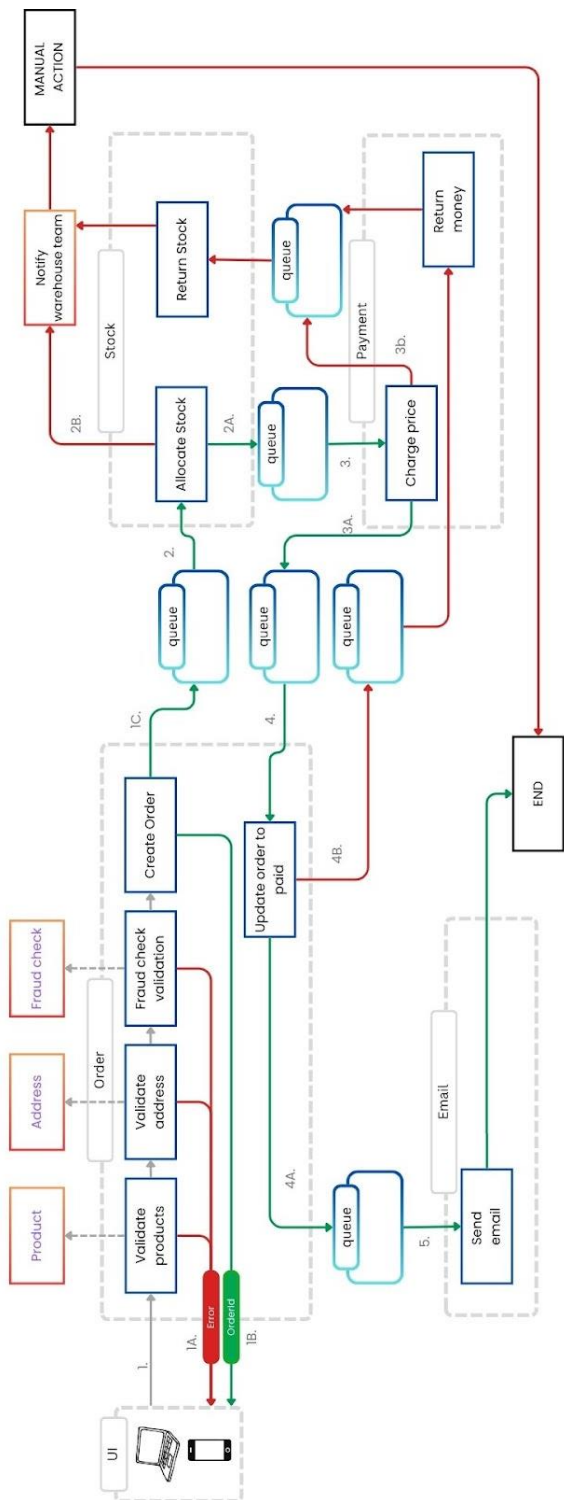
Pero la cosa no acaba ahí, la gran mayoría de tiendas con una arquitectura de microservicio utilizan también la comunicación asíncrona a través de los eventos y el patrón productor consumidor.

Esto permite desacoplar los procesos, pero, a su vez, complica el tema de las transacciones, porque para poder deshacer un cambio ya no vas a tener una transacción o un estado en memoria, sino que vas a utilizar el Patrón SAGA.

Definir una SAGA

El Patrón SAGA, o más bien una SAGA, consiste en una serie de acciones que están fuertemente ligadas entre ellas y completan un flujo de principio a fin. Estas transacciones incluyen, tanto el proceso cuando todo funciona bien, como la parte de compensación que es el flujo, o los flujos, cuando una parte del proceso falla.

Lo más importante de una SAGA es tener todos los procesos definidos, porque así nos aseguramos de no dejarnos el proceso a medias en un estado inválido.



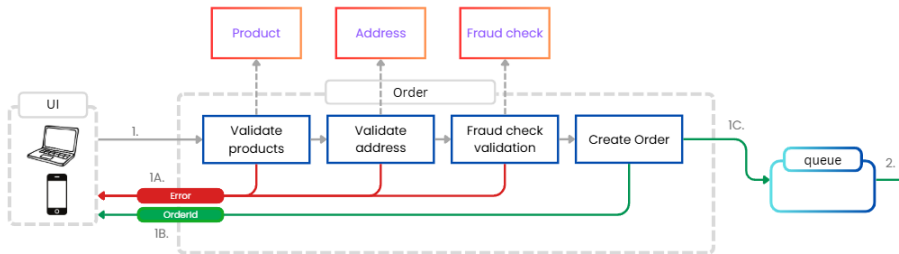
Como puedes ver (aunque no se si muy bien), está basado 100% en la comunicación asíncrona a través del patrón producto consumidor y los eventos, y ello complica mucho el proceso, especialmente si lo comparamos con un monolito.

Pero no solo eso, sino que hemos cambiado ciertas y la interacción entre ellas.

Nota: en cada una de las secciones voy a poner la sección del diagrama anterior que corresponde, porque creo que no se ve muy bien.

1 creación del usuario

El usuario crea la *request*, y, desde el servicio de pedidos, realizamos las validaciones y creamos el pedido en la base de datos. Pero el proceso no termina ahí.



En caso de que alguna de las validaciones no haya sido correcta, devolvemos el error al usuario (1A).

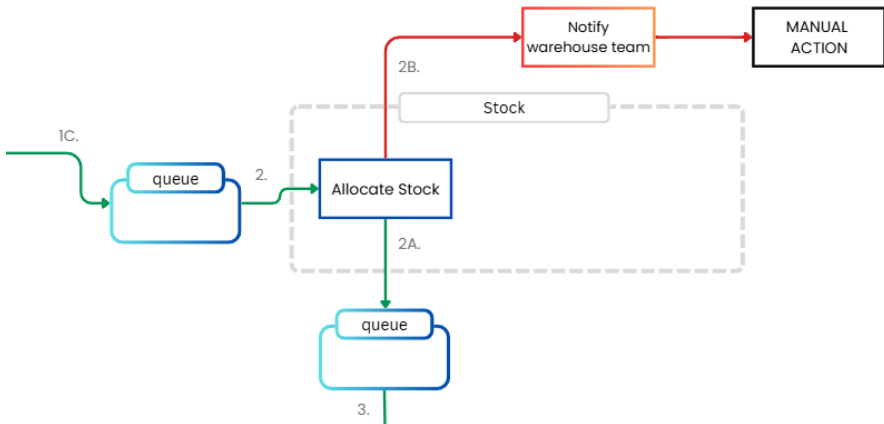
Si todo ha funcionado bien, como es esperable, devolvemos el ID del pedido al Usuario y le indicamos que todo está bien, y que recibirá un email de confirmación (1B).

Además, generamos un evento con la creación del pedido (1C).

Si todo va bien en el resto del proceso, está será la última interacción del usuario con el sistema.

2 sistema de inventario

Ahora toca modificar el inventario, donde asignamos los productos. Lo ideal es que sea un proceso sencillo y funcione sin problemas, con lo que generamos un evento (2A).



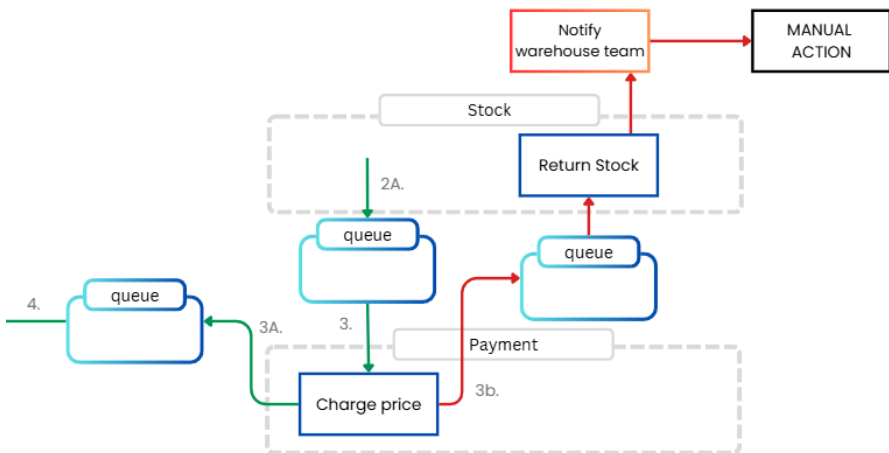
Pero puede ser que no tengamos suficientes elementos en el inventario. Recuerda que tenemos consistencia eventual, puede ser que al validar los productos en el paso anterior tuviéramos información que sea unos segundos anteriores y tuviéramos suficientes productos, pero cuando este nuevo evento ha llegado al servicio de stock, la realidad es que no los hay.

Ahora mismo el usuario no tiene acceso al sistema, con lo que no podemos devolver un error. En esta acción deberemos de notificar al

equipo de almacén (2B), o a las personas responsables, ya que lo más probable es que requiera una acción manual.

3 cobro del pedido

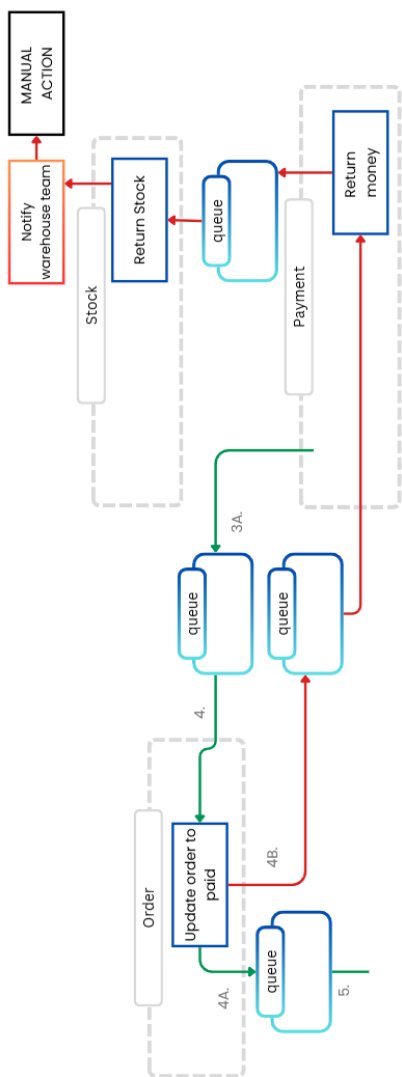
Como tercer paso pasaremos a cobrar el precio del pedido, y esperamos que todo funcione bien, con lo que podemos simplemente cobrar, y generar un evento como que hemos cobrado de forma correcta (3A).



¿Pero qué pasa si algo va mal? Sencillo. Que debemos volver a liberar los productos. Para ello generamos un evento de error y el sistema de inventario tendrá que asegurarse de que los productos no se quedan “perdidos” (3B).

4 actualizar el pedido

Una vez el pedido está pagado, lo que tenemos que hacer es actualizar el pedido. Este cambio puede ser tan sencillo como cambiar un valor en un registro de la base de datos, o ser más complejo. Idealmente, es fácil, todo funciona bien y generamos nuestro último evento del proceso (4A).

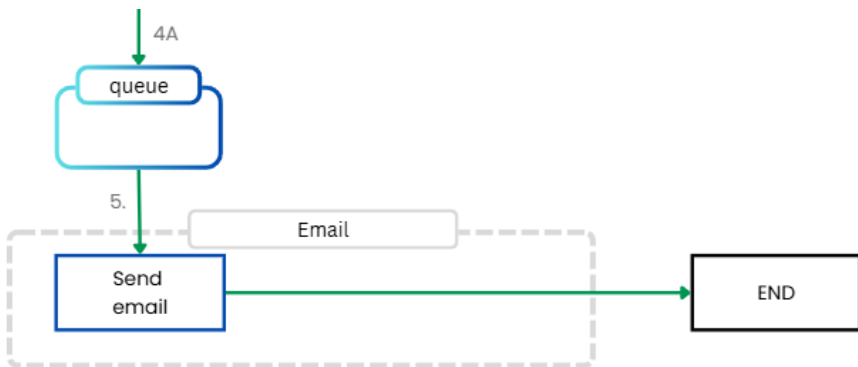


Desafortunadamente, todos sabemos que no todo es tan fácil. Posiblemente tengamos otras validaciones o, simplemente, el proceso puede fallar por algún motivo que desconocemos.

En este caso, dependerá de la lógica de dominio de sistema y lo que quiera la empresa, pero puede darse el caso de que, si tenemos un error en este punto, quizá queramos cancelar el pago y volver a poner el inventario como estaba (4B).

5 enviar email

Finalmente, el último paso de nuestro proceso es enviar un email al cliente.



Donde se termina nuestro proceso.

CAPÍTULO 11

Entrevista de diseño nivel medio

La siguiente de nuestras simulaciones de entrevista es un nivel un poco superior a la anterior, pero el proceso de implementación que vamos a seguir es el mismo.

A la hora de realizar la entrevista podemos ver que en algunas empresas pueden ser un poco diferentes, ya que hay empresas sin entrevista específica de diseño o, donde sí tendremos una entrevista exclusiva, lo que quiere decir que tendremos entre treinta y cinco y cuarenta y cinco minutos para esta sección.

Da prácticamente igual qué sistema te pidan, es muy improbable que, en cuarenta y cinco minutos de tiempo a diseñarlo. Lo que se busca en este tipo de entrevistas es ver cómo te desenvuelves, y si tienes en mente todos los apartados clave.

No olvides que sigue siendo una entrevista donde ambas partes quieren que todo vaya bien, lo que implica que puedes mantener una conversación. De hecho, como siempre, lo recomendable es pensar en voz alta y hacer muchas preguntas donde sea necesario.

El tipo de diseño puede variar mucho dependiendo del tipo de empresa para la que sea la entrevista.

Por ejemplo, un conocido en esta entrevista para una empresa de videojuegos tuvo que diseñar un sistema de emparejamiento de partidas.

En nuestro caso, vamos a ir por algo menos nicho y dentro del entorno empresarial.

Recuerda que no hay una solución válida, hay muchas; lo que importa es el proceso.

Diseño de un sistema de críticas

“Diseña un sistema para añadir críticas/opiniones de empresas, similar a Google Review o a Tripadvisor.

- El sistema debe permitir el uso de la aplicación tanto a empresas como a usuarios.
- Los usuarios pueden añadir críticas/opiniones y dejar una valoración de estrellas.
- Las empresas deben ser capaces de añadirse al sistema. Siendo capaces de ver las críticas u opiniones y responder a las mismas.
- Las empresas pueden acceder al sistema a través de una integración para recibir las últimas opiniones o críticas de los clientes y utilizarlas en sus sistemas.
- Los usuarios pueden acceder tanto a través de la aplicación móvil como del *FrontEnd*.

Como puedes ver, estos requerimientos son mucho más extensos y explícitos que en la entrevista para *Junior*, y eso es porque se espera más de ti. Aun así, el proceso de la entrevista es el mismo. Primero de todo debemos solventar cualquier duda que tengamos.

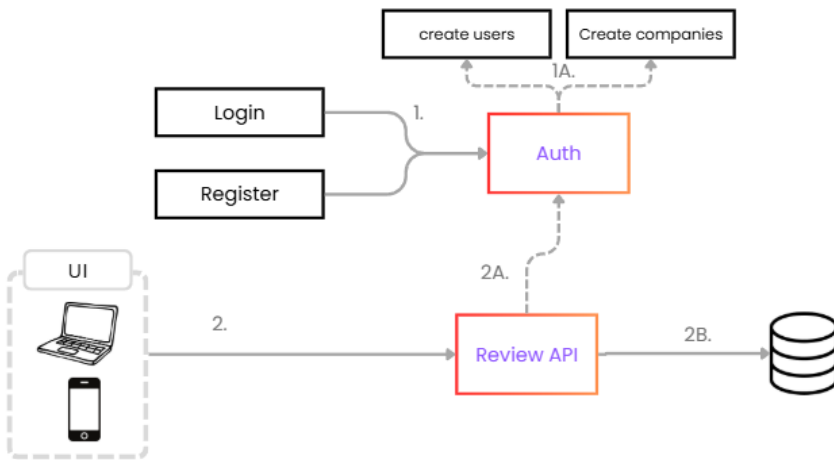
Candidato: ¿Son las opiniones y críticas lo mismo, o son entidades diferentes?

Entrevistador: Son lo mismo, puedes tratarla como una sola, puedes utilizar “crítica”.

Notas:

- En el diseño, utilizaremos la palabra *Review*.
- En cada sección habrá un diseño de dicha sección, pero solo al final estará el diseño completo debido a su tamaño.
- Cuando veas una lista numerada, significa un pensamiento en voz alta por parte del candidato, donde el entrevistador te escucha.

Recuerda que en esta entrevista se espera un nivel mayor del entrevistado. Es posible que con la información que tienes ahora, ya seas capaz de empezar con el diseño.



- 1) Tanto el *Login* como el registro suceden a través del servicio de autorización.
 - a) El proceso para dichos servicios es diferente, pues necesitan diferentes roles o permisos.
- 2) El *token* generado se puede utilizar desde el *FrontEnd* para acceder al sistema y ya bien sea añadir las críticas si son usuarios.

- a) Debemos comprobar siempre el sistema de *Auth* para validar los permisos.
- b) Finalmente, insertar la crítica o la información de la empresa.

Entrevistador: No veo una forma para crear empresas, has mencionado creación de usuarios con diferentes roles basándose en si son miembros de la empresa o usuarios normales.

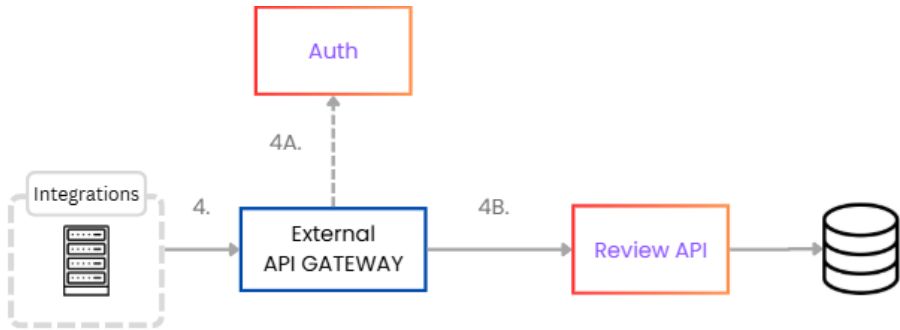
Candidato: Entonces debo añadir la creación de empresas; lo puedo hacer cuando se registran o creando un proceso separado.

Entrevistador: Como tú prefieras, es tu sistema. Pero ten en cuenta que crear empresas puede requerir un proceso manual de verificación.



- 3) El proceso de crear empresas genera un evento, el cual es leído por un sistema de verificación. Es posible que este sistema realice una llamada a la empresa y verifique la información fiscal de una forma manual.
- a) Si todo va bien, genera un evento, el cual es leído por un procesador de forma asíncrona.
 - b) Inserta los valores en la base de datos.
 - c) Genera un evento que envía un email a la empresa para confirmar que todo está correcto.

Candidato: Pese a no estar en el diagrama, si una empresa quiere cambiar sus datos, accederá a través de la aplicación web o móvil y el servicio de empresas.



- 4) Las empresas pueden acceder al sistema utilizando un servicio de integración en una API Gateway. En este caso la API Gateway va a asegurarse de que la integración utiliza un acceso con permisos para el uso de la API.
 - a) Esto lo podemos hacer con una API Key, o en el sistema de *Auth*. Las empresas pueden crear usuarios para la API con una *Key* y un *Secret*, para una integración sencilla.
 - b) Cogemos los datos de las últimas cinco críticas a través de la API de críticas.

Entrevistador: ¿No crees que le falta algo en la implementación?

Candidato: No sé a qué te refieres, parece todo correcto.

Entrevistador: Con la implementación actual es muy fácil abusar del sistema.

Candidato: Es fácil abusar, pero a la vez esperamos un uso normal ya que el uso de la única funcionalidad en la API externa es el acceso a las últimas cinco críticas, por lo que esperamos que las lean y las impriman en una sección de su web, que normalmente no tienen mucho tráfico.

Aun así, si fuera un problema podríamos implementar un *Rate limiting* o, incluso, una caché con un tiempo de vida de los elementos de diez o quince minutos, para que devuelva los mismos datos durante este tiempo sin sobrecargar el sistema.

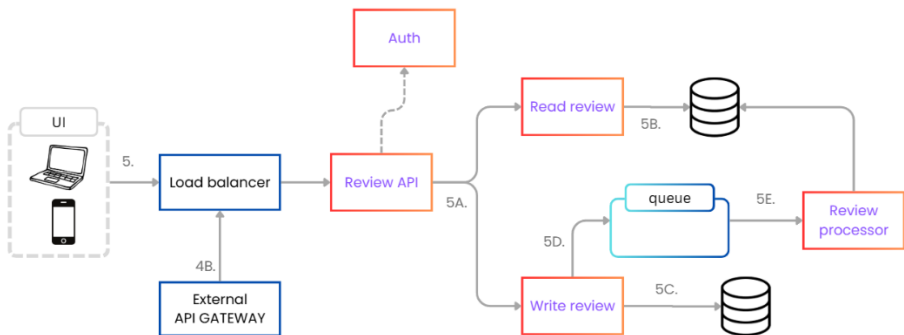
Entrevistador: Bien, pero sigue faltando una parte importante, que es crucial, principalmente cuando tenemos un uso elevado del sistema.

Candidato: ¿Te refieres a más instancias?

Entrevistador: Sí, tendremos más instancias. Pero para acceder a esas instancias necesitas algo.

Candidato: ¿El *Load Balancer*?

Entrevistador: Exactamente. Además, quiero que te pongas en una situación donde tienes miles de empresas en el sistema y, por lo tanto, un uso altísimo de la web, especialmente para consultar empresas. ¿Qué podrías hacer al diseño para que el rendimiento y la escalabilidad fuera mejor?



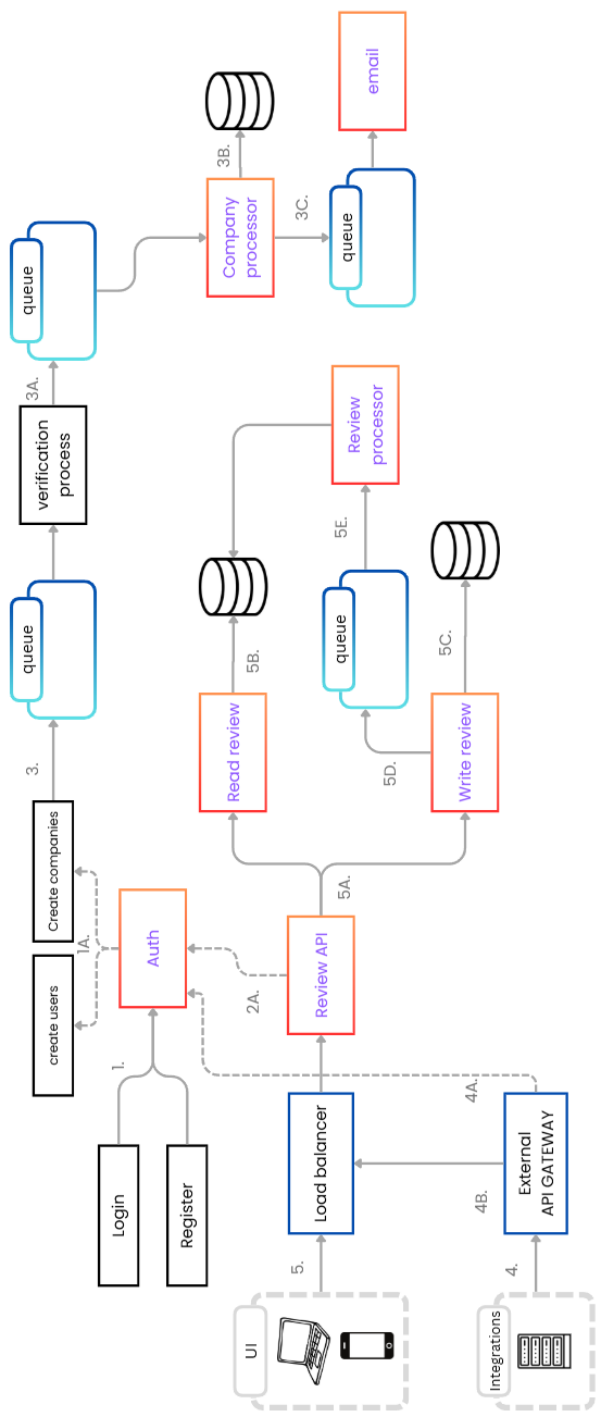
- Lo primero de todo es añadir el *Load Balancer*, el cual actuará como punto de entrada desde la aplicación web como desde la aplicación móvil. Si quisiéramos podríamos tener una API Gateway también.

- a) Lo que vamos a hacer es dividir la API de *Review* en dos, dejando la API en sí como un *proxy* hacia un servicio de lectura y otro de escritura exclusivos para dichas acciones.
- b) Todas las lecturas tendrán su propia base de datos.
- c) Las escrituras tendrán su propia base de datos.
- d) Cuando una crítica se crea, esta genera un evento.
- e) Dicho evento es consumido por un servicio que procesa los eventos e insertará la crítica en la base de datos de lectura.
- f) (4B) Al añadir el *Load Balancer*, la API Gateway con el acceso externo también apuntará al nuevo *Load Balancer*.

Lo más probable es que tu entrevista llegue hasta aquí, ya que cuarenta y cinco minutos dan para lo que dan, y siempre se suele dejar un par de minutos al final para que el candidato realice alguna pregunta.

Aun así, si eres rápido y tienes más tiempo, quizá el entrevistador te haga más preguntas, o tú mismo, si ves que hay tiempo, puedes mencionar elementos extra.

La arquitectura final la podemos en la siguiente página:



Donde podemos ver el uso de muchos de los elementos mencionados en el libro.

Utilizamos la comunicación asíncrona a través del patrón productor consumidor y los eventos para crear empresas en el sistema, así como enviar los emails o, junto al patrón CQRS, separar la información entre lectura y escritura y sincronizarla entre unas y otras.

Pese a que no está en el diagrama, ya que no suele ser necesario en las entrevistas, esta arquitectura sería imposible si no tuviéramos servicios para almacenar la información del resto de servicios o elementos de la infraestructura, así como sus contraseñas.

Tenemos especificado el acceso externo a través de una API Gateway, especificando que controlaremos el uso y aplicaremos un *Rate Limiting*, si fuera necesario.

Accediendo a los diferentes servicios a través de un *Load Balancer*, para así no acceder directamente a las instancias y que el tráfico de esa aplicación esté bien distribuido.

Finalmente, si hay tiempo, se pueden añadir explicaciones sobre que desde la API de críticas a su Writer/Reader el código utilizara gRPC y por qué realizamos esta acción. O, incluso, añadir una capa de composición que se encargará con una única consulta desde el *FrontEnd* en cargar la información de la empresa y las *Reviews* para mostrar toda la interfaz.

El arte de observar sistemas

Dentro de todos los sistemas tenemos partes que no suelen gustar, ya bien sea porque llevan mucho tiempo, como puede ser la documentación, o porque son tediosas y nunca vamos a utilizar, o bueno, eso es lo que se piensa, porque situaciones como *logs*, y monitorización siempre pensamos que no son necesarias, pero sí lo son.

Unos para ser preventivos a posibles fallos y situaciones inesperadas del sistema, y los otros para ser reactivos cuando algo no ha funcionado como se esperaba y debemos arreglarlo.

Por ello, este capítulo, es uno de los más importantes, o quizá el que más de todo el libro. Aquí vamos a hablar de lo que pasa detrás de la lógica, de cómo observar un sistema para evitar que colapse y que siempre esté funcionando bien, y de, si algo no va bien, saber cómo reaccionar.

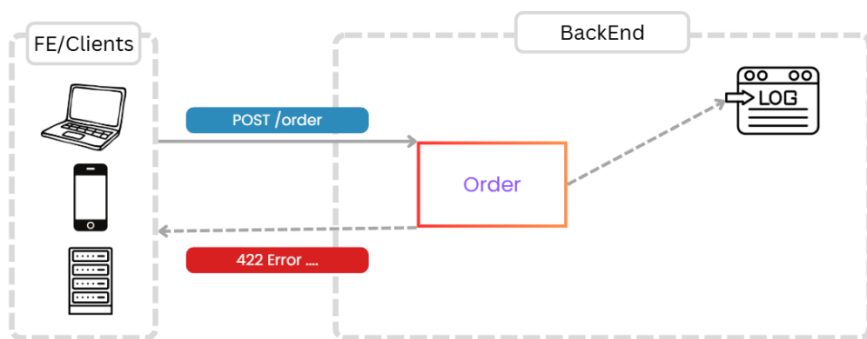
LOGS

La unidad más básica para observar aplicaciones y sistemas es a través de los *logs*, ya que estos son específicos de una única aplicación. Por ello, es la primera parte que vamos a explicar.

Los *logs* son la parte de nuestro sistema que guardan un registro de los errores o acciones que suceden en una aplicación específica.

Por ejemplo, si creamos un pedido en nuestro sistema sobre un producto que no existe, el sistema le devolverá al usuario un mensaje de error. Pero, a su vez, almacenará un registro sobre la acción que ha sucedido, en forma de error. Con la cantidad de información necesaria para ser capaces de no solo identificar el error, sino también ser capaces de solucionarlo.

Al haber enviado el mensaje de error al usuario, este será específico, pero el registro con toda la información extra no es enviada al usuario final; sino que es enviada a un servidor diferente o, bueno, a un almacén de *logs*.



La información que queremos almacenar dentro de los *logs* puede variar bastante dependiendo del tipo de aplicación, pero de forma habitual siempre vamos a querer guardar cierta información, como es:

- 1) Servicio que genera el problema.
- 2) Mensaje de error.
- 3) Ubicación del error (*stack trace*).
- 4) Nivel del *log*.

- 5) ¿Quién genera el problema, usuarios de la web, móvil o clientes?
 - a) En muchas ocasiones un problema viene generado únicamente por uno de los clientes o consumidores de nuestra aplicación. Incluir esta información puede ser vital para entender el impacto que puede llegar a causar.
- 6) Datos que hacen generar el error.
 - a) Generalmente todas las propiedades necesarias que puedan hacernos la vida más fácil a la hora de investigar el error, y van a ser datos específicos de cada aplicación.
 - b) Aquí debemos tener en cuenta, además, no guardar datos personales para no tener problemas con la Ley de Protección de Datos (GDPR).

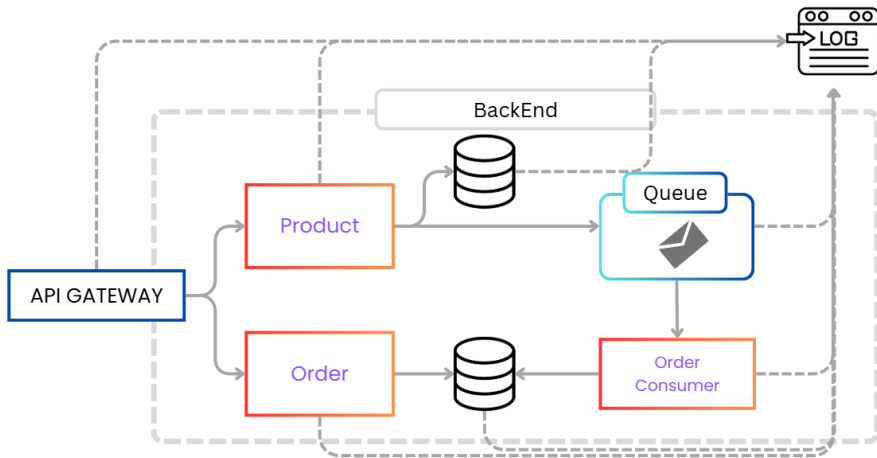
Los *logs* no son únicamente creados de forma manual, sino que los propios *frameworks*, o librerías, pueden generar los propios *logs*; aunque está en la responsabilidad de cada aplicación el configurar el servidor de logs para que no se pierdan.

Por defecto, en la gran mayoría de lenguajes y aplicaciones, los *logs* van a estar configurados para que se almacenen, ya bien sea en local, o salgan imprimidos por consola. Como te puedes imaginar, para hacer pruebas en tu máquina, mientras desarrollas, no necesitas más. Pero una vez despliegas la aplicación en producción, esta implementación no es aceptable.

Y no es aceptable por el simple hecho de que necesitarías un sistema capaz de ir por todas las aplicaciones recolectando los *logs* para estar seguros de que no haya ningún problema.

Es mucho más sencillo realizar el proceso inverso; configurar todas las aplicaciones para que envíen los *logs* a un sistema centralizado.

Cuando menciono “todas las aplicaciones”, significa que hablo tanto de las propias aplicaciones de dominio, con la lógica de nuestro negocio, como de cualquier aplicación de la infraestructura, como puede ser la base de datos, o el *Event Bus*.



Pero no solo eso. Lo más habitual que te puedes encontrar en las empresas es que el servicio de *logs* esté externalizado en un servicio de terceros. Ojo, no quiere decir que no puedas almacenar tú mismo los *logs*, pero si no has trabajado nunca en un sistema así, te darás cuenta de que el número de *logs* que se crean en los sistemas es masivo, y los *logs* suelen ser uno de los mayores costos de las empresas. Personalmente, he visto muchos escenarios donde el coste de los *logs* es mayor al costo de la ejecución de la aplicación que registra dichos *logs*, así que hay que tener cuidado.

Finalmente, hay que mencionar que los *logs* no son tan exclusivos de los sistemas distribuidos, y que hoy en día cualquier sistema, incluido sistemas monolíticos, utilizan *logs* y, sin ellos, estaríamos muy perdidos ante cualquier problema.

Niveles de *logs*

Al crear *logs*, estos se pueden crear con diferentes niveles, que hacen referencia a la importancia o a la prioridad de estos.

Antes de describir los niveles como tal, debes saber que en producción no deberías almacenar nada que sea un nivel menor a error, ya que la creación de *logs* cuesta dinero y tiene un efecto en el rendimiento de las aplicaciones. Incluso aunque el *log* en sí no sea ejecutado porque es de un nivel menor al mínimo configurado, la propia línea de crear el *log* afecta al rendimiento. Obviamente, en un sistema con diez llamadas por minuto no vas a notar la diferencia, pero si tienes varios miles de llamadas por segundo y varias líneas de *logs* que no aportan nada, sí lo vas a notar.

Ojo, no quiere decir que, en algún punto concreto del desarrollo del sistema, o, durante un tiempo limitado, para intentar encontrar la raíz de un problema en producción no puedas añadir *logs*. La norma que menciono es en general para el funcionamiento normal de una aplicación.

Estos niveles varían en nombre dependiendo de qué lenguaje de programación estés utilizando o, incluso, de qué proveedor de servicios estés utilizando. Al final lo que importa es saber que tienen un significado y cuándo utilizarlos.

Trace

Configurar *logs* como *Trace* suele significar que quieres registrar información del proceso, y suelen contener información sensible.

Lo recomendado en este caso es no habilitarlo nunca en producción. Este tipo de *logs* son utilizados por los propios *frameworks* para dar información en caso de que estén habilitados, y dan mucha, lo que implica un alto costo en el rendimiento de la aplicación.

Debug

El nivel *Debug* se utiliza principalmente para añadir *logs* que van enfocados a los desarrolladores, los cuales pueden ser útiles para *debuggear* problemas.

Por ello, contendrá información útil del proceso.

Su uso debe ser muy limitado, acotado exclusivamente a cuando no podemos replicar un fallo en los entornos de test, o desarrollo, y debemos intentar averiguar la raíz del problema en producción.

Information / info

El nivel Información suele incluir mensajes que informan sobre el estado de un proceso, por ejemplo, cuando una aplicación empieza a ejecutarse o, incluso, cuando va a parar. También es común verlo para indicar cambios en los estados de ciertos procesos, si son muy largos.

Warning / warn

El nivel *warning* lo solemos utilizar para especificar que algo ha sucedido pero que, posiblemente, no debería suceder.

Un ejemplo común es registrar cuánto tiempo tarda una dependencia en completar una *request*. Si por lo general tarda un segundo, podemos tener un *log* de *warning* que nos alerte si tarda más de cinco.

O quizá estamos migrando del proceso A al proceso B, y ponemos un *warning* en el A con la información de que parte del servicio está utilizando ese proceso, para así migrarlo sin romper nada.

La idea principal es evitar errores futuros. Si tienes un *warning*, quiere decir que, si lo ignoras, eventualmente será un error.

Error

El nivel *Error* lo utilizamos para indicar problemas o errores del sistema. Ojo, un error del sistema no es requerir a un usuario ser mayor de edad y que ponga que tiene 13, y que salte un error. Eso es una validación.

Un error del sistema es cuando hay un error inesperado. Por seguir con el ejemplo de antes, quizá una dependencia no funciona bien. En ese caso recibiremos un error. O quizá la base de datos está caída y, por ello, parte de nuestra aplicación no funciona.

En mi experiencia los *logs* de error son los más comunes, ya que suelen ser los únicos justificables a la hora de ser puestos en el código, ya que se ponen en situaciones donde, por norma general, el desarrollador no puede hacer nada para evitarlos. Por ejemplo, que una dependencia de terceros esté caída.

Sinceramente, van a ser los únicos que van a ser comprobados a la hora de mirar los *logs* de una aplicación por un error que nos pueda reportar un cliente.

Critical / Fatal

El nivel *Critical*, o fatal, según el lenguaje, se utiliza para indicar que una aplicación, o sistema, está en un estado donde no funciona. Por norma general este tipo de mensajes los traen los propios *frameworks* de los lenguajes donde ejecutamos las aplicaciones, no son escritos por los desarrolladores de una aplicación de dominio.

La idea de estos mensajes de error es que reciban un chequeo inmediato, porque si se dan, la aplicación no funciona.

El exceso de logs

¿Sabes qué es lo que sucede cuando tienes muchos *logs* por todas partes que no están agrupados correctamente y todos juntos causando ruido?

Sí, lo has pensado bien, que se ignoran.

A no ser que tengas aplicaciones de alto uso, tener muchos *logs* puede no ser un problema de rendimiento, aunque quizá sí lo sea monetario. Lo que implica que no afecta directamente a los equipos o desarrolladores que los añaden, y, por ello, ponen muchos *logs*.

Logs de información cuando empieza un proceso, un *log* al principio de cada llamada API, y otro al final, un *log* antes de llamar a la base de datos, y otro después. Así podríamos estar hasta el infinito. Este tipo de *logs* no sirven absolutamente para nada en circunstancias normales, y lo único que hacen es gastar recursos.

Hay situaciones donde este tipo de *logs* puede ser importante. Por ejemplo, en una aplicación nueva, quieres comprobar que todo funciona con normalidad durante un tiempo. En ese caso vale: pones tus *logs* y después los quitas.

Dejarlos ahí solo causa ruido y genera una maraña que nadie sabe, ni quiere, descifrar. Además, pueden generar confusión si alguna vez tenemos fallos de sistema y tenemos que mirar a los *logs*.

La recomendación es utilizarlos con cabeza, especialmente en producción, intentando añadir lo menos posible. Una regla muy buena es que, para poner un *log*, tenga que ser un motivo para dejar de hacer lo que se está haciendo, e ir a mirar que ha generado ese *log*.

Si sigues esta regla, te darás cuenta de que muchísimos *logs*, que tenemos en las aplicaciones, sobran.

Health Check

Al trabajar en sistemas distribuidos, cada parte del sistema o aplicación depende de una gran cantidad de aplicaciones y sistemas para su correcto funcionamiento. Por ello, debemos de estar comprobando continuamente si cada aplicación está siendo ejecutada como se espera.

Esta comprobación la podemos hacer de forma manual, comprobando cada aplicación una a una continuamente, o, la opción más normal, monitorizando el proceso a través de los *health check*.

Un *health check* no es más que una comprobación que se hace sobre una aplicación. Si es una Web API, este *health check* suele ser un *endpoint* de la propia aplicación.

Dentro de los *health check* hay dos características principales que se van a comprobar.

Nivel de respuesta de un *health check*

Asumamos que vamos a comprobar una API, por lo tanto, vamos a realizar una llamada HTTP a un *endpoint* específico. En nuestro caso puede ser `/status`, pero se puede configurar para que sea cualquier *endpoint*.

Este *endpoint* nos va a devolver tres niveles distintos a la hora de responder.

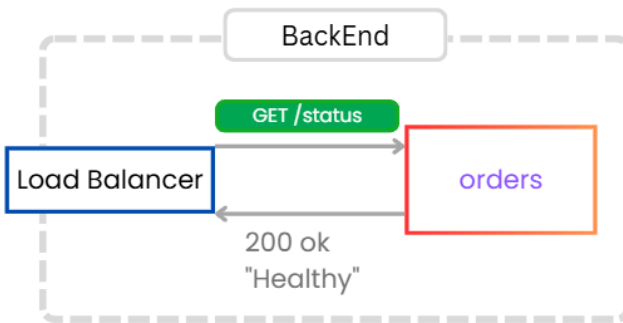
1. **Healthy:** todo funciona con normalidad.
2. **Degraded:** La aplicación funciona, pero algo no funciona bien. Por ejemplo, tener una dependencia que responde muy lento o, quizá, esa dependencia no funciona, y una parte de la API no funciona.
3. **Unhealthy:** la aplicación está caída, o su funcionamiento no es el esperado.

Comprobaciones en los *health check*.

A la hora de realizar comprobaciones podemos hacer las comprobaciones de dos tipos diferentes.

Comprobación básica

Este tipo de comprobación es la más básica. Tenemos nuestro *endpoint* /status, el cual devuelve un “ok” o quizá “healthy”, depende un poco de cada empresa e implementación. Lo que sí que hace es devolver un código de estado 200.

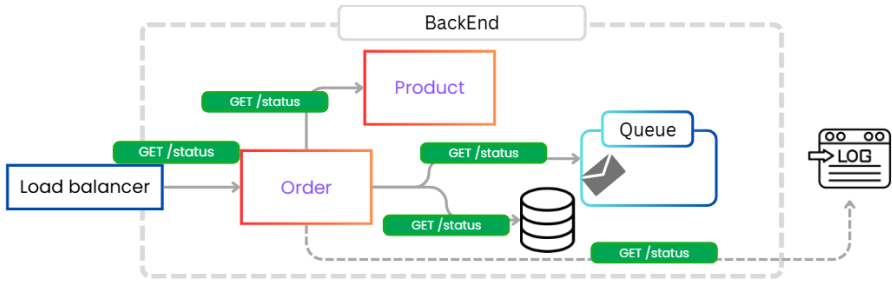


Este *healthy* es devuelto por el propio cuerpo del controlador, como una cadena de texto plana.

Comprobación de dependencias

Pero tenemos otro tipo de comprobaciones, y aquí es donde vamos a comprobar no solo que nuestro servicio funciona como es esperado, sino que vamos a confirmar que todas las dependencias de las que depende nuestro servicio funcionan como es esperado.

Por ello, desde la propia aplicación realizamos llamadas a los diferentes servicios para estar seguros de que todo funciona como debería.



De esta forma, cuando una aplicación comprueba el estado de nuestra aplicación, lo que vamos a devolver es la agrupación de todos los estados de todas las dependencias.

Y si una hay que falla, tendremos que evaluar, en la propia aplicación, si pasa a estar en un estado de *degraded*, o está en un estado de *unhealthy*.

Lo más normal es que en tu *endpoint /status*, el cual es abierto para cualquier aplicación con acceso únicamente devuelvas el estado de la aplicación.

Pero, también es tener un *endpoint* con acceso restringido para que nos devuelva la información sobre las diferentes dependencias, en un formato como el siguiente:

```
---
status: Unhealthy
entries:
  - Database:
      status: healthy
  - ProductService:
      status: healthy
  - QueueSystem:
```

```
    status: unhealthy
- LogSystem:
    status: healthy
```

Esta respuesta puede contener más información; como diferentes etiquetas, o la duración de hacer el propio *health check*, pero la idea es agrupar todas las dependencias en un solo punto y detectar cuál es la que falla. De esta forma, es más fácil investigar la causa de un problema.

Monitorización y alertas

Una vez salimos de la propia aplicación, debemos tener un sistema central que nos permita recolectar toda esa información. Pero no solo eso, debemos de ser capaces de incluir ciertas métricas de los sistemas donde las aplicaciones, o la infraestructura, están siendo utilizadas para asegurarnos de que el sistema funciona de la forma en la que estamos esperando.

Es cierto que, técnicamente, los *logs* y los *health check* son parte de la monitorización, pero va bien separarlos, porque son elementos importantísimos a nivel de aplicación que conviene saber bien, y que no quiero que se pierdan dentro de una explicación más grande dentro de la monitorización.

Idealmente, todos los sistemas deben tener monitorización y alertas, pero la dependencia que tenemos sobre la monitorización crece exponencialmente con cada servicio que añadimos al sistema.

Objetivos de la monitorización

El objetivo principal de la monitorización es asegurarnos de que todo funciona sin problemas y, en caso de que algo falle, detectarlo de una forma rápida o incluso antes de que suceda. Para ello, crearemos *dashboards* para poder ver la información en tiempo real y alertas para que, si algo se sale de la norma, nos avise.

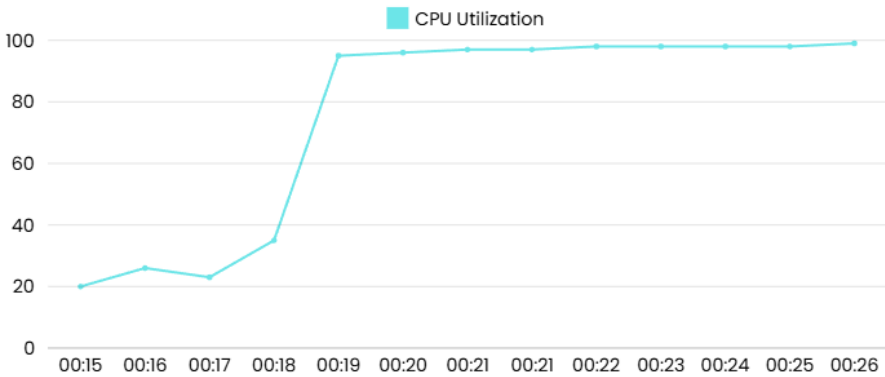
De esta manera, podemos identificar posibles problemas, o un estado del sistema degradado, antes de su fallo completo y que afecte a los usuarios.

Detección de problemas

La parte más importante a la hora de monitorizar aplicaciones es detectar cuándo un problema o fallo del sistema sucede, y más importante, detectarlo antes que tus clientes, o consumidores de la aplicación.

Un ejemplo muy normal es tener monitorización sobre el número de errores de una aplicación, o sobre la infraestructura que ejecuta dichas aplicaciones, ya sea la memoria RAM o la CPU.

Para este ejemplo, tenemos *dashboard* de uso de CPU, donde podemos observar que a las 00:19 ha pasado de tener sobre un 25% de uso de la CPU asignada a tener casi un 100%.



Si el uso de la CPU llega al 100%, nos arriesgamos a que las peticiones que llegan a esa máquina o instancia empiecen a fallar. Por ello, suele existir una alerta en la CPU para cuando pasa del 80% lanzamos una alerta a la que debemos responder, ya sea manual o automáticamente.

Disponibilidad

Cuando una aplicación colapsa y empieza a fallar es algo muy malo para el sistema y la reputación de la empresa. De hecho, la disponibilidad (combinación entre *uptime* y *downtime*) es tan importante, que la gran mayoría de servicios lo tienen anunciado en la propia página principal del servicio.

Si seguimos con el ejemplo de antes, lo ideal es tenerlo configurado para que, si tenemos un uso elevado de la CPU, como el 80% otra instancia se genere y así repartir la carga entre más instancias, lo que debería reducir la posibilidad de fallos.

El *uptime* es el tiempo que nuestra aplicación está siendo ejecutada sin problemas, y el *downtime* es el tiempo que la aplicación no funciona o no

responde como se espera. Ambas métricas están totalmente relacionadas, ya que una es el opuesto de la otra.

La gran mayoría de empresas suelen afirmar que sus aplicaciones tienen un *uptime* del 99.9% del tiempo, pero en términos reales, ¿qué significa eso?

Significa que, durante un año, la aplicación funciona un 99.9% del tiempo, o, lo que es lo mismo, no funciona durante 8 horas y 45 minutos al año.

Esta es una lista con la diferencia entre los diferentes valores de *uptime*:

- *Uptime* del **99%**: Máximo tiempo de *downtime* 87 horas y 36 minutos.
- *Uptime* del **99.9%**: Máximo tiempo de *downtime* de 8 horas y 45 minutos.
- *Uptime* del **99.99%**: Máximo tiempo de *downtime* de 52 minutos y 34 segundos.
- *Uptime* del **99.999%**: Máximo tiempo de *downtime* de 5 minutos y 15 segundos.
- *Uptime* del **99.9999%**: Máximo tiempo de *downtime* de 31 segundos.

Como puedes imaginar, cuantos más nueves en el decimal, más difícil y costoso es de mantener, y más cuidadosos tienen que ser quienes trabajen en esos sistemas.

Ojo, que este valor no lo puedes poner en tu web sin ser cierto, ya que normalmente irá acordado a través del *Service Level Agreement* (SLA), por lo que el tiempo máximo de *downtime*, es el máximo por contrato.

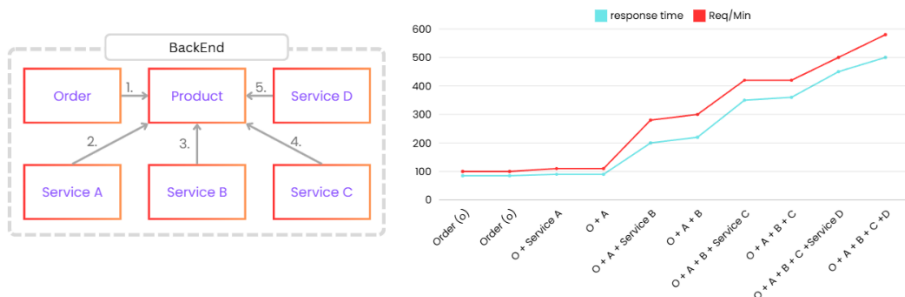
Rendimiento

Otro punto muy importante que se puede detectar de una forma sencilla con monitorización es el rendimiento. Técnicamente puede pasar lo mismo que hemos visto en la detección de problemas, que, de repente, nuestra API, que hace dos horas respondía en 200ms, ahora responda en 500ms, y salte una alerta a la que tenemos que reaccionar.

Otro escenario muy común, y que suele pasar desapercibido, es cuando los problemas de rendimiento son más graduales.

Un caso muy común es tener una API, la cual se utiliza mucho en nuestro sistema. Digamos que actualmente se utiliza por otros cinco servicios.

Al principio, cuando solo era utilizada por uno de los sistemas, y tardaba 80 milisegundos en el tiempo de respuesta, ahora que es utilizada por cinco servicios diferentes, tiene un tiempo de respuesta de 500 milisegundos. Por supuesto, el cambio ha sido gradual, conforme los diferentes servicios iban llamando a nuestra API, y como nunca ha fallado nada, no ha sido evaluada.



Este cambio tiene un efecto en cascada en todo el sistema, ya que la latencia es cuatro veces mayor.

La solución suele ser evaluar qué uso se hace de dicha API e intentar cambiar flujos de trabajo, ya sea internamente en la propia API, o, incluso, en quien consume de la API. La implementación suele ser un trabajo costoso, porque no es lo mismo crear una aplicación para responder a 100 llamadas por minuto, que a 60 mil.

¿Qué monitorizar?

El punto clave dentro de la monitorización es saber qué debemos monitorizar, ya que poca monitorización hace que monitorizar no sirva para nada, mientras que un exceso de monitorización hace que sea ignorado. Así que hay que encontrar el balance perfecto, lo que es mucho más difícil de lo que parece.

La parte crucial es monitorizar métricas útiles y que vayan a ser evaluadas de alguna manera, si una métrica no va a ser evaluada, no la crees, porque lo único que causa es problemas y dependiendo del tipo de aplicación estas pueden cambiar.

Por ejemplo, si tenemos una app de móvil, quizá queremos monitorizar la respuesta del usuario, que la app vaya fluida y que no rompa, con eso en mente monitorizamos la versión del dispositivo, etc.

Pero, si nos centramos en la monitorización en los sistemas distribuidos, podemos centrarnos en las siguientes métricas:

Métricas de infraestructura

Aquí incluimos todas las métricas de la instancia o máquina física que está ejecutando una aplicación, algunas son:

- Uso de la CPU
- Uso de la memoria
- Uso del disco
- Tráfico de red

Métricas de aplicaciones

Todas las métricas relacionadas a la ejecución de una aplicación o servicio.

- Latencia: evalúa el tiempo de respuesta de un servicio.
- *Throughput*: El número de *request* por minuto de un servicio, y a qué *endpoint* en concreto van.
- Ratio de errores: monitorizamos los códigos de estado HTTP 4xx/5xx. Normalmente, a mí me gusta no registrar los 4xx, pero, en ciertos casos, sí es bueno guardarlos.

- *Health check*: los propios sistemas de monitorización te permiten configurarlos para ejecutar *health check* cada X segundos y, así, generar la alerta ellos mismos si algo falla.

Estas son las básicas, pero puede ser que ciertas aplicaciones requieren métricas extra.

Métricas de sistemas distribuidos

Quiero separar en un grupo servicios adicionales, como pueden ser las colas, donde tenemos consumidores, los cuales puede ser que no consuman tan rápido como llegan los mensajes, lo que significa que estos se van apilando y el sistema funciona más lento, esta métrica también es importante.

O, en el caso de tener múltiples bases de datos donde una actúa como máster y el resto como réplicas, una monitorización del tiempo que los datos tardan en sincronizarse también es importante, para asegurarnos que los datos se sincronizan de forma casi instantánea.

Cada servicio, o cada sistema, que utilicemos va a tener diferentes métricas que podemos y debemos observar.

Observabilidad

Si la monitorización es registrar cuando un evento sucede en el sistema, la observabilidad es la reacción a dicho evento y saber cómo y por qué dicho evento ha sucedido.

Para ello, veremos que la observabilidad está fuertemente ligada a la monitorización, y, comúnmente, se explican o se exponen en el mismo apartado, ya que, de los tres pilares fundamentales de la observabilidad, dos de ellos se comparten con la monitorización.

Pilares fundamentales de la Observabilidad

Estos tres pilares son los siguientes:

Logs

Los *logs* registran los eventos que suceden en el sistema, con un contexto detallado. Una vez comprobamos dicho evento, estamos observando para un posible diagnóstico.

Métricas

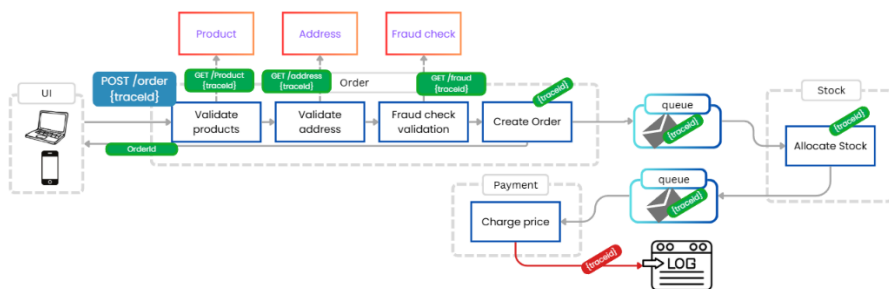
Son estadísticas puras del sistema. Como hemos mencionado antes, el número de *request* por minuto a un servicio, o el tiempo de respuesta.

Traces

Es el contexto de por qué las cosas suceden. El curso de una acción a lo largo de nuestro sistema de principio a fin.

Pongamos como ejemplo el caso de la creación del pedido utilizando el patrón SAGA.

Cuando el usuario hace la llamada POST para crear el pedido, internamente creamos un *TraceId*, el cual es un identificador único de esa llamada. Este identificador va a pasar por cada uno de los elementos necesarios para crear dicho pedido. Ya sea una llamada a otro servicio, o un evento, debemos propagar dicho identificador, para así poder relacionar todas las acciones entre sí.

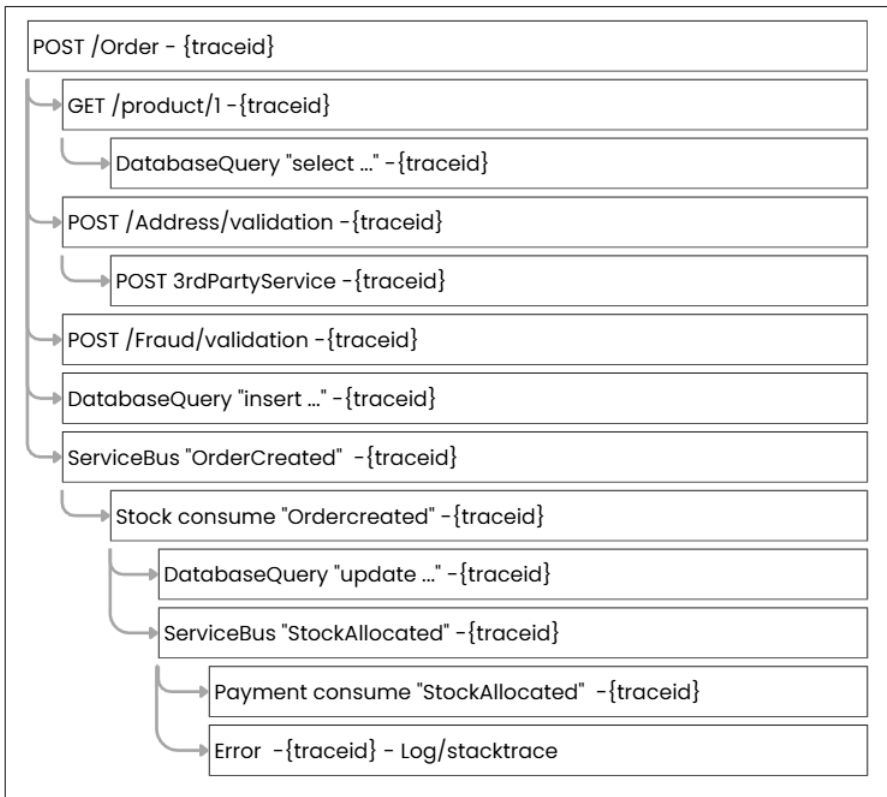


Ya que, si tenemos un error en cierto punto, seremos capaces de leer dicho identificador y enlazarlo por todos los *logs* del sistema para encontrar cual es el causante de dicho error, aunque el error esté en un servicio que sea una dependencia de otra dependencia.

Dependiendo de qué sistema de *traces* tengas, verás un diagrama como el de la imagen, donde tienes los elementos a los que estás invocando

desde los diferentes servicios, e, incluso, es posible que tengan los *logs* integrados.

Otra opción es ver una lista, o tabla, con los eventos que suceden en cascada, similar a lo siguiente:



Ambas opciones son exactamente igual de válidas, aunque, como te puedes imaginar, una es mucho más barata y sencilla de mantener que la otra.

Asegurar el correcto funcionamiento del sistema

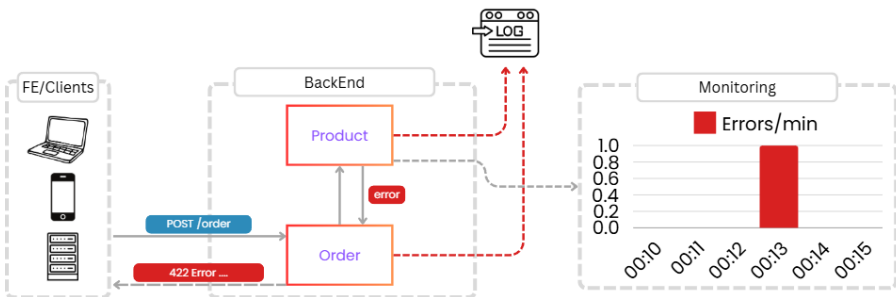
Todas las características que hemos visto hasta ahora hacen que nuestro sistema sea más fiable y robusto; ya bien sea a través de acciones automáticas, o manuales.

Las automáticas están claras, cuando tenemos monitorización sobre los recursos del sistema y estos pasan cierto nivel, por ejemplo, el 80%, lanzamos una nueva instancia y así la carga se reparte entre más máquinas.

No quiero terminar esta sección sin mencionar un posible flujo dentro de la intervención manual, ya que combina todo lo que hemos visto.

Pongámonos en el supuesto de que tenemos una app, la cual genera un error y ese error se registra en nuestro sistema de *logs*.

A su vez, el sistema de monitorización detecta que hemos tenido un error, ya que la monitorización de las aplicaciones comprueba su correcto funcionamiento y registra un error en el sistema.



Este error genera una alerta, la cual es escuchada por el equipo correspondiente, en este caso, el equipo de productos. Hasta aquí, todo ha sido monitorización.

Desde este punto, pasamos a la parte de observabilidad.

Ahora el equipo debe ir a los *logs* y leer el *log* para ver si pueden detectar el problema.

Para este escenario, simularemos que el pedido se ha intentado crear con un producto que no existe. Por lo tanto, al ir a la base de datos y no encontrarlo, la API ha respondido con un 404 (*Not Found*). Pero como es una situación extraña, el equipo tenía un *log* para ello.

Si no tuviéramos más observabilidad en el sistema, ahí se acabaría todo el análisis y el problema se ignoraría, pero como somos gente responsable con cada *request*, tenemos el identificador de la *request*, el cual se genera en el que realiza la llamada y se va propagando entre los diferentes servicios. Ello hace que, en el propio sistema de *logs*, podamos buscar por dicho identificador. Y podemos ver como la llamada ha venido desde el servicio de productos y, a su vez, qué cliente ha realizado dicha llamada.

A partir de aquí, dependiendo de la gravedad del problema y la cantidad de errores que tengamos, podemos decidir cuál puede ser la acción que tomar. Si es un error puntual por un cliente de integración, puede ser simplemente un error o *bug* en sus sistemas, así que, si el error no sucede más, se podría ignorar.

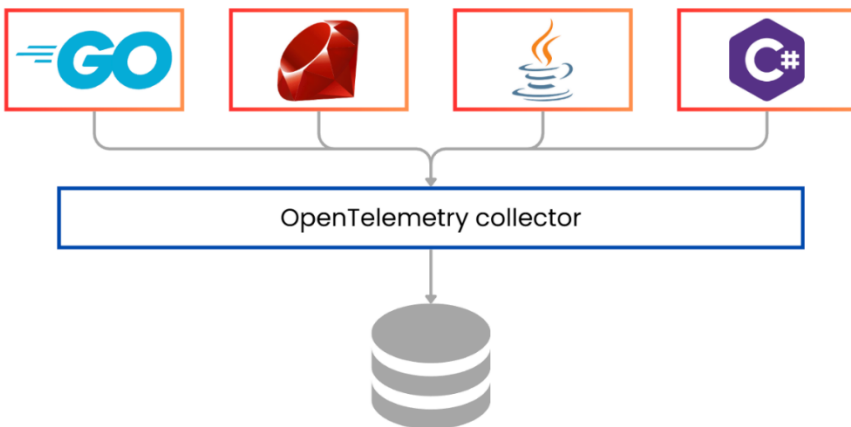
Si es un error que viene desde la interfaz, tendríamos que ver si es un fallo nuestro del sistema o un usuario intentando hacer cosas que no debería y actuar en consideración.

OpenTelemetry

Finalmente, hay que mencionar el estándar que se está empezando a aplicar, y, pese a que ya tiene varios años, cada vez las empresas y los diferentes lenguajes se van adaptando más a la hora de implementar la misma solución.

Antes de la llegada de *OpenTelemetry*, cada lenguaje generaba las métricas de una manera, después, el servicio que las consumía requería que los datos fueran en un formato u otro. Como puedes imaginar, eso requería que si tenías varios lenguajes de programación en tu sistema tuvieras que reescribir lo mismo varias veces, y, además, era imposible migrar de un proveedor a otro porque requería sobre escribir todas las aplicaciones.

Aquí es donde entra *OpenTelemetry*, ya que estandariza la forma en la que las aplicaciones envían las métricas, y así, da igual que proveedor o servicio utilices, será capaz de leerlas y entenderlas.



Como puedes ver en la imagen, disponemos de un elemento llamado *OpenTelemetry Collector*, que es *básicamente* un servicio que recoge esos datos para procesarlos y mostrarlos, ya que OpenTelemetry no es una aplicación, sino un estándar que provee APIs y SDKs en los diferentes lenguajes.

El *OpenTelemetry Collector*, en cambio, sí que es una aplicación, la cual vamos a desplegar juntos a nuestras aplicaciones para que dicha aplicación envíe los datos de las métricas al servicio que utilices como base de datos.

Tenemos dos formas de desplegar el *OpenTelemetry Collector*:

- **Agente:** Cuando despliegas la aplicación debes correr otro servicio junto a ella en el mismo *host*. Esta acción se hace normalmente con el patrón *sidecar* o un *Daemon set*.
- **Puerta de entrada:** El servicio se despliega como un servicio más dentro de la arquitectura del sistema.

En mi experiencia, la opción más común es como agente, ya que cada instancia tiene dicho agente configurado en ella por lo que no hay una única instancia que pueda fallar.

Una vez la información está en el *OpenTelemetry Collector* esta es procesada y enviada al proveedor de servicios que puedas utilizar (NewRelic, Dynatrace, Appdynamics, etc), las propias de Azure, AWS o GCP, o si vas por *Open Source*, la opción más común es *Prometheus* como base de datos, y *Grafana* como interfaz web.

Diseño de aplicaciones robustas

A la hora de diseñar aplicaciones y servicios estos van a ser consumidos por otros servicios diferentes, lo que significa que hay que tener ciertos aspectos en consideración. Desde la autenticación y seguridad a utilizar hasta una normalización de las aplicaciones.

Siempre se dice que lo ideal es dar a cada equipo total libertad para hacer lo que mejor vean en sus aplicaciones, y si bien esto es cierto, hay que poner límites a tal libertad, pues no quieres que cada parte del sistema funcione de una forma diferente y que dependa de las personas que lo escribieron.

Estos estándares van desde la forma en la que se accede a la base de datos, qué base de datos utilizamos, qué sistema de *logs*, o, incluso, las abstracciones para acceder a los secretos, *Event Bus* o sistema de ficheros.

Estos accesos suelen estar abstraídos en librerías comunes, y los equipos por norma general no suelen reescribir las librerías, porque estas ya funcionan y reescribirla no proporciona ningún valor adicional.

En este capítulo voy a hablar del diseño de aplicaciones, pero no voy a entrar en si una aplicación debe implementar arquitectura hexagonal, clean architecture, Core-driven architecture, o la implementación que

quieras hacer; donde me voy a centrar es en los elementos que pueden y afectarán al resto de servicios que dependen de dicha aplicación.

Unificación de respuestas de una API

Uno de los puntos críticos, que muchas empresas parecen ignorar, son las respuestas de las diferentes APIs del sistema.

Nos pasamos la vida hablando de patrones y documentación, pero, a la vez, no nos paramos a pensar por el punto de entrada o del punto de salida de una aplicación, siendo este muy importante y crucial para la comunicación síncrona.

Veremos que, en situaciones donde una API devuelve un valor u objeto cuando responde el valor correcto, a su vez responde un Sting cuando hay un error, y si es un error del sistema que no tenemos controlado el error es completamente diferente.

Request **POST /order**

Responses

Status code 201

```
{  
  Orderid: 1234  
}
```

Status code 404

```
"product AA0 does not  
exist"
```

Status code 500

```
{  
  type: example/type1  
  title: "internal error"  
  status: 500  
  instance: 00-9388nCmaid139  
}
```

En este escenario tenemos tres posibles respuestas a una única *request*. Y lo que suele suceder en el mundo real es que todo lo que no sea un código de error de éxito se considera como uno de error. Cosa que es cierta, pero un código de estado 404 no es un error, es una validación. Hay situaciones donde únicamente se responde el código de error sin responder un mensaje. En resumen, cada uno hace lo que quiere.

Si me hubieras preguntado hace unos años sobre qué debería de responder una API, te hubiera contestado que lo ideal es responder un objeto que engloba el objeto que quieres responder en sí, junto con la información del error. Esto es la definición de un patrón muy común llamado patrón *result* que se implementa con `Result<T>`. Con ese cambio, las respuestas serían las siguientes.

Estándar *Problem Details* para APIs

La parte donde hemos mostrado el error con el código 500 es un objeto diferente, pero lo es por un motivo, ese objeto representa la idea del estándar RFC-9457 (<https://datatracker.ietf.org/doc/html/rfc9457>) publicado en julio de 2023 y su objetivo final es normalizar de una vez y por todas cómo creamos respuestas de una Web API, en concreto cuando tenemos errores.

Este estándar no se centra en cuándo el funcionamiento de una API es correcto, sino que se centra en la parte de tener problemas. ¿Qué sucede si la validación es errónea? ¿O qué sucede si tenemos un error inesperado? ¿Cómo devolvemos esa información?

La idea del estándar *Problem Details* no es más que un JSON que contiene una serie de campos específicos y normalizados para que todas

las API, en todos los lenguajes, los utilicen de igual manera, así cualquier consumidor pueda entender la respuesta de la API sin ningún problema.

Los miembros de este JSON son los siguientes:

Request POST /order

Responses

Status code 201

```
{
  value:
  {
    Orderid: 1234
  },
  errors: [],
  success: true
}
```

Status code 404

```
{
  value: null,
  errors: [{
    errorCode: NO_PT,
    message: "Product
AA0 does not exist"
  }],
  success: false
}
```

Status code 500

```
{
  type: example/type1
  title: "internal error"
  status: 500
  instance: 00-9388nCmaid139
}
```

De esta forma estamos siempre devolviendo un objeto, que contendrá el valor, si la operación ha sido exitosa. Así como una *flag* en el propio objeto que indica que es exitosa, y un objeto con los errores, que en este caso está vacío.

En caso de que sea una llamada fallida por un elemento que podemos controlar, lo que hacemos es devolver el mismo objeto, esta vez el valor es nulo, y la *flag* de exitosa está en falso. Pero la gran diferencia es en el objeto de los errores, el cual contiene los errores en un objeto que contiene el código interno de error y el propio mensaje de error.

Desafortunadamente, puedes ver el problema con el 500 lo seguimos teniendo, ya que este tipo de errores son generados por los diferentes *frameworks* y debemos hacer un *try catch global* para detectarlos, cosa que muchas veces no se hace.

El motivo por el que el error con el código de estado 500 es diferente, lo vamos a ver a continuación.

Type (Tipo)

El miembro tipo es un *string* que, idealmente, contiene una URL la cual redirige a la explicación del error en la documentación. Por ejemplo:

```
Https://docs.mycompany.com/ref/NO_PT
```

En la comunicación B2B (empresa a empresa) es muy común devolver únicamente el código en sí, ya que la URL no siempre es pública.

Title (Título)

Este campo es únicamente para consumo humano, ya que contiene el error o problema en texto plano.

```
“Product AA0 does not exist.”
```

Detail (Detalles)

El campo detalles es una ampliación al campo anterior, y se centra en dar un error más concreto.

```
“The Product AA0 is not activated.”
```

Dependiendo del caso de uso, este campo puede estar vacío porque puede que no quieras dar tanta información a un cliente.

Instance (instancia)

Representa un identificador único para la resolución del problema. En algunos casos será un UUID, otras veces será un ID numérico. La idea es que sea único y nos permita hacer un seguimiento de quien ha realizado la *request*.

`Https://api.mycompany.com/error/12344`

En muchas ocasiones, el Identificador único será el *TraceId* de la *request*, así, cuando el cliente registra el error en el sistema, podemos ir a los *logs* y ver si tenemos algo.

Status (Código de estado)

Representa el código de estado de HTTP, y normalmente será un código de error acorde. Pero, durante mi experiencia, he visto empresas que devuelven códigos de estado correcto incluso cuando devuelven errores. No lo hagáis, pero cualquier cosa es posible.

Extensions (Extensiones)

Las extensiones son miembros que pueden contener detalles adicionales sobre el error. Internamente, es tratado como un diccionario clave valor, lo que quiere decir que las *Keys* serán únicas, pero puedes poner tantas como quieras.

Para poner un ejemplo, podemos simular la creación de un usuario, donde pedimos que se introduzca un email y un teléfono. Si esos valores no son válidos la API nos devolverá una respuesta como la siguiente:

```
{
  "type": "https://website.net/code-error-1",
  "title": "Error en la validación de los datos",
  "status": 400,
  "detail": "Los parámetros no son correctos",
  "Errors": {
    "Email": "El email no es válido",
    "Phone": "El número de teléfono no es válido"
  }
}
```

En nuestro caso, el campo *Errors* es el campo extensible, pero podemos tener tantos como queramos, y pese a que el estándar trata esos campos, técnicamente podemos utilizarlos o no, y son opcionales para ampliar los detalles del error.

Si volvemos al escenario anterior, la representación de las posibles respuestas de una *request* es la siguiente:

Request POST /order		
Responses		
Status code 201 <pre>{ Orderid: 1234 }</pre>	Status code 404 <pre>{ type: docs.mycmpay.com/ref/NO_PT title: "Product AA0 does not exist", details: "The product AA0 is not activated" status: 404 instance: 00-9388nCmaid139 }</pre>	Status code 500 <pre>{ type: example/type1 title: "internal error" status: 500 instance: 00-9388nCmaid139 }</pre>
Content type: application/json	Content type: application/problem+json	Content type: application/problem+json

Además, podemos ver como el tipo de la respuesta (*content type*) de la respuesta va a ser un tipo diferente, en caso de que la respuesta sea satisfactoria o no, lo cual es importante a la hora de generar consumidores de dicha API, al especificar de forma explícita cuándo tenemos un problema.

En el mundo laboral estoy viendo últimamente que se está empezando a aplicar este estándar y, personalmente, espero que cada vez más empresas lo implementen. Así es muy fácil saber si tenemos un error o no, y a qué tipo de objeto realizar el mapeo desde la respuesta del servicio.

Breaking changes

El punto de entrada o contrato que tu aplicación, librería o sistema tiene con el exterior es sagrado, y realizar un cambio en dicho punto de entrada lo consideramos un *breaking change*. Ya que el contrato que teníamos apalabrado entre las diferentes entidades deja de ser válido.

Técnicamente, un *breaking change* puede ser también un cambio en la funcionalidad de un proceso, pero la realidad es que este tipo de cambios

no suelen ser tan llamativos, o afectar tanto a los usuarios como un cambio en el contrato.

Este cambio en el contrato afecta a todos los clientes, ya sean internos o externos, y el motivo por el que se le llama *breaking change* es porque rompe no solo el contrato, porque si no se toca el código consumidor, romperá el código también.

Un *breaking change* puede ser un cambio tan simple como cambiar el nombre de una propiedad porque tenía un typo, o directamente eliminar cierto servicio o recurso.

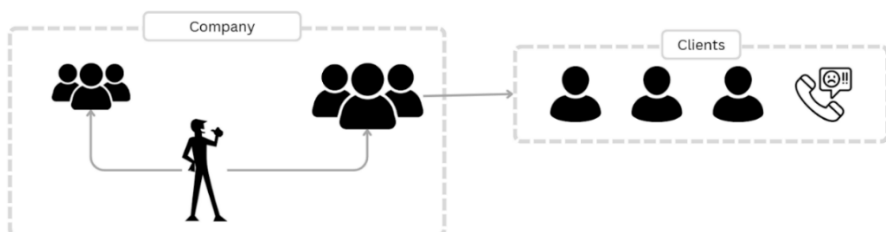
Impacto de un *breaking change*

Afectar el funcionamiento actual de una aplicación no es una acción menor; de hecho, es algo bastante serio y lo recomendable es siempre evitar tener dichos cambios, o, por lo menos, desarrollarlos de una forma en la que los clientes no se vean afectados. Ya que, si tú realizas un cambio en un contrato, todos los consumidores necesitan acomodar dicho cambio, lo que no es fácil.

Puedes pensar que, técnicamente, es fácil; es un cambio en un par de líneas de código y ya está, y esa es la realidad. El problema es la burocracia que va junto a dicho cambio.

Pongámonos en situación de que hay que hacer un cambio que genera un *breaking change* sí o sí, y no puedes hacer nada para remediarlo. Una vez el equipo de desarrollo ha decidido dicho cambio, hay que comunicarlo a

la parte de la empresa que se comunica con los clientes externos y, además, debemos comunicarlo internamente.



Lo normal es que internamente no cause ningún problema, quizá te toque asistir a algún equipo para hacer dicho cambio, pero eso es más o menos normal. El problema puede venir del lado de los clientes, ya que, técnicamente, ellos han firmado un contrato y en dicho contrato pueden venir cláusulas de funcionamiento de servicio, donde se especifica que no puede haber *breaking changes*.

Y parece un sinsentido, pero no lo es, ya que el cliente ve un *breaking change* de la siguiente forma:

1. **Interrupción del servicio:** Algo que ha pagado para que funcione, ya no lo va a hacer.
2. **Coste económico y laboral:** El cliente debe hacer cambios en el código, lo que implica que va a tener que poner a un desarrollador (mínimo) a hacer el cambio, después hay que testarlo todo, desplegar, etc. Un cambio que, a priori, parece que son “10 minutos”, se convierte en una semana de trabajo que puede trastocar los planes de la empresa.
3. **Pérdida de confianza:** Este tipo de cambios van a hacer que los clientes te vean como alguien menos confiable.

Si tus clientes son gratuitos, tienes algo más de libertad, ya que, al no pagar, lo más probable es que no tengan contrato, y se tengan que regir

por tus términos y condiciones del servicio, así que hay más libertad, pero el impacto reputacional está ahí.

Aun así, los clientes, ya sean externos, internos, de pago o gratuitos esperan una serie de condiciones cuando lo comunicas.

1. **Guía de migración:** a la hora de comunicar el cambio debes tener listo el reemplazo de dicho cambio, si no, no lo comuniques, porque algo que tienes pensado que puede tardar unos días o semanas puede tardar mucho más tiempo, y no quieres que la reputación empeore más.
2. **Periodo de transición:** Durante un tiempo, lo ideal es que ambas versiones estén disponibles de forma simultánea, si es posible. De esta forma facilitamos la migración durante un periodo de tiempo en vez de tener que sincronizar a todos los clientes el mismo día.
3. **Ayuda y soporte:** Si un cliente tiene dudas, ayudarle a realizar la migración.

La idea detrás de estas ayudas es evitar que los clientes tengan que realizar el cambio de forma instantánea y sin avisar, ya que, para muchos de ellos, puede ser un cambio con baja prioridad y que tarde en completarse seis meses, o incluso un año.

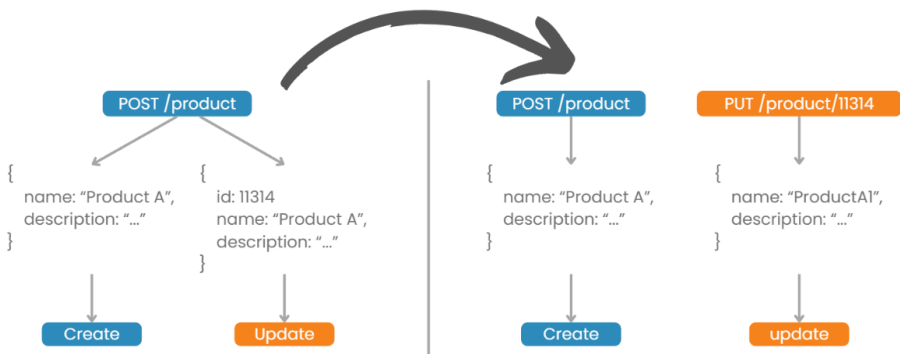
Mitigar el impacto de los *breaking changes*

Lo ideal con los *breaking changes* es no tenerlos. Si tu API realiza una acción, seguir teniendo soporte para dicha acción hasta el fin de los días es lo ideal. Incluso cuando no queremos que se utilice, ya que si seguimos dando soporte no tenemos que notificar a los clientes.

Un ejemplo muy común, que he visto con anterioridad, es tener un *endpoint* en una API para crear o actualizar.

Pongámonos en un supuesto donde tenemos un *endpoint* en nuestra API `/POST`, que recibe un producto. Si el producto contiene un ID, se actualiza, y si no recibe el ID del producto, se crea uno nuevo.

Con la idea de evitar que un único *endpoint* realice ambas acciones, creas uno nuevo donde, para editar, hay que mandar el ID del producto en la URL.



Este cambio, pese a que es ideal, puede no ser un motivo suficiente para generar un *breaking change*, ya que es un cambio muy grande en la parte del cliente, cuando son tres líneas de código soportar ambos procesos en el código.

Podemos seguir manteniendo el *endpoint* que haga ambas acciones, pero crear el *endpoint* PUT para actualizar. En este caso, notificar a los clientes no justifica el coste. Y una semi-solución es no indicar en la documentación que el post original actualiza los productos si ya existen.

Lo mismo implica si vamos a aumentar el número de parámetros en una respuesta. Eso no es un *breaking change*, ya que aumentar funcionalidades no rompe nada.

Un *breaking change* es una funcionalidad donde cambiamos el funcionamiento. Por ejemplo, a la hora de crear, o incluso de leer, un producto cuando introducimos el precio, lo hacemos siempre en euros (€) pero queremos hacer un cambio para que sea en la moneda que la tienda que está utilizando nuestro sistema tiene como moneda predeterminada. Por lo tanto, en cada país va a ser una moneda diferente.

Nota: Para simplificar el escenario ponemos un precio en una moneda por producto.

Luego, cuando un usuario de un país que no usa dicha moneda quiere ver dicho precio en su moneda local el cambio va a ser algo similar a lo siguiente:

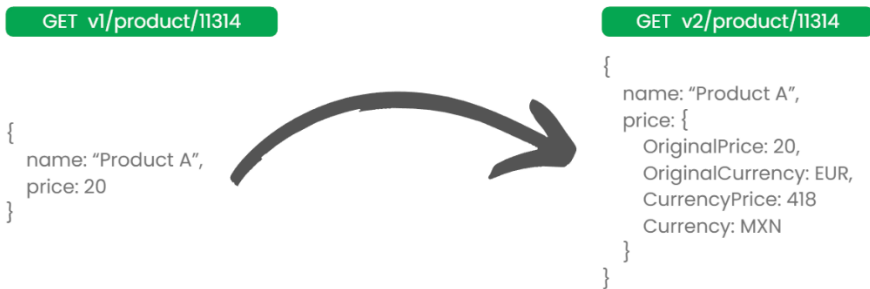


Ya no estamos únicamente ampliando la información, sino que la estamos cambiando, y esto es un *breaking change*.

Para soportar este *breaking change* lo ideal es utilizar versiones. Estas versiones van a ser versiones de la API. Estarás acostumbrado a ver el versionado con el versionado semántico “Major.Minor.Patch”

- **Major:** Se incrementa cuando introducimos *breaking changes*.
- **Minor:** Se incrementa cuando añadimos nuevas funcionalidades que son compatibles con las versiones anteriores.
- **Patch:** Se incrementa al solucionar *bugs*.

En las API, o sistemas, seguimos la misma lógica, pero únicamente aumentamos la versión *major*. Por lo tanto, cuando un usuario accede a nuestra API, lo hace indicando la versión en la URL y, dependiendo a qué versión acceda, devolveremos un valor u otro.



Lo ideal sería marcar tu V1 como deprecada y notificar a todos los clientes que dejen de utilizarla, anunciando cuándo será la fecha límite donde dejaras de permitir el acceso a V1.

Con eso en mente, y con un sistema con monitorización y observabilidad, podemos monitorizar el uso que se hace de la V1 e ir notificando a los

clientes conforme se acerca la fecha límite, con la idea de que no les pille el toro y se queden con un sistema que no funcione correctamente.

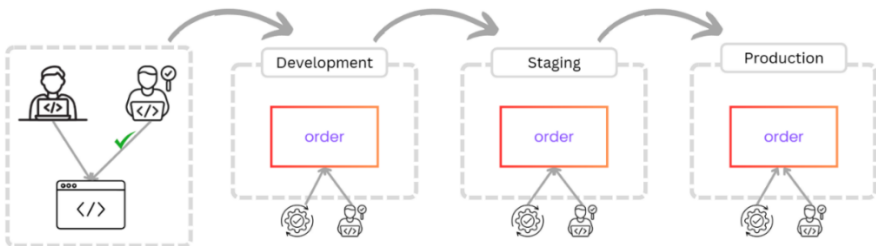
Despliegues a producción

A la hora de desplegar aplicaciones, tenemos diferentes opciones sobre cómo ejecutar el proceso. Algo importante de saber es que todas implican parte de infraestructura.

Antes de ir a los propios despliegues como tal, debemos mencionar el proceso para llegar al despliegue de producción.

Desde mi punto de vista, en un mundo ideal, cada *commit* en el código (en el *branch principal*) debe poder llegar a producción sin causar ningún desastre. Para ello, necesitas tener un proceso de desarrollo, despliegue y testeo.

Todo empieza por el desarrollo. El desarrollador hace su cambio, lo testea en su máquina y hace la *Pull Request*. Los compañeros lo aprueban y el código se despliega en el entorno de desarrollo.



Por ahora, entiendo que es un proceso familiar para todo el mundo. Dependiendo del tamaño, tipo y recursos de la empresa, tendremos más o menos entornos entre desarrollo y producción, pero la idea es la misma en todos ellos.

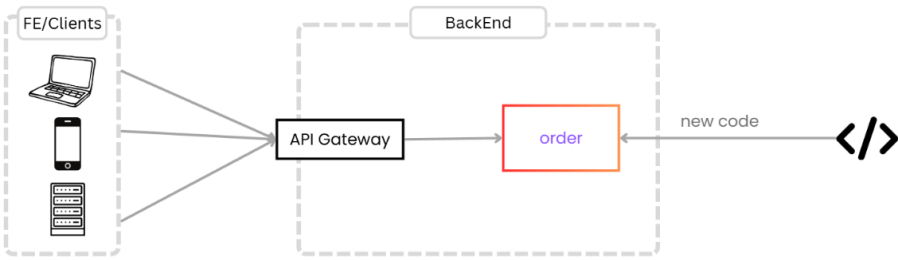
Una vez el código ha sido desplegado en cada uno de los entornos debemos correr los tests. Idealmente serán tests automáticos, que se ejecutan automáticamente una vez se despliega. Pero en ciertos casos serán test manuales, los cuales se ejecutan por el equipo de QA (*Quality Assurance*).

Si la empresa no es muy grande (QA no existe), deberá ser un compañero quien verifique los cambios.

Este proceso se debe hacer en todos los entornos y puede parecer un proceso pesado, pero así es como se elimina la posibilidad de tener *bugs* que puedan afectar a clientes.

Despliegue a ciegas - *Big bang*

Por norma general, desplegar servicios no es tan simple como simplemente reemplazar los ficheros y reiniciar el servicio. Si fuera el caso de que tu sistema es capaz de funcionar de esa forma, lo más probable es que no necesites microservicios.



En este caso, el software va por los diferentes entornos donde, eventualmente, se verifica antes del despliegue a producción con manuales. Lo mismo sucede una vez desplegamos en producción. Quizá, en algunos casos, se confirme algo más, similar a que ciertos procesos clave de la web funcionan, pero una vez eso está hecho, se da el visto bueno.

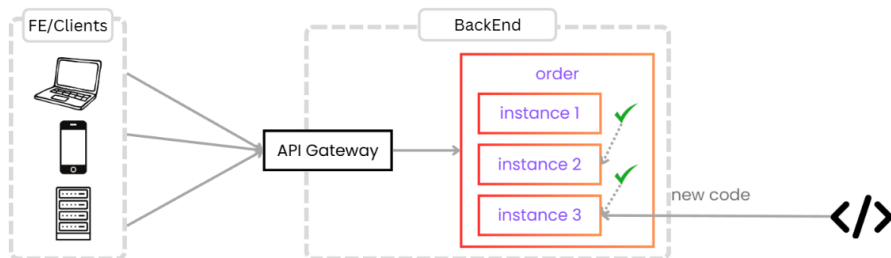
Este proceso es común en empresas pequeñas porque se entiende que no hay un presupuesto para mantener una monitorización constante y el impacto de que un proceso no funcione por un tiempo, o incluso horas, hasta que un cliente te lo notifica no es crucial.

Piensa, por ejemplo, en el caso de uso de una imprenta de un pueblo. Una web la puede ayudar a hacer negocios, pero nunca va a ser su fuente principal de ingresos, por lo tanto, no merece la pena una inversión enorme en infraestructura y capital humano para administrar.

Despliegue gradual - *rolling deployment*

Cuando desplegamos de forma gradual, lo que hacemos es reemplazar las instancias de una aplicación por las nuevas.

Por ejemplo, si una aplicación tiene tres instancias, primero reemplazamos una, verificamos que funciona con el *health check* y de ahí pasamos a la siguiente instancia. Así hasta que terminemos.



El reemplazo de una instancia no comienza hasta que la instancia anterior ha sido verificada, y, en caso de que suceda un error y no se pueda verificar durante el despliegue, se cancela dicho despliegue y se vuelven a poner las instancias con la versión anterior.

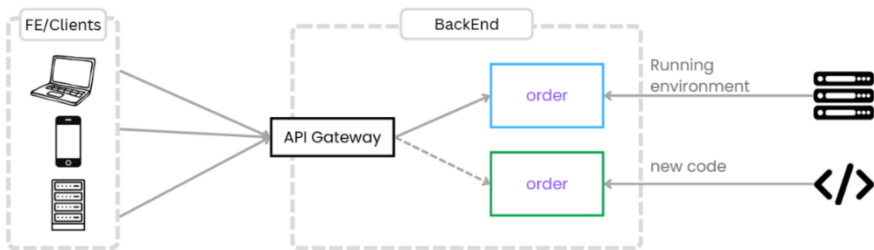
Algo que hay tener en cuenta es que esta validación que hace el *Load Balancer* la hace solo sobre el *health check*, por lo que tenemos que hacer un testeo y comprobar de forma manual y automática que todo funciona como era esperado.

También debemos tener monitorización para ver que la aplicación no falla o empeora. Si lo hace, el proceso de *rollback* es igual de largo que el de testear, porque es lo mismo, solo que desplegando la versión anterior de la aplicación.

Despliegue Azul/verde - *Blue/Green deployment*.

La estrategia *Bue/Green* consiste en crear dos entornos completamente iguales que van a ser ejecutados en paralelo.

El entorno original estará en la parte azul, y es el entorno o sistema donde nuestra aplicación está corriendo de cara a nuestros clientes. La parte verde es el nuevo entorno, con los cambios que queremos implementar en el servicio.



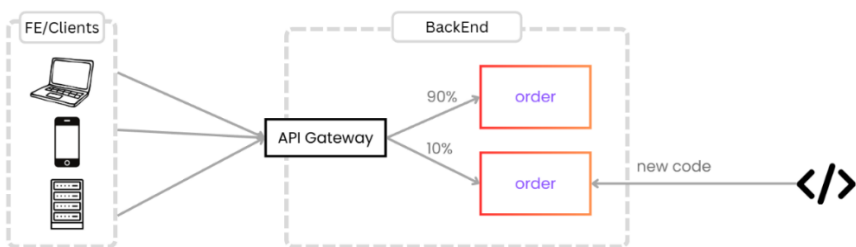
Una vez tenemos ambos sistemas corriendo en paralelo podemos ejecutar diferentes test contra el nuevo entorno, mientras los clientes siguen llamando al entorno antiguo. Una vez verificamos que todo funciona, apuntamos la *API Gateway*, el *Load Balancer*, o incluso el DNS hacia nuestro nuevo entorno.

Si la monitorización durante unos minutos nos hace indicar que todo funciona bien, podemos eliminar el entorno antiguo, pero si algo falla podemos hacer *rollback* de una forma casi inmediata, porque el código antiguo sigue existiendo y sigue siendo ejecutado en el entorno anterior, por lo que únicamente necesitamos deshacer el cambio del DNS o la puerta de acceso hacia los entornos.

Despliegue canario - *Canary deployment*

Podríamos entender que el *canary deployment* es un tipo de *Blue/Green* ya que se basan en la misma idea. Tener dos entornos corriendo de forma simultánea. La gran diferencia en este caso es que sí vamos a tener tráfico externo en el nuevo sistema.

Este tráfico se migra de forma gradual o incremental, normalmente se empieza con algo pequeño y, poco a poco, se va subiendo más, siempre y cuando la nueva versión no reciba ningún error.



Como puedes imaginar, este tipo de despliegues solo se realizan en empresas de un tamaño considerable, ya que el coste de montar y mantener esta infraestructura es muy elevado.

Es más, se puede llevar el despliegue canario más allá cuando la empresa es una multinacional con clientes en múltiples lugares del mundo.

Si tienes clientes en LATAM, España y Filipinas, lo más normal es que tengas múltiples entornos replicados, lo cual implica que tus despliegues no van a ser globales de todos a una. Primero desplegarán en una región y luego en otra.

Por ejemplo, en nuestro caso, si tenemos un 10% de los usuarios en Filipinas, un 40% en España y un 50% en LATAM, desplegamos los cambios primero en Filipinas. Al ver que todo funciona, desplegamos en España y, finalmente, en LATAM.

El objetivo es hacer un despliegue gradual para que, en caso de algo ir mal, no afecte a todos tus usuarios.

Finalmente, hay que mencionar que es muy común ver una combinación entre *canary deployment* y *rolling deployment*, ya que no hay ningún motivo por el que no podamos implementar ambos escenarios en el mismo proceso.

Feature flags

Llegan momentos durante el ciclo de vida de desarrollo del software donde debemos migrar funcionalidades de un servicio a otro. Quizá estemos cambiando una dependencia, o, incluso, modificando nuestro código para extraerlo en una dependencia.

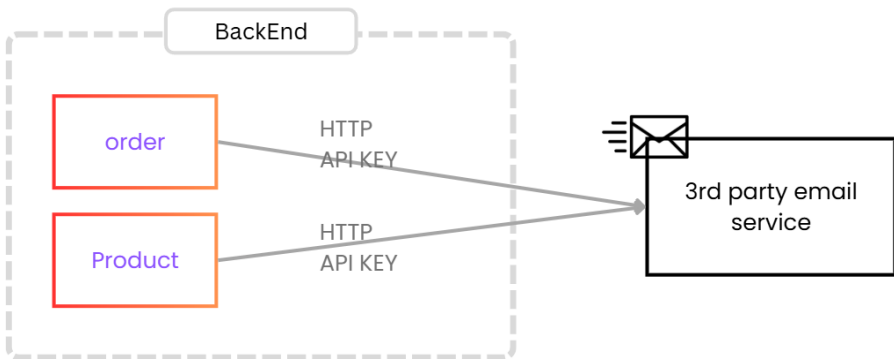
Este tipo de cambios suelen llevar bastante tiempo de programar, lo que implica que el código que estás escribiendo va a funcionar a la vez que el código existente que está funcionando en la máquina.

Aquí es donde entran las *feature flags* (también llamadas *feature toggle*), una técnica que nos permite habilitar o deshabilitar una funcionalidad en tiempo de ejecución sin volver a tener que desplegar el código.

Obviamente, para que funcione bien, debemos tener ambas versiones del código existiendo y corriendo.

Patrón *Strangler Fig*

Pongamos el ejemplo sencillo de cuando tenemos una *feature flag*. Es posible que tu aplicación tenga un servicio para enviar emails, o, incluso, que cada uno de los servicios del sistema realizan una llamada http a un servicio de email de terceros el cual envía los emails.



Esta llamada se hace por HTTP, con la *Key* correspondiente y sin mucha complicación.

Sin mucha complicación siempre que no haya que cambiar nada.

Llega un día en el que te das cuenta de que la API Key se ha comprometido y tienes que cambiarla, y aquí llega el problema. Depende de cómo tengas el sistema de contraseñas, puede ser tan fácil como cambiar la API Key del sistema de email una única vez y con tu abstracción funcionara bien.

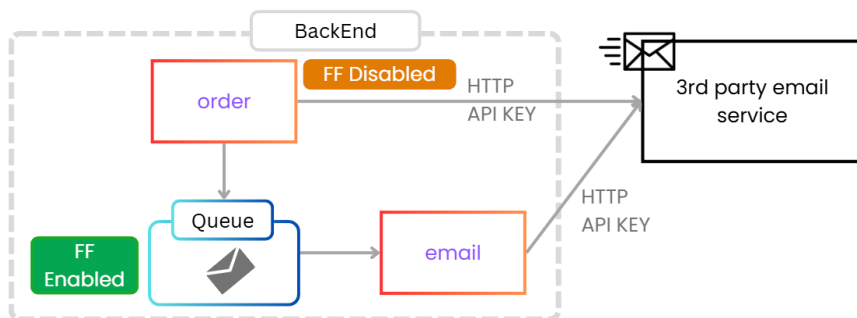
Pero si esa API Key la estás pasando como parte de la configuración, tienes que volver a desplegar cada aplicación que utiliza el sistema de email. Eso suponiendo que tienes conocimiento de todos los servicios que usan el sistema de email.

Te das cuenta de que esta implementación es un problema, porque se tarda mucho tiempo en arreglar cuando tenemos un problema, por ese motivo decides centralizar los envíos por email en una única aplicación, y esta aplicación será la única que tenga acceso al servicio de terceros. Además, lo harás con comunicación asíncrona a través del patrón productor consumidor.

Una vez tienes el servicio creado, debes ir por todos los servicios y actualizarlos para incluir el código, pero, por miedo a que algo no funcione bien, decides mantener el código antiguo, solo durante el tiempo de testeo. Eso sí, en tiempo de ejecución, pese a que ambos códigos están en la aplicación, solo uno será ejecutado.

La decisión sobre qué implementación utilizar el código la hacemos a través de una *feature flag*.

Si la *feature flag* está desactivada, utilizaremos la implementación antigua donde llamamos al servicio directamente, mientras que, si está activa, generamos un evento.



De esta forma podemos, no solo volver a la implementación antigua si algo no funciona bien, sino decidir la fecha y la hora que queremos activar dicha funcionalidad y tener un control total sobre la misma.

Lo que acabamos de describir es el patrón *Strangler Fig*, un patrón que gradualmente va reemplazando el sistema o la forma antigua de procesar un servicio, *request* o proceso y, eventualmente, lo podemos eliminar completamente.

En sistemas más grandes puede incluso ayudarte a habilitar características en múltiples servicios con un único cambio de configuración.

Ojo, estamos mencionando que un cambio de configuración puede afectar a muchos sistemas y no solo eso, también a los clientes. Pese a ello, las *feature flags* no son configuraciones específicas de cierto cliente. Si estamos haciendo un despliegue gradual de una funcionalidad sí podemos tener en cuenta el cliente, pero NO debemos utilizar la *feature flag* como configuración.

En el caso de que actives una *feature flag* y la funcionalidad falle, volver a la versión anterior puede ser tan sencillo como deshabilitar la *feature flag*. A esta acción se le llama *kill switch*.

A/B Testing

Una vez tienes las *feature flags* implementadas es muy fácil implementar la técnica de *A/B Testing*.

La idea de *A/B Testing* es comparar dos versiones de un proceso y comprobar cuál funciona mejor. Hoy en día lo asociamos principalmente a las páginas webs, pero es un concepto antiguo que se utiliza en ventas desde hace cien años, ya que ese es su objetivo final, conseguir el mayor rendimiento posible, ya bien sea vendiendo patatas, manteniendo a los usuarios en nuestra web o haciendo que el proceso sea más rápido.

El *A/B Testing* suele estar decidido por tiempo. Por ejemplo, cambiar la forma en la que un botón está en la pantalla cada 5 minutos, o, también, por cliente. Por ejemplo, ahora, todos los usuarios de X país ven la nueva implementación, mientras que el resto siguen con la antigua.

La clave es monitorizar todos los valores para poder llegar a la mejor decisión posible.

Migración de monolitos a microservicios y sistemas distribuidos

Me gustaría acabar este capítulo con una sección muy importante hoy en día en el entorno empresarial, que es el proceso de la migración desde los monolitos a los microservicios.

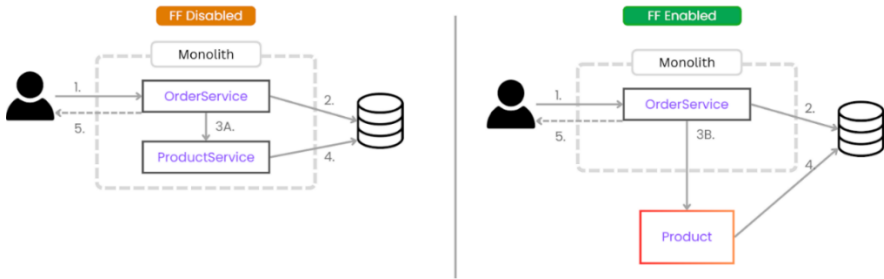
La idea es simple, dividir un monolito en múltiples aplicaciones pequeñas, pero su implementación no es para nada simple.

El primer paso en toda migración de este estilo es separar e identificar el código o funcionalidades que debemos migrar, y de ahí cambiar el código.

Disponemos de múltiples formas de implementar dicho cambio, y un elemento crucial de esta decisión es la base de datos. Si tenemos una sola base de datos para todos los microservicios el cambio es tan simple como una vez identificado, desarrollado y desplegado el nuevo servicio con la lógica, creamos una *feature flag* y utilizamos el nuevo proceso.

Supongamos que hemos migrado el servicio de productos, todas las acciones, tanto de lectura como de escritura, se deben hacer a través de la API de productos.

La implementación segura de esta funcionalidad es con el patrón *Strangler Fig* tras una *feature flag*, que nos permita ir a través de la API.



Como vemos, el proceso es similar en ambos escenarios.

- 1) El usuario genera la *request*.
- 2) El código obtiene la información de los pedidos.
- 3) Eventualmente, necesita información de los productos:
 - a) (*Feature Flag* deshabilitada) Realiza la comprobación dentro del propio monolito, en la parte de productos.
 - b) (*Feature Flag* habilitada) Realiza una llamada HTTP (o gRPC) para obtener la información de los productos a través de la API.
- 4) Obtenemos los productos de la base de datos.
- 5) Devolvemos la información al usuario.

Este proceso es exactamente igual para consultar información como para añadir nueva información; el único punto de posible control es la *feature flag* para habilitar o deshabilitar el uso del nuevo microservicio, ya que, aunque no tengamos toda la parte de los productos migrada, todo puede seguir funcionando igual, ya que es la misma base de datos.

Por ejemplo, si tenemos lectura y escritura separadas, podemos empezar migrando la parte de lectura a la API, y no tenemos que preocuparnos de la parte de escritura hasta terminar la API de lectura ya que utilizan la misma base de datos.

En cambio, si además de migrar a microservicios, estamos migrando bases de datos, la cosa cambia, y no sólo cambia, sino que se complica.

Ahora, ya no tenemos una única base de datos, lo que implica que cualquier consulta o parte del sistema que esté utilizando el dominio de productos va a verse fuertemente afectado. Si tenemos una clara separación de dominio, quizá no sea muy difícil identificar todos los puntos donde tocamos la parte de productos, Pero, seamos realistas, en el mundo real rara vez está todo bien separado en módulos de una forma correcta.

Para que funcione bien hay que aplicar una fórmula un tanto diferente, ya que debemos mantener dos bases de datos completamente sincronizadas.

Nos volvemos a poner en situación, donde estamos migrando la parte de productos a su propio API con su propia base de datos, pero, por ahora, únicamente nos ha dado tiempo a migrar la parte de lectura.



Mientras que la de escritura sigue escribiendo en la base de datos antigua.

Lo primero que te puede venir a la mente es que, debido a que escribimos en una base de datos, y leemos en otra, no podemos habilitar la API, pues la de lectura no tiene información en ella.

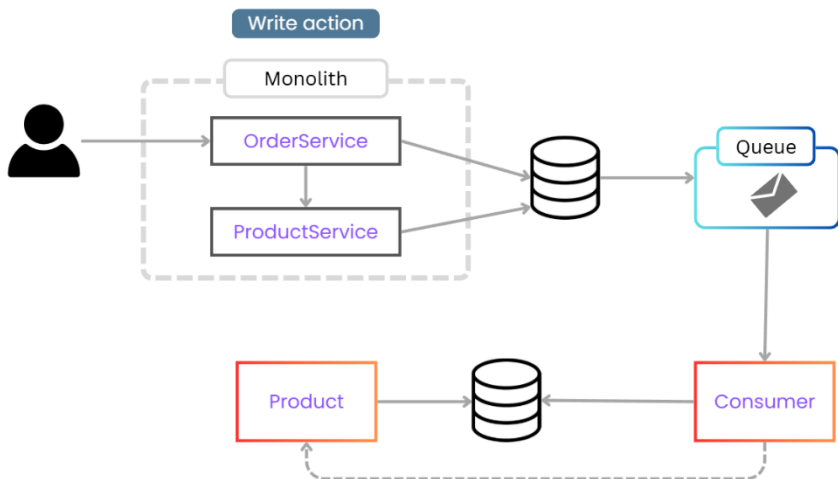
Aquí es donde entra este proceso, el cual consiste en generar eventos para mantener ambas bases de datos en sincronización.

Para ello tenemos una capa extra, llamada la capa de anticorrupción (ACL, *Anti Corruption Layer*). Esta capa se encarga de hacer la traducción entre ambos sistemas, y va a permitir: primero al monolito modificar la base de datos del microservicio, y luego al microservicio sin tocar la base de datos *legacy* el mantener la información sincronizada.

Algo que hay que tener en cuenta, es que esta sincronización funciona en ambos lados. Lo más probable es que al migrar a microservicios cambies ciertos procesos, o que, algún cliente, ya sea externo o interno, necesite integrarse lo haga a través de la API y no el microservicio, por ello, la sincronización en ambas direcciones es necesaria.

Vamos a empezar con la sincronización desde la parte del monolito a la de microservicios. Recuerda, esto solo se hace a la hora de crear o alterar registros de la base de datos.

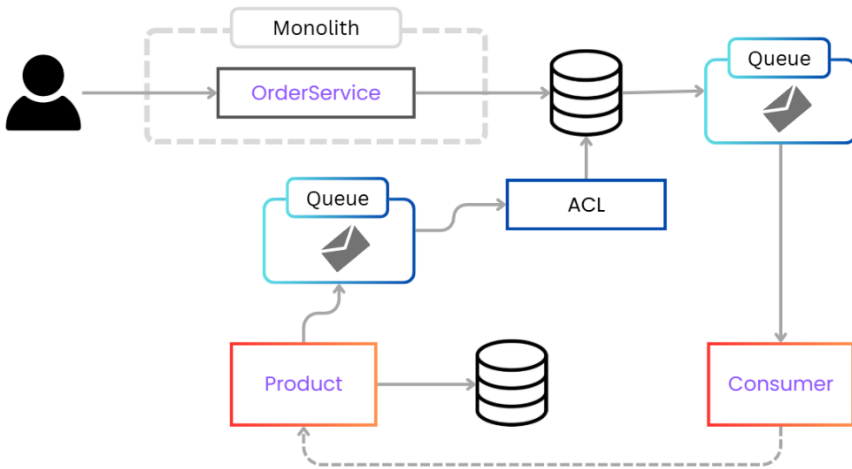
Si tienes el código bien estructurado es posible que solo tengas un punto de entrada a la hora de crear pedidos y otro para actualizar. Si es así, enhorabuena, puedes implementar el código de una forma más correcta, generando un evento desde el código. Como seguramente no sea el caso, tendrás que crear eventos desde la propia base de datos. Utilizando un *Change Data Capture* (CDC) generamos un evento, el cual es escuchado e inserta los datos en la nueva base de datos.



De esta forma, la información está sincronizada.

Idealmente, el consumidor debería ir por la API a la hora de crear los registros, pero recuerda que no tenemos nada de código de la parte de crear en nuestra API, así que, por ahora, inserta o modifica datos directamente en la base de datos.

Un escenario similar sucede cuando modificamos la base de datos del microservicio. En este caso, debemos generar un evento, ese evento va a ser escuchado por la *Anti Corrrption Layer*. Esta capa no solo se encarga de introducir los datos, sino que también se encarga de asegurarse de que están completos. Es muy común, al migrar a microservicios, que reduzcamos el tamaño de las tablas, por ello es posible que tengamos que consultar información a otros servicios antes de insertarlo todo en la base de dato *legacy*.



Finalmente, debemos asegurarnos de que los eventos no se generan en bucle; cuando modifiques algo en el microservicio, que se actualice en el monolito, pero que no lance otro evento que actualice el microservicio y así hasta el infinito.

Si en el monolito generas los eventos desde el código, no tienes problemas, pues el ACL escribe directamente en la base de datos. Pero si estás generando los eventos desde la base de datos, debes tener algo más en consideración.

Una opción puede ser incluir la fuente o quién ha modificado o creado el último registro, y tenerlo como una columna de la propia tabla. Si el cambio ha venido desde el microservicio, no generamos un evento.

La segunda opción, disponible en muchas bases de datos, es identificar quién hace cada acción. Por ejemplo, utilizando en la conexión a la base de datos (*connectionstring*) podemos habilitar la propiedad "Source=microservice". De esta forma, podemos configurar nuestro sistema CDC para que ignore cualquier cambio de datos que haya sido generado por dicho *Source*.

CAPÍTULO 14

Introducción a los servicios en la nube

La gran mayoría de empresas que utiliza microservicios no tiene sus propios servidores, lo más común es tener un proveedor de servicios que te proporcione dichos servidores. Estos proveedores no proveen únicamente con los servidores físicos, sino que además nos proporcionan software que nos facilita el uso de diferentes patrones o funcionalidades.

Los proveedores de servicios más populares son AWS (Amazon Web Services), Azure (Microsoft Azure) y GCP (Google Cloud Platform), siendo el más popular, con un amplio, margen AWS.

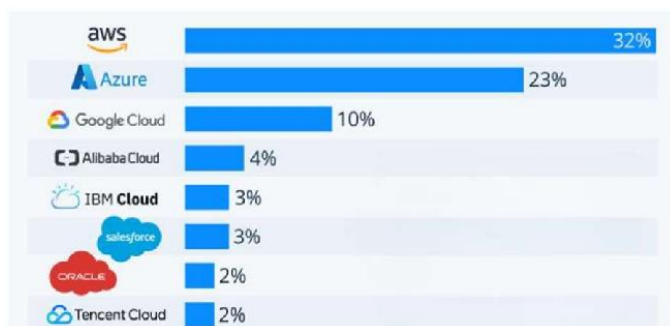


Imagen a octubre de 2024, Fuente:

https://www.researchgate.net/publication/385856698_Research_and_analysis_of_issues_and_challenges_in_ensuring_cyber_security_in_cloud_computing

A la hora de trabajar con ellos el proceso es similar, ya que todos aportan servicios que realizan prácticamente las mismas funciones: cambian los nombres y la implementación, pero la idea detrás de dicha implementación, o sea, el funcionamiento para el usuario final es el mismo.

Por ejemplo, tanto Azure, AWS como GCP tienen en sus webs oficiales que proveen más de 200 servicios cada uno. Al final son competidores directos y les interesa tener lo que tiene el otro.

Serverless

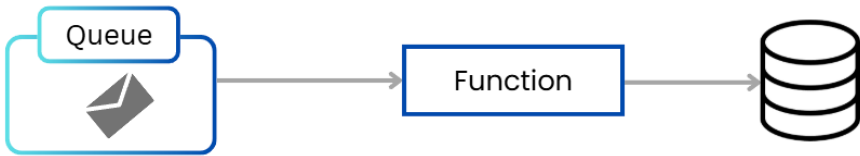
El primer punto donde debemos parar es *serverless*, que, técnicamente, significa sin servidor. Pero voy a contar un secreto. Las aplicaciones no funcionan en una nube mágica, corren en un servidor, solo que es un servidor de otro, ese otro, es en la gran mayoría de casos nuestro proveedor de servicios.

Durante 2024 y comenzando este 2025 se ha visto un amplio ámbito de empresas que están empezando a migrar de la nube a sus propios servidores, porque sí, todo lo que puede dar la nube es posible replicarlo en un servidor propio, si eres capaz de administrar la infraestructura.

Así que podemos decir que *serverless* es un modelo de ejecutar aplicaciones o, incluso, sistemas donde nos centramos en la lógica, dejando toda la infraestructura a cargo del proveedor de servicios. Y será el proveedor quien asigne los recursos necesarios para ejecutar cualquier software que necesitemos.

El caso más común, o con el que vemos más claro que es *serverless*, son las Azure functions/Lambda functions. El proveedor de servicios nos proporciona la infraestructura necesaria para ejecutar una función de código en respuesta a una acción del sistema.

Por ejemplo, podemos tener una cola, y cuando un elemento llega se ejecuta una Azure/Lambda function.



Uno de los grandes beneficios de *serverless* es que escala con el uso. Desde 0 hasta casi el infinito, como es el caso de las funciones que hemos mencionado.

Por ejemplo, si tenemos una cola de mensajes que cuando recibe un mensaje se ejecuta una función, vamos a ejecutar una función por cada uno de los mensajes de forma individual. Si recibimos diez millones de mensajes en un día, invocamos diez millones de funciones. Mientras que si no recibimos ningún mensaje no invocamos ninguna función, por lo tanto, mantener ese código nos sale gratis.

Y no solo eso, la parte de infraestructura es muy simple de configurar, ya que a tu proveedor de servicios le interesa que puedas hacer todo esto. Es más, ese mismo mensaje, podrías almacenarlo en un sistema de ficheros con una línea de configuración.

En un sistema *serverless* tenemos dos actores:

- El proveedor, el cual nos da la infraestructura necesaria para correr un sistema.
- La empresa (nosotros), la cual debe escribir el código con la lógica de dominio y conectar todas las piezas.

Grosso modo, esta podría ser una comparación y tiene sentido, pero también es cierto que el proveedor quiere que la empresa tenga que escribir el menor código posible.

Cuanto más código, más posibilidades de fallo y mayor descontento con los servicios, aunque no sean culpa del proveedor. Por ello, el proveedor no solo nos da una infraestructura, nos da los elementos necesarios para que las empresas escriban el menor número de código posible. La idea es que el consumidor escriba lógica de negocio, y, a través de *triggers*, ir ejecutando distintas partes del sistema, por ejemplo, un evento llega a una cola.

La otra parte es conectar servicios. Los proveedores dan la infraestructura, pero, además, nos dan una serie de servicios populares. Por ejemplo, un sistema de ficheros, bases de datos, sistemas para la autenticación, propios sistemas para hacer el patrón productor/consumidor. Todo esto, que es complejo, lo administra el proveedor y, desde el código o lógica del sistema, nos centramos en conectar las cosas juntas de una forma correcta.

Por supuesto, tiene un coste. El coste en *serverless* puede ser muy barato, si tu sistema tiene poco uso, o puede ser muy caro. En este tipo de sistemas pagas por utilización, no por asignación de recursos.

Si tienes una pequeña web desplegada en un servidor, tendrás un servidor con 1 o 2 vCPU con 2 o 4gb de RAM. Esos recursos los tienes siempre disponibles, el 100% del tiempo que contrates el servicio, y puedes poner en dichos servidores lo que quieras. Esta opción existe en los proveedores, y son las máquinas virtuales, pero normalmente utilizaremos un proveedor por los servicios que nos da. Nos cobrará X dinero por cada vez que leamos un fichero del sistema de ficheros nos cobrará X por cada invocación de una *lambda function*, pero si no lo utilizamos, no pagamos.

Además, también se paga por el consumo de recursos durante el tiempo de ejecución. Cada servicio y proveedor varía en precio en función de lo que se hace, pero, obviamente, no cuesta lo mismo una *lambda/Azure*

function que se ejecuta en 200 milisegundos, que una que se ejecuta en 20 segundos.

El uso de *serverless* facilita mucho la velocidad de desarrollo de todo lo que no es lógica de negocio, ya que no te tienes que preocupar de si un servidor está roto y hay que arreglarlo, o de si cierto servicio funciona como se espera. De eso se encarga el proveedor y te lo garantiza con un *Uptime* en el contrato.

Antes de terminar este punto, he dicho que no definimos la infraestructura, pero debemos poner límites. Al trabajar con *serverless* o arquitecturas en la nube en general si no ponemos ningún límite, el coste se nos puede elevar muchísimo, una mala configuración, una tarea que dejas ejecutándose 24/7 y te encuentras a final de mes con facturas de miles y miles de dólares completamente inesperados que pueden quebrar empresas.

Por ello, aunque el proveedor nos deje escalar hasta el infinito, hay que poner unos controles para que eso no pase. Cuando la empresa lleva un tiempo utilizando un proveedor de servicios, pararse a ver qué es lo que se está haciendo y reevaluar u optimizar el uso es crucial para salvar miles de euros mensuales.

Servicios específicos populares

Los proveedores de servicios, además de *serverless*, nos dan servicios propios suyos. En este libro hemos estado hablando todo el tiempo de contenedores. Estos contenedores son servicios que se ejecutan 24/7, por lo tanto, no son *serverless*.

Algunos de los servicios puede que funcionen internamente como si fueran *serverless*, pero a nosotros no nos afecta directamente, ya que

únicamente los utilizamos, no los administramos. En este apartado, voy a mencionar algunos de los doscientos servicios que los proveedores tienen, los que, en mi experiencia, son más populares, porque la realidad es que, de los doscientos, la gran mayoría de empresas usan un puñado de ellos de forma “común”, y luego hay otra serie de servicios ya específicos para cada caso de uso.

Gestión de contenedores

El software de gestión de contenedores más popular en entornos de producción es Kubernetes, el cual es un software libre para la administración de contenedores. Entre otras cosas, podemos configurar como las aplicaciones van a escalar. Y en los proveedores de servicios puedes utilizarlo, o puedes utilizar la versión propia de los propios proveedores de servicios, donde es todo igual, pero administran el *control plane*.

Estos servicios son Amazon EKS (Amazon *Elastic Kubernetes Service*), AKS (Azure *Kubernetes service*) o GKE (Google *Kubernetes Engine*). Los tres funcionan con la misma idea, permitirle desplegar contenedores utilizando Kubernetes.

Contenedores “*serverless*”

Lo más común que he visto en mi experiencia en el ámbito de contenedores al utilizar proveedores de servicios es no querer administrar nada, o mejor dicho, lo menos posible. No vas a tener que lidiar con *clusters*, o ningún tipo de infraestructura. Simplemente definen las características de cada uno de tus contenedores, cuánta CPU, cuánta RAM y cuántas instancias quieres (tanto mínimas, como máximas) y a funcionar.

En GCP tenemos Cloud Run, en Azure tenemos Container Instances y en AWS tenemos Fargate.

Personalmente, diría que es la forma más común de correr contenedores en la nube.

Almacenamiento de objetos

Como puedes imaginar, cualquier proveedor de servicios te va a dar un sistema de almacenamiento. En la nube es común almacenar objetos y estos objetos son tus ficheros, cualquier tipo de fichero.

En Amazon tenemos S3, en Azure tenemos Blob Storage, y en GCP tenemos Cloud Storage. Estos servicios, además de permitir la administración de los ficheros, nos permiten añadir versiones, seguridad de acceso a nivel muy granular, encriptar la información, o, incluso, configurar un tiempo de vida para que los objetos se eliminen automáticamente.

Además, los tres tienen la opción de archivar los objetos, donde el almacenamiento de dichos objetos es mucho más barato que en el sistema normal pero su acceso es más lento y caro.

La idea del almacenamiento de objetos es tener los ficheros con clave/valor. Cada nombre será una clave única, no solo en tu empresa, sino una clave única en todo el sistema de objetos del proveedor, por lo que el almacenamiento de objetos se parece más a una base de datos que a un sistema de ficheros.

Finalmente, no está optimizado para rendimiento (por ejemplo, leer miles de objetos en bucle), y puede tener limitaciones con el tamaño de los ficheros. Si ese fuera el caso tenemos otra alternativa, que es el propio sistema de ficheros. Esta vez sí actúa como un disco duro y si tienes máquinas virtuales, lo puedes añadir a dichas máquinas.

Si se diera el caso de que esto es lo que necesitas, que será muy poco, tienes *Elastic file system* (EFS) en Amazon, *Azure Files* y *FileStore* en GCP.

Máquinas virtuales

En ciertas ocasiones, o en ciertos lenguajes, necesitaremos máquinas virtuales para ejecutar nuestras aplicaciones. Estas máquinas virtuales se pueden configurar con una imagen personalizada y actúan como un servidor o un ordenador normal.

El uso más común que vemos hoy en día es que, con ciertos lenguajes, o versiones de dichos lenguajes, no se pueden desplegar en contenedores, como es el caso de .NET Framework, por lo que necesitamos una máquina virtual, a la que le asignaremos recursos y desplegamos la aplicación en ella. A estas máquinas virtuales les llamaremos *instancias*, y las podemos monitorizar igual que monitorizamos un contenedor, para saber si debemos escalar o no, aunque tardan algo más de tiempo.

Otro uso de estas máquinas virtuales es para acceder a los recursos de un entorno. Es muy común (y recomendado) no tener acceso a la base de datos de producción desde una máquina local, y, para acceder a ella, tienes que acceder a través de una máquina virtual desplegada en dicho entorno. Estas máquinas tienen configuración para saber el usuario, los permisos y, por supuesto, donde dicho usuario puede acceder.

En los tres proveedores principales tenemos máquinas virtuales, en AWS se llaman *EC2*. En Azure es *Azure Virtual Machine*, y, finalmente en GCP es *Google compute engine*.

Algunas empresas hacen trabajar en ellas a sus trabajadores en vez de en su máquina local, si este fuera el caso, escapa de dicha empresa, porque trabajar así es, posiblemente, la peor experiencia que tengas en tu vida.

Funciones *serverless*

Todo el punto anterior lo hemos pasado explicando qué es *serverless*. Podemos resumir en que el proveedor de servicio nos da un marco de trabajo donde poner pequeñas partes de nuestro código, y nos cobra tanto por número de requisitos como por el uso en tiempo y recursos de estas.

En AWS tenemos *las Lambda functions*, en Azure tenemos las *Azure functions* y en GCP tenemos las *GCP Cloud functions*. La idea es la misma en todas ellas.

Dependiendo de qué proveedor tengas, tendrás unas limitaciones u otras en términos de RAM, tiempo de ejecución o uso del procesador, pero la idea de una función *serverless* es que se ejecute de una forma rápida, por lo que, en la mayoría de los casos, no vas a tener dichos problemas.

DNS

Si queremos sacar nuestra web al exterior, ya sea de forma privada o pública, necesitaremos un DNS con el que podemos registrar dominios, hacer enrutamientos DNS o, incluso, hacer *health checks*.

En AWS podemos hacerlo todo dentro de *Route53*, mientras que en Azure tendremos dos servicios diferentes DNS, para los registros DNS y *Traffic manager* para enrutamiento, o los *health checks*. En GCP tenemos *Google Cloud DNS*.

Redes privadas

Todo nuestro sistema se ubica en algún lugar. Ese lugar es en el proveedor de servicios, pero para que nuestros servicios no se vean afectados por los de otra empresa, o, incluso, la misma empresa, pero distintas cuentas, utilizamos lo que se llama una Red privada.

En esta Red privada tendremos un control total del entorno: las aplicaciones que utilizamos, conectividad y seguridad. Esto incluye qué selección de IPs vamos a utilizar, o *subnets* dentro de la misma Red privada.

Existen en todos los proveedores, en GCP se llaman *Virtual Private Cloud* (VPC), exactamente igual que en AWS, mientras que en Azure se llama *Virtual Network*.

Estas redes son privadas y controladas. Puede darse el caso de que necesites conectar varias, no solo eso, sino que al conectarlas ellas se verán como una sola. para ello utilizaremos en Azure *VNET Peering*, en AWS tenemos *VPC Peering* y en GCP tenemos *VPC Network Peering*.

También tenemos la posibilidad de tener que conectarnos desde una red privada a un servicio de otra red privada, pero en este caso, no queremos que ambas redes se “combinen”, simplemente queremos acceder de una a la otra. Para ello en Azure utilizaremos *Private Endpoint*, en AWS tenemos *VPC endpoint* y en GCP usamos *private Service connect*.



En este escenario, tenemos dos redes privadas, y el servicio A necesita acceder a datos del servicio B, por lo que tenemos una VPC/Private endpoint con acceso únicamente al *Load Balancer* del servicio B.

Balanceadores de carga

En este libro tenemos toda una gran sección para hablar de los balanceadores de carga. La idea es tener estos balanceadores para que distribuyan el tráfico en las diferentes *Availability Zones* si tenemos más de una, claro, que siempre es lo ideal.

En AWS utilizamos *Elastic Load Balancer* (ELB), en GCP usamos *Cloud load balancing*, y en Azure es *Azure elastic load balancer*.

La idea es la misma en las tres plataformas, pero, además, puede haber otras subdivisiones, ya que, dependiendo del tipo de tráfico, tendremos que utilizar un tipo específico de *Load Balancer*.

Para el tráfico HTTP/HTTPS, o gRPC, utilizaremos en AWS el *Application Load Balancer* (ALB), el mismo nombre es utilizado en GCP, mientras que en Azure se llama *Application Gateway*. Este Load Balancer también permite *cookies* y trabaja sobre la capa de aplicación (capa 7). Además, tiene soporte para terminación de SSL, enrutamiento, o mapeo de URLs.

Para el tráfico TCP/UDP utilizaremos un balanceador de carga diferente. En AWS será el *Network load balancer* (NLB), mismo nombre en Google Cloud, y en Azure se llama *Load Balancer*. En este caso trabaja sobre la capa de transporte (capa 4) y permite distribuir de forma eficiente el tráfico TCP y UDP preservando la IP del cliente original.

En la gran mayoría de escenarios, especialmente si trabajas con contenedores, el tipo de balanceador de carga será un balanceador de carga que funcione sobre la capa de aplicación.

Distribución de mensajes - Comunicación asíncrona

En el capítulo sobre la comunicación asíncrona hemos hablado de *Event Buses*, *colas* y *streams*, y hemos ido en detalle sobre Kafka y RabbitMQ. Ambos servicios son servicios populares y, posiblemente, los más populares, aun así, los distintos proveedores de servicios nos ofrecen diferentes opciones.

Para trabajar con colas podemos utilizar en GCP el servicio *Google Cloud Pub/Sub*, con el modo suscripción, mientras que en Azure utilizamos *Azure Service Bus* y en AWS utilizamos *Amazon Simple Queue Service* (SQS).

Para un *stream* de eventos podemos usar *Event Stream* en AWS (*kinesis stream*), y *Azure Event Hubs*. En GCP seguimos con *Google Cloud pub/sub*.

Finalmente, para un *eventbus* con productor-consumidor, en Azure tenemos *azure event grid*, en AWS tenemos *Amazon event bridge* y en GCP seguimos con *Google Cloud pub/sub*.

Saber si debemos utilizar un servicio que nos provee un proveedor o uno administrado por nosotros, va a recaer en saber si somos capaces de hacer nuestro servicio redundante, fiable y que escale si necesita escalar. Si no sabemos cómo hacer esto, mejor ir por uno de un proveedor de servicios, que nos costará más dinero, sí, pero no colapsará.

Bases de datos

No hay mucho que explicar; bases de datos dentro de los propios proveedores.

Para bases de datos relacionales en AWS tenemos *Amazon RDS*, en Azure tenemos *Azure SQL Database managed instance* y en GCP tenemos *GCP Cloud SQL*.

En AWS tenemos soporte para *SQL Server*, *Oracle*, *MYSQL*, *PostgreSQL* y *MariaDb*, en Azure es lo mismo, pero sin *Oracle*, y en GCP debemos eliminar además de *Oracle* el soporte de *MariaDB*.

Alternativamente, en AWS podemos tener *Amazon Aurora*, que es una base de datos *serverless*, y distribuida con un escalado prácticamente ilimitado, y sin ninguna infraestructura que administrar. Por cierto, es compatible con *PostgreSQL*.

En el mundo de NoSQL, o bases de datos de documentos, en AWS tenemos *DynamoDB*, que es compatible con MongoDB, en Azure tenemos *CosmosDB* y en GCP tenemos *Cloud Firestore*.

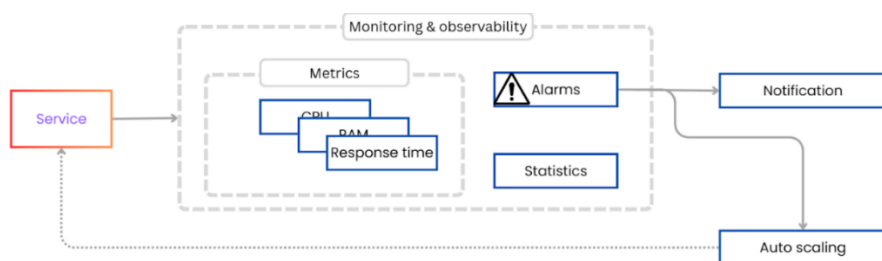
Hay más tipos de bases de datos, como pueden ser *key-value*, *big data*, *data warehouse*, *time series*, etc. pero relacionales y no relacionales son las más comunes.

Monitorización y observabilidad

Como hemos visto en este libro, tanto la observabilidad como la monitorización son partes cruciales dentro de cualquier sistema en producción.

Para los *logs*, métricas y alertas, si utilizamos AWS tenemos *cloudwatch*. Además, una gran cantidad de alertas nos vienen casi configuradas por defecto, donde únicamente tenemos que especificar qué queremos controlar, por ejemplo, el uso de la CPU, y podemos configurar con una línea de código que salte una alerta si P95 falla, por ejemplo.

Lo mismo podemos hacer en Azure, con *Azure monitor*. Mientras que en GCP utilizaremos *Cloud Monitoring* y *Cloud Alerting*.



Estas mismas alertas y monitorización son utilizadas para el auto escalado de las aplicaciones, si usamos los propios servicios de los proveedores.

Otros servicios comunes

Una última sección donde voy a mencionar otros servicios comunes que hemos visto en este libro, como son:

- *Secret manager* para la administración de credenciales.
- *API Gateway* para una puerta de enlace a nuestro sistema.

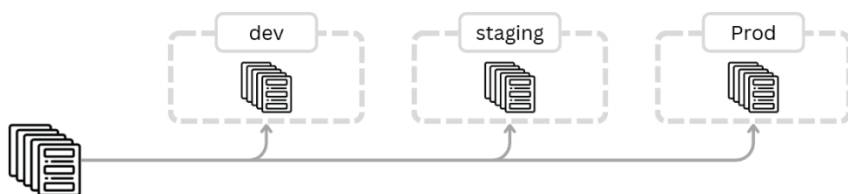
- CDN para devolver datos estáticos reduciendo la latencia y mejorando el rendimiento.
- Autorización y autenticación, tanto para nuestras aplicaciones como para el sistema en sí, los proveedores nos permiten tener sistemas para administrar usuarios y sus respectivos accesos.

Infraestructura como código

Al usar servicios en la nube, o un proveedor de servicios, tenemos la posibilidad de hacer todo con *clicks* en la pantalla, o lo podemos hacer a través de ficheros de configuración. Si elegimos la segunda opción, lo que estamos haciendo es Infraestructura como código (*Infrastructure as code*, IaC), donde proveemos la infraestructura del sistema a través de software, con el objetivo de conseguir consistencia en la configuración a través de los entornos.

Este proceso es tan sencillo como tener los ficheros de configuración y ejecutarlos en el servicio correspondiente dentro del servicio correspondiente, como puede ser *Cloudformation* en AWS, *Azure Resource Manager* en Azure, o, en GCP, podemos utilizar *Deployment manager*. Alternativamente podemos escribir nuestros propios *scripts* o, incluso, implementar *terraform* en nuestro sistema; que es otro software para la administración de la infraestructura.

En cualquier caso, lo que importa es saber que la infraestructura no la vamos a hacer a través de *clicks* en la pantalla, sino que la haremos a través de código; el cual será ejecutado en cada uno de los entornos.



Estos ficheros contienen la información necesaria para construir el sistema en cada uno de los entornos.

Por ejemplo, podemos definir una configuración para que una aplicación en *dev* y en *staging* tenga una instancia, y esa instancia tenga 1 vCPU y 0.5gb de RAM.

Características de la infraestructura como código

La infraestructura como código es código

Como su nombre bien indica, la infraestructura como código es código, y, como tal, lo tendremos en un fichero (o varios), y almacenado en el sistema de control de versiones. Como cualquier otra parte del sistema, requiere de revisiones de código, que alguien apruebe dicho cambio, y un historial sobre quién ha cambiado cada parte.

Infraestructura como código declarativa e imperativa

Al definir la infraestructura como código lo podemos hacer de dos formas. La primera, **declarativa**, es cómo funcionan los servicios de los proveedores, donde definimos qué queremos que nuestra infraestructura contenga.

Por ejemplo, queremos un servidor web con un balanceador de carga y ciertas alertas. En el fichero de configuración de la infraestructura, definiremos el balanceador de carga y pondremos una referencia al mismo desde la configuración del servidor web. Lo mismo para la alerta, enlazamos la alerta con nuestro servidor.

En cambio, si utilizamos la forma **imperativa**, no vamos a utilizar los servicios que nos dan los propios proveedores, sino que utilizaremos la línea de comandos a través de scripts.

Consistencia y variabilidad

Da igual cuánta información tengamos en la configuración, en todos los entornos se ejecutará con las mismas reglas y dará el mismo resultado.

También puede darse el caso de que, en producción, esa misma aplicación, puede estar configurada para tener tres instancias, cada una con 2 vCPU y 2GB de RAM. Además, le podemos indicar el escalado que deseamos.

Imagínate que pasan los años, y esa misma aplicación ahora tiene mucho más uso. En vez de tres instancias, necesitamos diez como mínimo. Cambiando el fichero y desplegando en nuestro proveedor, este se encargará automáticamente de hacer todos los cambios necesarios para que eso suceda.

Además, mientras no cambiemos los valores, todo el sistema será consistente y no se verá afectado por otros posibles cambios, o cambios en otras partes del mismo sistema, lo que da una consistencia muy importante, ya que, incluso si ejecutamos el fichero veinte veces, al no tener cambios, nada cambiaría.

Plantillas

Otro punto clave, son las plantillas, no tendría mucho sentido él si tenemos cien aplicaciones con cien ficheros distintos con toda la configuración. Lo que sí tiene sentido es tener ficheros con la

configuración que es común entre ellos y variables que pueden ser sobrescritas en cada uno de los ficheros específicos de las aplicaciones.

Si cada aplicación, por defecto, son tres instancias de cierto tamaño, junto con un *Load Balancer* y las alertas necesarias para que escale hasta cierto nivel. Para ello tenemos una plantilla con esa configuración y podemos sobrescribir en los servicios que tienen una mayor carga, simplemente el número de instancias o el tamaño de estas.

Como puedes imaginar, el uso de plantillas es esencial a la hora de tener sistemas grandes, porque de otra forma se vuelve imposible de administrar.

Regiones y *Availability Zones*

Para terminar el capítulo hablemos de las regiones y las *Availability Zones*, que son las ubicaciones físicas donde nuestro sistema va a estar siendo ejecutado.

Regiones

Una región es una localización física donde los proveedores de servicios nos proveen de los propios servicios de una forma aislada e independiente del resto de regiones.

La idea de estas regiones es que están esparcidas a través del mundo y depende a qué parte del planeta tu software dé servicio, o requiera diferentes regulaciones utilizarás unas regiones u otras.

Al ser localizaciones físicas, si tus clientes son la mayoría de Europa, lo ideal es elegir una región europea, por ejemplo, AWS tiene una en Irlanda (eu-west-1).

Si tus usuarios están en México, lo ideal es elegir una región que esté en México, o cerca, en concreto en Azure tenemos una en Querétaro State (Mexico Central).

O, si damos soporte en Australia, GCP tiene una en Sídney (Australia-southeast1).



Es importante elegir bien el sitio donde vamos a desplegar nuestra aplicación, porque la latencia va a ser clave. Si un usuario está en Italia, la latencia entre las diferentes regiones será diferente:

- Italia a Irlanda: 100 ms
- Italia a México: 200 ms
- Italia a Australia: 350 ms

Como vemos, cuanto más lejos, más latencia y peor experiencia de usuario, así que dependiendo del tamaño de la empresa y de la cantidad de clientes que tengamos, tendremos el código duplicado en múltiples regiones para que nuestros usuarios puedan acceder de una forma más

rápida, ya que los usuarios en Europa acceden a la región europea, mientras que los de México o LATAM, en general, acceden a la región de México.

Algo que hay que tener en cuenta es que no tenemos regiones en todas partes. Al final son edificios físicos que lleva tiempo construir. Hay zonas del mundo con varias, como Estados Unidos, que las hay repartidas por el país, pero puede ser que tengamos zonas sin cubrir, así que no siempre es posible tenerlas en un sitio muy cercano.

De la misma forma, hay regiones donde las novedades llegan antes que, a otras, por ejemplo, en AWS, us-east-1 (*North virginia*) recibe las actualizaciones o los nuevos servicios antes que el resto de las regiones, ya que hacen un despliegue similar al canario, donde comprueban que todo funciona, y ya, de ahí, despliegan al resto de regiones.

Availability Zones

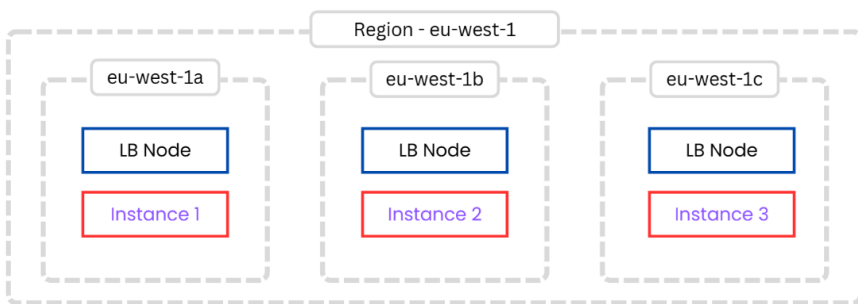
Dentro de cada región tenemos lo que se llama una *Availability Zone*, que son los centros de datos (*datacenters*) de los proveedores, lo que quiere decir, el servidor físico donde va a correr el software.

Cada región puede tener una o múltiples *Availability Zones* y, normalmente, se numeran de 1-N o de A-N. Estas regiones son independientes unas de otras completamente, por lo tanto, son edificios diferentes, con diferentes sistemas eléctricos, alcantarillados, etc. La idea es que si un *datacenter* tiene un problema no afecte al otro. Aun así, están comunicados por cables de red para asegurar que la latencia de la región es mínima.

Por ejemplo, en el caso de AWS con eu-west-1 en Dublín, los centros de datos están ubicados uno al noreste de la ciudad, otro al noroeste y otro al suroeste.

NOTA: en realidad son 32 edificios diferentes, pero actúan como 3 grupos; eu-west-1a, eu-west-1b, eu-west1c.

Ahora viene lo importante. Si tienes una aplicación con tres instancias, técnicamente puedes desplegarlas todas en una *Availability Zone*, pero lo ideal sería desplegarlas a través de las distintas *Availability Zones*.



La forma en cómo funciona es que cada *Availability Zone* recibe un nodo del *Load Balancer* y los usuarios se conectan a dicho *Load Balancer* a través del DNS que enruta a la localización más cercana.

Depende de que proveedor utilices, necesitas diferente configuración como una VPC o diferentes *subnets* con permisos para poder acceder a las diferentes instancias. Pero la idea está clara, repartir el software entre diferentes servidores físicos para así evitar cualquier problema que pueda pasar en dicho servidor.

Entrevista final de diseño

Para la última de nuestras entrevistas vamos a ir con una que es muy popular. Igual que pasó en el caso anterior, no suele dar tiempo en una hora. En mi opinión, lo mejor es empezar con una implementación básica y desde ahí ir iterando. Siempre manteniendo una conversación con el entrevistador ya que eso, también permite a dicho entrevistador tener una idea de lo que sabemos, y qué cosas tenemos en cuenta. Porque no todo lo tenemos que diseñar, muchas partes sirve con comentarlas, porque no da tiempo en una hora.

Bueno una hora, que en verdad son 40 o 45 minutos máximo. Esta entrevista te la pueden hacer tanto para niveles *mid*, *senior*, como *staff*, o, incluso, para puestos de arquitectura. Dependiendo a qué nivel optes simplemente van a esperar más, y un mejor diseño, teniendo en cuenta más casos extremos.

Diseño de una aplicación de mensajería instantánea

“Durante este supuesto vamos a diseñar una aplicación de mensajería, como puede ser WhatsApp, Signal, Telegram, discord, etc.

Tenemos varios requerimientos:

- Chats 1 a 1 y chats grupales.
- Tres ticks, uno cuando se envía, otro cuando se recibe, y otro cuando el usuario receptor lo lee.
- Queremos saber si un usuario está online o no.
- Tenemos un millón de usuarios diarios.
- El sistema está desplegado en un proveedor de servicios como Azure, AWS o GCP.”

Estos requerimientos, pese a ser aparentemente pocos y simples, no lo son, porque en entrevistas de niveles elevados (*staff* o *senior* +) se espera que no te quedes únicamente en los requerimientos. En la descripción mencionan que el diseño va a ser como WhatsApp, como Signal o como Telegram, pero la forma en la que las tres aplicaciones implementan las conversaciones puede ser muy diferente. Esa diferenciación también se espera ver, por lo menos mencionar múltiples opciones. O puedes, incluso, mencionar una implementación que a ti te gustaría.

Recuerda que no hay una única solución válida y que la tuya puede ser tan válida como cualquier otra.

Candidato: ¿Por lo que parece en la descripción el sistema no soporta contenido multimedia?

Entrevistador: Por ahora no. Lo puedes incluir si quieres, pero no es un requerimiento obligatorio.

Candidato: Perfecto. Me gustaría empezar por los requerimientos de almacenamiento y ancho de banda, porque debido al número de usuarios puede ser determinante.

Para simplificar, vamos a asumir que cada mensaje es 1KB, aunque posiblemente sea menos.

Entrevistador: ¿En qué te basas para ese tamaño de mensaje?

Candidato: Entiendo que al enviar mensajes no vamos a enviar únicamente el texto, vamos a enviar el emisor, receptor, la fecha y *timestamp*, así como el propio mensaje. Emisor, receptor y *timestamp* son valores fijos, asumiendo que tanto emisor como receptor son un UUID y del *timestamp* tenemos únicamente los dígitos nos cabe todo en 50 bytes y luego el resto para el mensaje.

Entrevistador: Tiene sentido. Lo podemos dejar así porque será más fácil para los cálculos, pero la media de tamaño de mensaje no pasa los 50 caracteres, así que una estimación más acertada serían 250 bytes o incluso 500 bytes si quieres dejar margen. Para simplificar, podemos utilizar 1KB, no pasa nada.

Candidato: Y si pudiéramos multimedia podríamos poner esos mensajes a 150kb de media.

Entrevistador: Sí, pero no los tengas en cuenta para los cálculos.

Candidato: Al tener un millón de usuarios diarios, vamos a asumir que cada uno envía cincuenta mensajes diarios. Esto significa:

- Almacenamiento diario: $1\text{KB} \times 50 \text{ mensajes} \times 1 \text{ Millón de usuarios} = 50 \text{ GB diarios}$.
- Almacenamiento anual $50\text{GB} \times 365 \text{ días} = 18.25\text{TB}$.

Y para el ancho de banda, podemos asumir que el 15% de los mensajes diarios suceden en la hora pico.

- $15\% \text{ de } 50\text{Gb} = 7.5\text{GB en la hora pico}$.
- $7.5\text{GB} / 60 \text{ minutos} = 125\text{MB/min}$.

Viendo estos números, son elevados, pero no tanto como pensaba. Con una conexión normal de empresa de 100Mb/s debería ser suficiente.

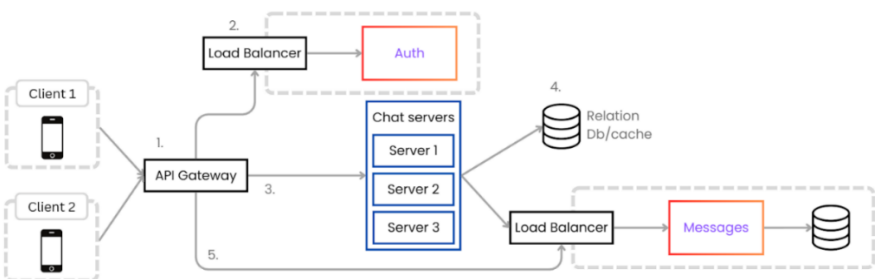
Entrevistador: antes de continuar, dos cosas, ¿te hace falta almacenar todos los mensajes? Y luego, ¿esos cálculos son en base a lo que envían los diferentes usuarios? En los requerimientos hemos indicado que debemos marcar cuando un mensaje es enviado, recibido y leído. Oh, incluso quiero mencionar que, aunque no recibamos mensajes, vamos a seguir consultando constantemente si hemos recibido mensajes nuevos. Con el tamaño que has indicado de 1KB por mensaje, nos sobra tamaño, pero hubiera estado bien una mención.

Candidato: ¿A qué te refieres con lo de almacenar los mensajes?

Entrevistador: bueno, que igual no tenemos necesidad de almacenar todos los mensajes, pero si tu solución lo tiene, perfecto, no hay ningún problema.

En esta primera conversación hemos demostrado que somos capaces de identificar requerimientos implícitos y manejar escenarios reales. Que las matemáticas sean exactas o no, no es lo importante, en lo que el entrevistador se va a fijar es que entendemos los fundamentos del sistema.

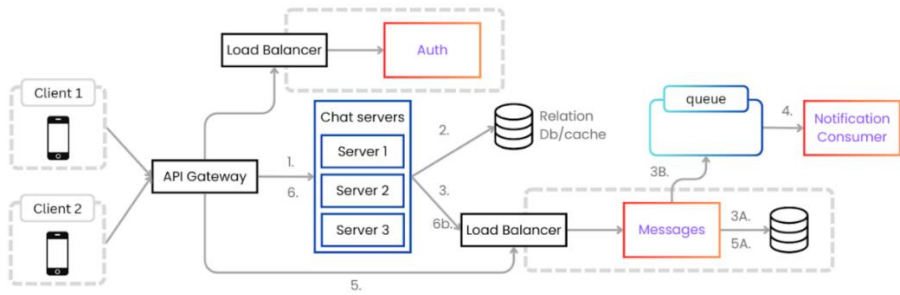
Ahora pasaremos al diseño de la propia aplicación. Donde podemos empezar con un diagrama como el siguiente:



1. Los clientes acceden al sistema a través del único punto disponible, una API Gateway.
2. La API Gateway se encarga de verificar que el cliente es quién dice ser a través de la autenticación.
3. Si es el usuario quien inicia la acción conectamos al usuario a un *chat server*. Cada uno de estos *chats servers* administra un gran número de conexiones y todas ellas son conexiones bidireccionales abiertas, lo que quiere decir que utilizaremos *websockets* para dicha conexión. Así, si recibimos un mensaje mientras estamos en la app, el mensaje lo recibiremos automáticamente en el chat correspondiente, ya que el servidor nos lo enviará a través del *websocket*.
4. Debemos mantener una relación sobre qué usuario está conectado a cada servidor, para que así podamos propagar los mensajes entre los distintos usuarios de forma rápida. Esta información la podemos mantener en una base de datos, o en una caché como Redis.
5. Puede ser que no sea el usuario el que inicie la acción desde la app, que sea la propia app la que busque por actualizaciones; en ese caso irá directamente al *Load Balancer* del servicio de mensajes para ver si hay alguno sin leer.

En esta primera interacción hemos visto los elementos principales de la arquitectura, pero queda mucho más. Algo esencial que debemos explicar es el proceso del envío de mensajes entre el cliente 1 y el 2. Y, en la misma explicación, podemos introducir la explicación a los tres *ticks*, e, incluso el último visto.

Para este paso vamos a añadir un elemento extra. Podemos mencionar al entrevistador que tenemos un servicio para consumir notificaciones, el cual envía *push notification* a los teléfonos móviles. No tenemos que ir más allá, pues la lógica de hacer dicho *push* está abstraída en el servicio y no es lo que nos interesa para este supuesto.



Para el proceso de enviar un mensaje entre el cliente 1 y el 2.

1. El cliente se conecta a su *ChatServer*. Cuando se conecta, reutilizamos el mismo *server* que ya teníamos basándonos en la información de la base de datos.
2. A su vez, esta base de datos tiene un campo adicional que incluye un *timestamp* con la última vez que nos hemos conectado.

ClientID	Server	Last Active
1	server 1	2025-01-23 23:00:00
2	server 1	2025-01-23 14:00:00
3	server 3	2025-01-222:30:00

- 3) Una vez escribimos el mensaje y lo enviamos, este se propaga al servidor donde nuestro cliente está conectado, y si está conectado lo recibirá. Además, se manda al servicio de mensajes.

- a) Donde se almacena en la base de datos, y generaría el primer *tick*, como enviado.
 - b) Si el cliente no está conectado, se genera un evento.
4. El evento es leído por el sistema de notificaciones y genera una notificación *push*.
- 5) La aplicación del cliente recibe el mensaje y se conecta (hace una llamada http) al servicio de mensajes. Esta conexión no se hace a través de los chats servers, sino que se salta esa parte para no tener que conectarse a un *websocket*, y así no actualizar la última vez que estuvo activo.
- a) Lo que hace es conectarse al servicio de mensajes para indicar que ha recibido dicho mensaje, generando el segundo *tick*.
- 6) El usuario entra en la aplicación y en el chat que le ha mandado el mensaje.
- a) El *ChatServer* lee de la base de datos los mensajes nuevos y los marca como leídos, tercer *tick*.

Candidato: Con estos pasos, hemos cubierto el envío de mensajes 1 a 1, los diferentes *ticks* o estados, y la última vez que se han conectado.

Aunque en el estado de la conexión posiblemente la aplicación cliente del móvil deba tener lógica para ver que si el *Last Active* es en menos de los últimos 30 segundos, ponga “en línea”, para dejar cierto margen.

Entrevistador: ¿Algo que indicar sobre la base de datos de los mensajes?

Candidato: Una base de datos creo que es la mejor de las opciones, ya que había pensado un sistema de colas como Kafka, pero la necesidad de hacer los tres *ticks* me hace pensar que es más sencillo y simple con una base de datos. El problema de usar una cola para los mensajes es que sabríamos si lo hemos mandado al servidor gracias a que el mensaje existe y si el usuario lo ha leído gracias a leer el *offset* del topic, pero no si lo ha recibido, lo que puede causar problemas. Algo para tener en cuenta

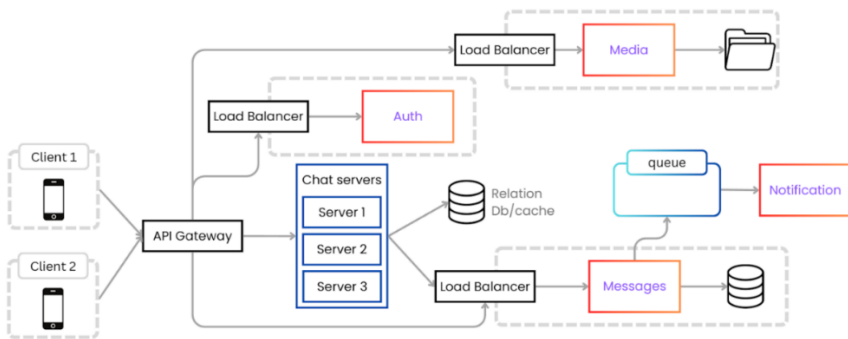
de la base de datos es que debe soportar muchas escrituras, con lo que una base de datos NoSQL es ideal.

Entrevistador: ¿Hay algo que añadir sobre los grupos?

Candidato: Desde mi punto de vista, únicamente necesitamos un microservicio adicional que se encargue de saber quién hay en cada grupo; además de ser capaces de cambiar nombre, información o miembros, ya que si tienes la lista de qué cliente está en cada grupo luego solo tienes que mirar a la base de datos de las relaciones y enviar el mensaje a todos los clientes.

Entrevistador: Como vamos bien de tiempo, me gustaría que incluyeras el contenido multimedia que has mencionado antes.

Candidato: En este caso el diagrama es muy similar. Únicamente debemos añadir un servicio para media el cual se encarga de almacenar en disco, ya bien sea S3 en AWS, o *BlobStorage* en Azure, el contenido multimedia.



Esta información se hace al enviar el mensaje, antes de publicarlo. Subimos el multimedia, el servicio de media nos devolverá una URL con el acceso a dicho contenido, y como mensaje, en vez de enviar texto

plano, enviamos esa URL. Eso sí, la aplicación cliente debe ser capaz de entender dicha URL para, así, descargar la imagen automáticamente en vez de recibir un link.

Lo que debemos hacer es tener múltiples tipos de mensajes, para poder especificar a quien lo consume que no lo lea como texto, sino que vaya al sistema de ficheros y lo lea.

Entrevistador: Háblame de la seguridad. ¿Cómo nos aseguramos de que la información es segura?

Candidato: Idealmente, encriptamos la comunicación entre ambos con certificados.

Entrevistador: Y esos certificados, ¿Dónde están?

Candidato: En cada teléfono. Cuando abrimos la conversación, compartimos la clave pública, y leemos la clave pública. Y hacemos la doble encriptación con nuestra clave privada, y la clave pública del destinatario.

Entrevistador: Pero en esa implementación los mensajes en el servidor estarían encriptados y no sería posible leerlos por las autoridades. Oh, otra cosa. ¿Qué sucede si te cambias el teléfono? ¿Ese certificado se pierde, o qué sucede con él y con las conversaciones?

Candidato: En ese caso, si cambiamos de dispositivo perderíamos la clave, lo que implica que perderíamos los mensajes anteriores.

Entrevistador: Y no solo eso, si un mensaje está en camino, también lo perderíamos, ya que está cifrado con la clave pública del dispositivo antiguo. ¿Qué se te ocurre para mitigar este problema?

Candidato: En ese caso, quizá encriptar los mensajes hacia el dispositivo cuando recibimos dichos mensajes. Mantener las claves públicas centralizadas en el servidor podría ser una opción. De esta forma también podríamos recibir mensajes antiguos, ya que en nuestro lado encriptamos los mensajes al leer.

Entrevistador: La última pregunta, que nos quedamos sin tiempo. Te he dicho que tenemos un millón de usuarios, pero eso no es cierto, tenemos casi quinientos millones de usuarios repartidos por todo el mundo, con tres grupos principales. Norteamérica, Europa y Japón son los tres núcleos principales, aunque, obviamente, hay usuarios en todas partes. ¿Cómo adaptamos el diseño, o qué cambiamos para cubrir estos requerimientos? Ten en cuenta que la latencia es muy importante.

Candidato: Deberíamos tener la infraestructura desplegada en múltiples regiones, donde los usuarios se conectan a una de estas regiones. Pero claro, para mandar mensajes de un usuario de una región a un usuario de otra región deberíamos, o bien duplicar la información en todas las regiones, que no sé si tiene mucho sentido, o en nuestra base de datos de la relación mantener un campo adicional a en qué región está ubicado el usuario.

La arquitectura debería cambiar un poco, ya que ahora no insertamos en la base de datos de esa región el 100% de las veces, sino que si está en otra región por lo que necesitaremos un servicio o un proceso para propagar este mensaje. Y luego crearemos otro proceso adicional para cuando un usuario cambia de una región a otra, mover todos sus mensajes.

Conclusión

Llegar a construir una arquitectura eficiente no es una cuestión de seguir un proceso o un patrón mágico, es un aprendizaje constante tomando decisiones basándose en el contexto y las necesidades que tienes en dicho momento.

En este libro hemos recorrido, desde los conceptos básicos de los sistemas distribuidos, explorando patrones, prácticas, herramientas y estrategias que te ayudarán a crear y diseñar sistemas capaces de cumplir con tus necesidades.

Cuando vayas a aplicar cualquier patrón, aplica el sentido común para ver si funciona o no en la arquitectura que quieres montar. Esto se incrementa en empresas donde los empleados quieren hacer microservicios porque es lo que está en tendencia. Pero es verdad que en muchos casos no es necesario, podemos seguir aplicando patrones de arquitecturas distribuidas en un sistema donde el 90% del contenido está en un monolito. Idealmente, debemos migrar a microservicios como una evolución natural del sistema, no imponerlos.

Al final, los sistemas distribuidos no solo impactan en el código. También cambian la forma en que los equipos trabajan, colaboran y gestionan los servicios. Crear un sistema que evolucione junto con la empresa es mucho más importante que seguir cualquier patrón de libro.

Pero no te engañes. El trabajo no termina cuando implementas microservicios. Al contrario, es justo ahí cuando empiezan los verdaderos desafíos. Las aplicaciones cambian, los equipos cambian y lo que hoy tiene sentido, mañana puede no tenerlo. La clave está en pensar siempre en cómo mantener el sistema a largo plazo. Es mejor una solución

sencilla, que puedas escalar y mantener, que una solución compleja que te haga la vida imposible con cada cambio.

La mejor arquitectura no es la más compleja ni la más moderna. La mejor arquitectura es la que aporta valor y permite que el sistema siga vivo y funcionando.

Porque, seamos claros, construir sistemas es fácil. Mantenerlos es lo complicado.

¡Gracias!

Finalmente, gracias por haber leído hasta aquí. Estoy seguro de que el contenido de este libro te será útil en la mejora de tu desarrollo profesional y laboral.

Si te ha gustado el libro, no dudes en dejar una reseña y una valoración con estrellas dentro de Amazon, ya que eso ayuda muchísimo a los creadores pequeños como yo.

Si tienes alguna duda, sugerencia, o te has percatado de un error, por favor, contacta a través del formulario de contacto de la web:



<https://www.netmentor.es/ivan-abad>